

Fundamentos del formato Portable Ejecutable (PE)

- Fundamentos del formato Portable Ejecutable (PE)
- Introducción
- 1. Qué es un PE
- 2. Estructura general de un PE
 - 2.1. IMAGE_DOS_HEADER
 - 2.2. DOS Stub
 - 2.3 IMAGE_NT_HEADERS
 - 2.3.1 Firma PE (PE Signature)
 - 2.3.2. IMAGE_FILE_HEADER
 - 2.3.3. IMAGE_OPTIONAL_HEADER
 - 2.3.3.1. AddressOfEntryPoint
 - 2.3.3.2. ImageBase
 - 2.3.3.3. SectionAlignment
 - 2.3.3.4. FileAlignment
 - 2.3.3.5. SizeOfImage
 - 2.3.3.6. SizeOfHeaders
 - 2.3.3.7. Subsystem
 - 2.3.3.8. DllCharacteristics
 - 2.3.3.9. DataDirectory
 - 2.3.3.9.1. Import Table
 - 2.3.3.9.2. Export Table
 - 2.3.3.9.3. Resource Table
 - 2.3.3.9.4. Base Relocation Table
 - 2.3.3.9.5. TLS Table
 - 2.4. Tabla de secciones del PE
 - 2.4.1. IMAGE_SECTION_HEADER
 - 2.4.2. Name
 - 2.4.3. VirtualSize
 - 2.4.4. VirtualAddress
 - 2.4.5. SizeOfRawData
 - 2.4.6. PointerToRawData
 - 2.4.7. Characteristics
 - 2.4.8. La Section Table contiene descriptores, no los datos de las secciones
- 3. PE en disco frente a PE en memoria
 - 3.1. En disco
 - 3.2. En memoria
- 4. Secciones típicas y señales de interés
- 5. Discrepancias útiles durante el análisis
- 6. Direccionamiento en el Formato PE: RVA, VA y File Offset
- 7. Sistema de importaciones y resolución de API en archivos PE
 - 7.1. Conceptos previos: DLL, exportaciones y Windows API
 - 7.2. Enlace estático y enlace dinámico
 - 7.2.1. Enlace estático

- 7.2.2. Enlace dinámico en tiempo de carga
 - 7.2.3. Enlace dinámico en tiempo de ejecución
 - 7.3. Import Directory e `IMAGE_IMPORT_DESCRIPTOR`
 - 7.4. Import Lookup Table o Import Name Table
 - Importación por nombre
 - Importación por ordinal
 - 7.5. Import Address Table (IAT)
 - Diferencia resumida entre Import Directory, ILT/INT e IAT
 - 7.6. Resolución de importaciones durante la carga
 - 7.7. Resolución dinámica y ocultación de API
 - Pistas para el análisis estático
 - Pistas para el análisis dinámico con x64dbg
 - 7.8. Interpretación de importaciones en análisis de malware
 - Combinaciones relacionadas con process injection
 - 7.9. Inspección de importaciones
 - Herramientas gráficas
 - Herramientas de línea de comandos
 - Enumeración con Python y `pefile`
 - 7.10. Detección y análisis de IAT hooking
 - 7.11. Relación con Manual Mapping y PE Injection
 - 7.12. Metodología práctica de análisis
 - Recursos de consulta
- 8. Relocations
 - 9. Importancia del formato PE para estudiar PE Injection
 - 10. Carga de PE y creación de procesos
 - 10.1. ¿Qué es un proceso?
 - 10.2. Memoria virtual y memoria física
 - 10.3. ¿Qué es un hilo?
 - 10.4. Estructuras TIB, TEB y PEB
 - 10.4.1. TIB
 - 10.4.2. TEB
 - 10.4.3. PEB
 - 10.5. Creación de un proceso paso a paso
 - 10.5.1. Inicio del programa
 - 10.5.2. Creación del proceso y del hilo primario
 - 10.5.3. Inicialización de la memoria virtual
 - 10.5.4. Inicialización del cargador y de la imagen PE
 - 10.5.5. Transferencia a la aplicación
 - 10.6. Carga de un archivo PE paso a paso
 - 10.6.1 Análisis de las cabeceras
 - 10.6.2 Análisis de la tabla de secciones
 - 10.6.3 Mapeo del archivo en memoria
 - 10.6.4 Carga de librerías externas
 - 10.6.5 Aplicación de relocations
 - 10.6.6 Transferencia al punto de entrada
 - 10.7. Procesos WOW64

- 10.8. Importancia de WOW64 en análisis de malware
 - 10.9. Resumen del proceso completo
- 11. Ficheros ejecutables Linux
- 12. EJEMPLO 1: Análisis de la estructura PE de una muestra
- 13. EJEMPLO 2 : Malware que tiene una dll incrustada en la sección .rsrc
- 14. Herramientas para análisis de archivos PE
 - 14.1. pecheck.py
 - 14.2. pe-tree
 - 14.3. peframe
 - 14.4. radare2
 - 14.5. Otras utilidades
 - 14.6. Interpretación de mitigaciones
 - 14.7. Recursos PE
- 15. Conversión de RVA a File Offset en archivos PE
 - 15.1. Conceptos necesarios
 - 15.2. Fórmula de conversión
 - Condición imprescindible
 - 15.3. Procedimiento paso a paso
 - Paso 1. Obtener la RVA de interés
 - Paso 2. Comprobar si pertenece a las cabeceras
 - Paso 3. Localizar la sección candidata
 - Paso 4. Calcular el desplazamiento dentro de la sección
 - Paso 5. Verificar que existen bytes en disco
 - Paso 6. Calcular el File Offset
 - Paso 7. Validar los límites del archivo
 - 15.4. Ejemplos prácticos
 - 15.4.1. Conversión general
 - 15.4.2. Localización del punto de entrada
 - 15.4.3. Localización de la Import Table
 - 15.4.4. RVA sin File Offset equivalente
 - 15.4.5. RVA situada en las cabeceras
 - 15.5. Algoritmo de conversión
 - 15.6. Casos especiales y limitaciones
 - 15.6.1. Datos inicializados a cero
 - 15.6.2. Certificate Table
 - 15.6.3. Overlay
 - 15.6.4. Secciones alineadas o malformadas
 - 15.6.5. ASLR y base real de carga
 - 15.7. Errores frecuentes
 - 15.8. Resumen operativo
 - 15.9. Referencias
- 16. Técnicas que manipulan archivos o imágenes PE
 - 16.1. Alteración del archivo PE
 - 16.2. Manipulación de importaciones y exportaciones
 - 16.3. Manipulación durante la carga o en memoria
 - 16.4. Distinción fundamental

- [16.5. Clasificación resumida](#)
- [17. Laboratorios progresivos](#)

Introducción

- Qué es una imagen PE.
 - Cómo se representa un PE en disco.
 - Cómo Windows organiza una imagen PE en memoria.
 - Qué información contienen sus cabeceras.
 - Cómo se describen y localizan sus secciones.
 - Qué son las importaciones, las relocations y el punto de entrada.
 - Qué artefactos puede dejar una técnica de inyección.
 - Cómo reconocer esos artefactos mediante análisis estático y dinámico.
-

1. Qué es un PE

El **Portable Executable (PE)** es el formato utilizado por Windows para representar ejecutables, bibliotecas y otros módulos que pueden ser cargados por el sistema operativo. Entre los archivos que habitualmente utilizan este formato se encuentran:

- Ejecutables (`.exe`).
- Bibliotecas de enlace dinámico (`.dll`).
- Controladores (`.sys`).
- Otros módulos compatibles con el formato PE/COFF.

El formato PE deriva de **COFF (Common Object File Format)** y contiene, además del código y los datos de la aplicación, una serie de cabeceras, tablas y directorios que describen cómo debe interpretarse y organizarse la imagen en memoria.

Desde el punto de vista del análisis de malware, comprender la estructura PE es fundamental porque técnicas como el empaquetado, la ofuscación, la carga manual, el process hollowing y distintas formas de process injection manipulan o reconstruyen partes de esta estructura.

2. Estructura general de un PE

De forma simplificada, un archivo PE presenta la siguiente organización:

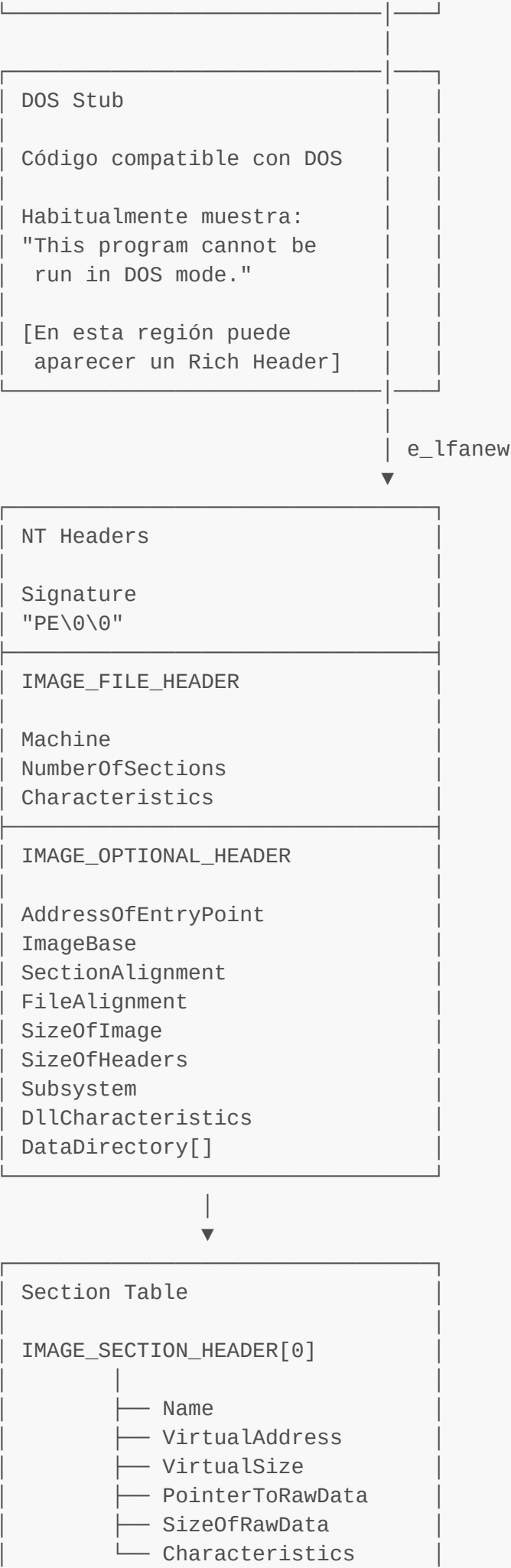
Inicio del archivo



IMAGE_DOS_HEADER

e_magic = "MZ"

e_lfanew



```
IMAGE_SECTION_HEADER[1]  
IMAGE_SECTION_HEADER[2]  
...
```



Datos de las secciones

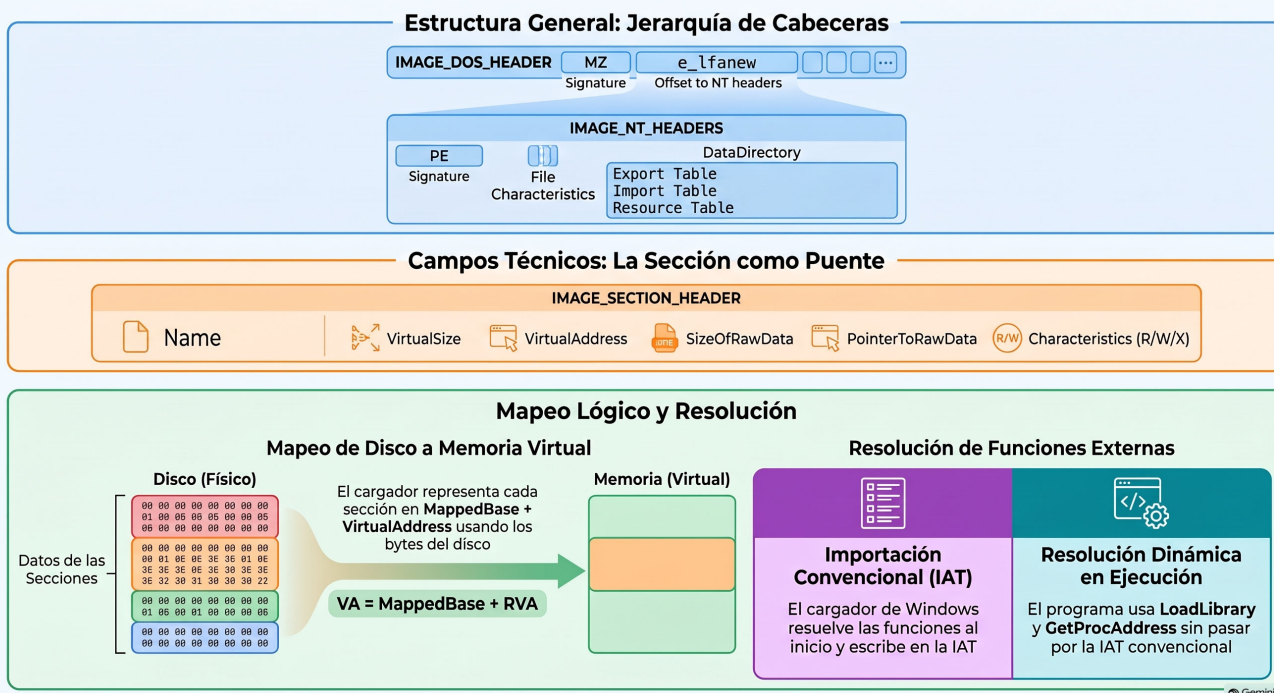
```
.text      (habitual)  
.rdata     (habitual)  
.data      (habitual)  
.idata     (posible)  
.rsrc      (posible)  
.reloc     (posible)  
otras...
```

El formato Portable Ejecutable (PE) es el estándar de Windows para archivos .exe, .dll y .sys. Esta infografía desglosa su arquitectura interna, desde las firmas iniciales de compatibilidad hasta la organización lógica de sus secciones de datos y código.





Anatomía y Estructura del Formato Portable Ejecutable (PE)



Las **cabeceras PE** proporcionan al cargador de Windows la información necesaria para interpretar el archivo y organizarlo en memoria. Entre los datos más relevantes se encuentran:

- **Tipo y características del archivo**: permiten distinguir, entre otros, ejecutables, DLL y controladores.
- **Arquitectura**: indica para qué plataforma se compiló el módulo, por ejemplo x86 o x64.
- **Dirección base preferida**: especifica dónde espera cargarse la imagen.
- **Punto de entrada**: indica la RVA desde la que comienza normalmente la ejecución del módulo.
- **Secciones**: describen áreas de código, datos, recursos, relocations y otra información.
- **Importaciones y dependencias**: identifican bibliotecas y símbolos externos utilizados.
- **Recursos**: pueden contener iconos, cadenas, diálogos, manifiestos y otros datos.
- **Información de seguridad y mitigaciones**: puede incluir características relacionadas con **ASLR**, **DEP/NX** o **Control Flow Guard**.

Examinar esta información es una de las primeras tareas del **análisis estático de un PE**.

2.1. IMAGE_DOS_HEADER

El IMAGE_DOS_HEADER:

El Punto de Partida del Formato PE

Propósito de Compatibilidad

Es la estructura obligatoria de 64 bytes que permite que un binario moderno sea reconocido por entornos legados e indica el inicio de las cabeceras NT.

Campo e_magic (Offset 0x00)

Contiene la firma "MZ" (0x5A4D en bytes 4D 5A), nombrada en honor a Mark Zbikowski; es la "huella digital" necesaria para que Windows considere el archivo un ejecutable.

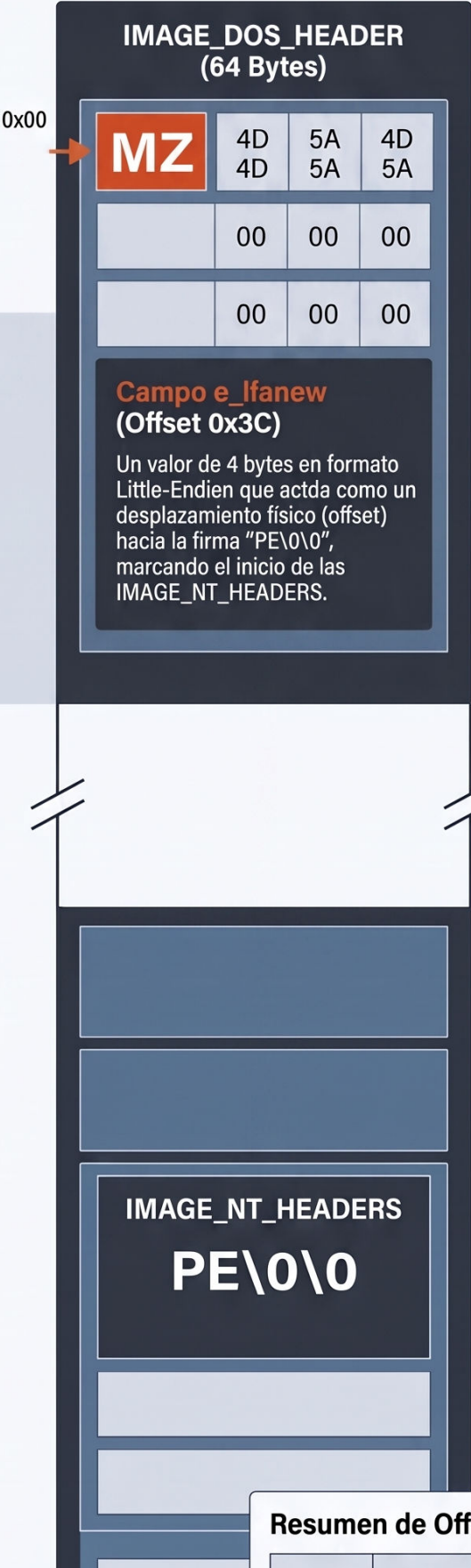
El Entorno Estructural Inmediato

El DOS Stub

Ubicado justo después del encabezado, es código legado que habitualmente muestra el mensaje "This program cannot be run in DOS mode" si se intenta ejecutar en MS-DOS.

El Rich Header (Opcional)

Metadatos insertados por el entorno de compilación MSVC que se sitúan entre el DOS Stub y las NT Headers; es útil para el "fingerprinting" de la herramienta de compilación.



Importancia en Ingeniería Inversa y Malware

Validación de Integridad

La presencia de "MZ" no garantiza un PE válido; un analista debe verificar que e_lfanew apunte a una firma "PE\0\0" coherente para confirmar que la estructura no ha sido manipulada.

Punto de Entrada para Herramientas

Es el punto de partida crítico para parsers y herramientas de análisis estático como PeStudio, PE-bear y radare2 para comenzar el mapeo del binario.

Resumen de Offsets Críticos

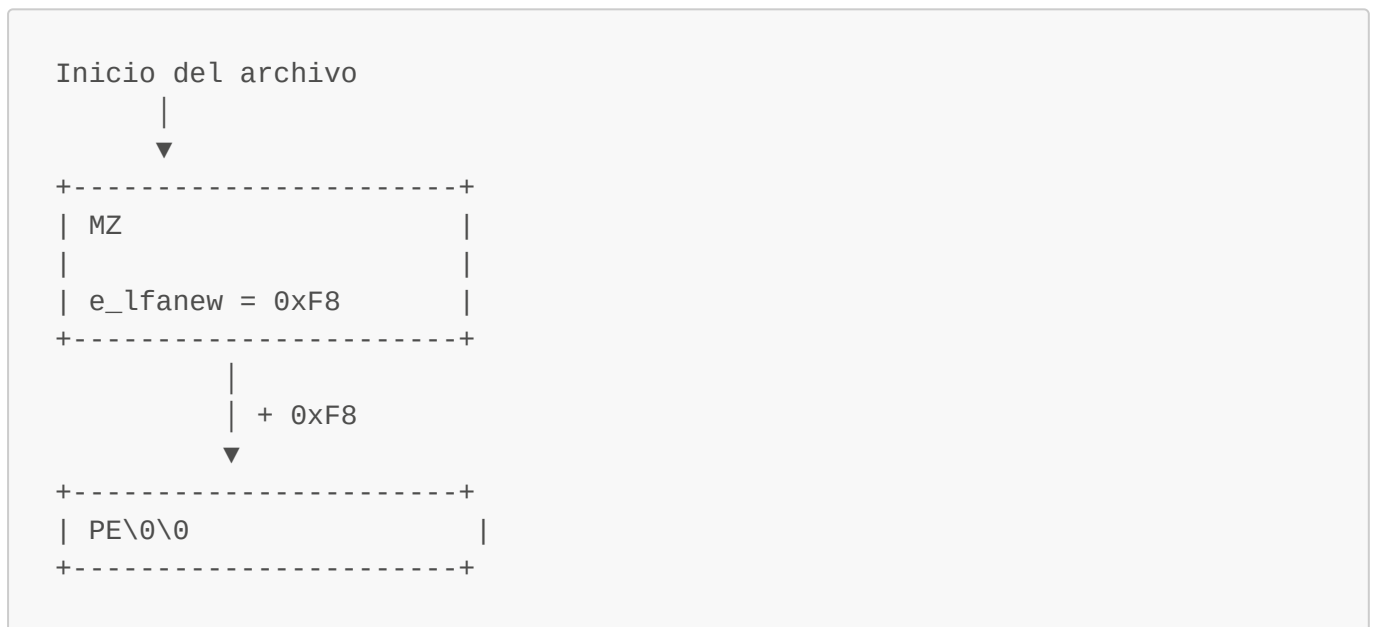
Offset	Campo	Valor típico / Ejemplo	Descripción
0x00	e_magic	4D 5A (MZ)	Firma obligatoria del encabezado DOS.
0x3C	e_lfanew	08 01 00 00	Offset hacia la firma PE (ej. 0x108).
0x40	DOS Stub	Código / Texto	Mensaje de error de compatibilidad.

Gemini Notebook

Un archivo **PE** comienza normalmente con una estructura **IMAGE_DOS_HEADER**. Dos campos son especialmente importantes:

- **e_magic**: contiene la firma **MZ**.
- **e_lfanew**: contiene el desplazamiento desde el inicio del archivo hasta la firma de las **cabeceras NT** (**PE\0\0**).

De forma conceptual:



La firma **MZ** corresponde a los bytes:

4D 5A

El nombre procede de **Mark Zbikowski**, uno de los desarrolladores del formato ejecutable de DOS. La presencia de **MZ** por sí sola **no demuestra que el archivo sea un PE válido**: también deben ser coherentes **e_lfanew**, la firma **PE\0\0** y las estructuras posteriores.

El campo **e_lfanew** se encuentra en el offset **0x3C** del **IMAGE_DOS_HEADER**. Por ejemplo:

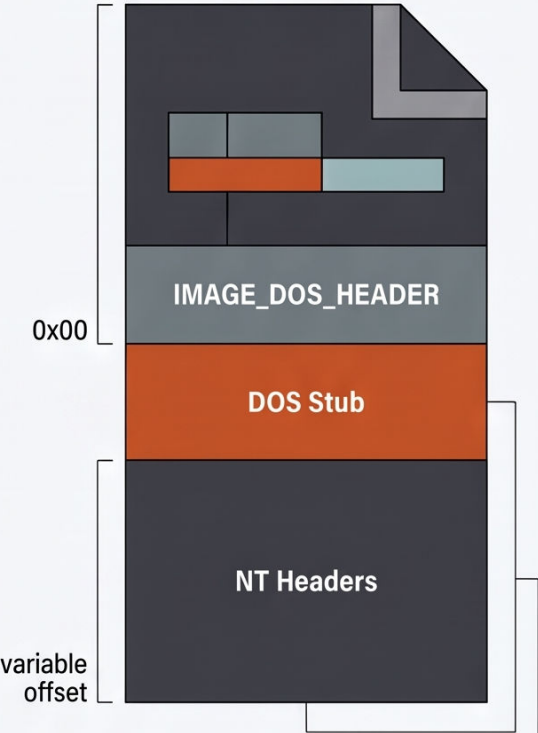
```
Offset 0x00: 4D 5A      -> MZ
Offset 0x3C: 08 01 00 00 -> e_lfanew = 0x108
Offset 0x108: 50 45 00 00 -> PE\0\0
```

Para localizar las **cabeceras NT**, el cargador toma el valor de **e_lfanew** y lo interpreta como un desplazamiento desde el inicio del archivo.

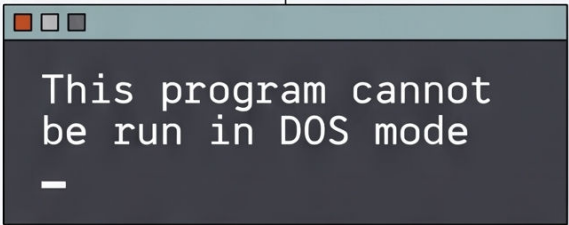
2.2. DOS Stub

EL DOS STUB: EL LEGADO DE 16 BITS EN EL FORMATO PE

ANATOMÍA Y PROPÓSITO HISTÓRICO



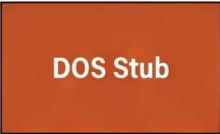
Ubicación y Estructura: Se sitúa en el offset 0x40, inmediatamente después del IMAGE_DOS_HEADER y antes de las cabeceras NT.



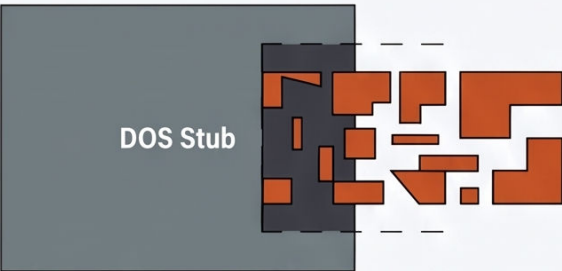
Compatibilidad MS-DOS: Pequeño programa de 16 bits que imprime el mensaje en entornos legados.



COMPORTAMIENTO MODERNO Y SEGURIDAD



Omisión por el PE Loader: El cargador de Windows moderno lo ignora, saltando directamente a la firma PE mediante el campo e_lfanew.



Refugio de Malware: Al ser una zona inactiva, es utilizada por atacantes para ocultar payloads, shellcode o marcas de agua.



Huella Digital (Fingerprinting): Esencial en análisis estático para identificar manipulaciones o metadatos del compilador (como el Rich Header).

RESUMEN DE LOS CAMPOS CRÍTICOS ADYACENTES

ELEMENTO	UBICACIÓN	FUNCIÓN
----------	-----------	---------

int 21h

0D0D0A24

La Secuencia de Interrupción: Utiliza la interrupción 'int 21h' y termina con '\$' como delimitador de cadona.

	(OFFSET)	CRITICA
IMAGE_DOS_HEADER	0x00	Contiene la firma M? inicial
e_lfanew	0x3C	Dirección de salto para ignorar el Stub
PE Signature	Variable (e_lfanew)	Marca el inicio real de las NT Headers

Gemini Notebook

Después del **IMAGE_DOS_HEADER** suele encontrarse un pequeño programa denominado **DOS Stub**, situado antes de las **cabeceras NT**.

En numerosos ejecutables su mensaje característico es:

```
This program cannot be run in DOS mode.
```

Históricamente, este código permitía proporcionar un comportamiento mínimo si el archivo se ejecutaba bajo DOS.

En la región comprendida entre el **DOS Header** y las **cabeceras NT** también puede aparecer un **Rich Header**, especialmente en binarios generados con herramientas de Microsoft. El **Rich Header** no forma parte obligatoria de la especificación **PE**.

Desde el punto de vista del análisis de malware, esta zona puede ser modificada o reutilizada para almacenar datos, por lo que conviene examinarla cuando existan inconsistencias.

2.3 IMAGE_NT_HEADERS

Anatomía de IMAGE_NT_HEADERS: El Núcleo del Formato PE

Desglosar los componentes y campos críticos de las cabeceras NT para entender cómo Windows configura y carga un archivo ejecutable.

COMPONENTES INICIALES Y ARCHIVO (COFF)

Firma PE y Ubicación Física

0x50450000

Identificada por PE\O\0, es apuntada desde el inicio por el campo e_lfanew.

IMAGE_FILE_HEADER (Cabecera COFF)

Machine

Especifica la arquitectura

Campo	Valor Comilo	Significado
Machine	0x14C / 0x2084	Arquitectura x86 / x64
Magic	0xD10B / 0x2089	Entorno PE32 / PE32+
Subsystem	2 / 8	Entorno GUI / Console

Características y Tamaño

Define si el archivo es un .exe o .dll mediante flags en el campo Characteristics.

Marca de tiempo de compilación

IMAGE_OPTIONAL_HEADER (Configuración de Carga)

Control de Ejecución y Memoria

Memory Map

AddressOfEntryPoint (RVA de inicio)

ImageBase (dirección virtual preferida)

Mitigaciones de Seguridad

DllCharacteristics

El campo DllCharacteristics activa protecciones críticas como ASLR, DEP/NX y Control Flow Guard.

ASLR

Aleatoriza direcciones de memoria clave.

DEP/NX

Previene la ejecución de código en regiones de datos.

CFG

Verifica la integridad del flujo de control.

DataDirectory: El Mapa Lógico

Arreglo de 16 entradas que ubican tablas clave

Import Table

Export Table

Resource Table

Relocation Table

Gemini Notebook

13 / 95

2.3.1 Firma PE (PE Signature)

En la posición indicada por `e_lfanew` debe encontrarse la firma:

```
50 45 00 00
```

que corresponde a:

```
PE\0\0
```

Esta firma marca el comienzo de las **cabeceras NT**. Su presencia aislada tampoco garantiza que todo el archivo sea válido; las estructuras y tamaños posteriores deben ser coherentes.

2.3.2. IMAGE_FILE_HEADER

Después de la firma `PE\0\0` se encuentra el **IMAGE_FILE_HEADER**, también llamado **COFF File Header**.

Entre sus campos se encuentran:

```
Machine  
NumberOfSections  
TimeDateStamp  
SizeOfOptionalHeader  
Characteristics
```

Algunos campos de especial interés son:

- **Machine**: identifica la arquitectura.
- **NumberOfSections**: indica cuántas entradas contiene la tabla de secciones.
- **SizeOfOptionalHeader**: especifica el tamaño del **Optional Header**.
- **Characteristics**: contiene flags generales del archivo.

Valores habituales del campo **Machine**:

```
0x014C -> x86  
0x8664 -> x86-64
```

2.3.3. IMAGE_OPTIONAL_HEADER

A pesar de su nombre, el **Optional Header es esencial para una imagen ejecutable PE**. Contiene gran parte de la información que utiliza el cargador para organizar el módulo en memoria.

Entre los campos más relevantes, y en el orden relativo en que aparecen dentro de la estructura, se encuentran:

```
AddressOfEntryPoint
ImageBase
SectionAlignment
FileAlignment
SizeOfImage
SizeOfHeaders
Subsystem
DllCharacteristics
DataDirectory[]
```

Nota: existen otros campos intermedios. Además, `IMAGE_OPTIONAL_HEADER32` y `IMAGE_OPTIONAL_HEADER64` presentan algunas diferencias estructurales.

2.3.3.1. AddressOfEntryPoint

`AddressOfEntryPoint` contiene la **RVA (Relative Virtual Address)** del punto de entrada del módulo.

No es una dirección virtual absoluta. Para obtener la dirección virtual correspondiente se utiliza:

$$VA = ImageBase + RVA$$

Por ejemplo:

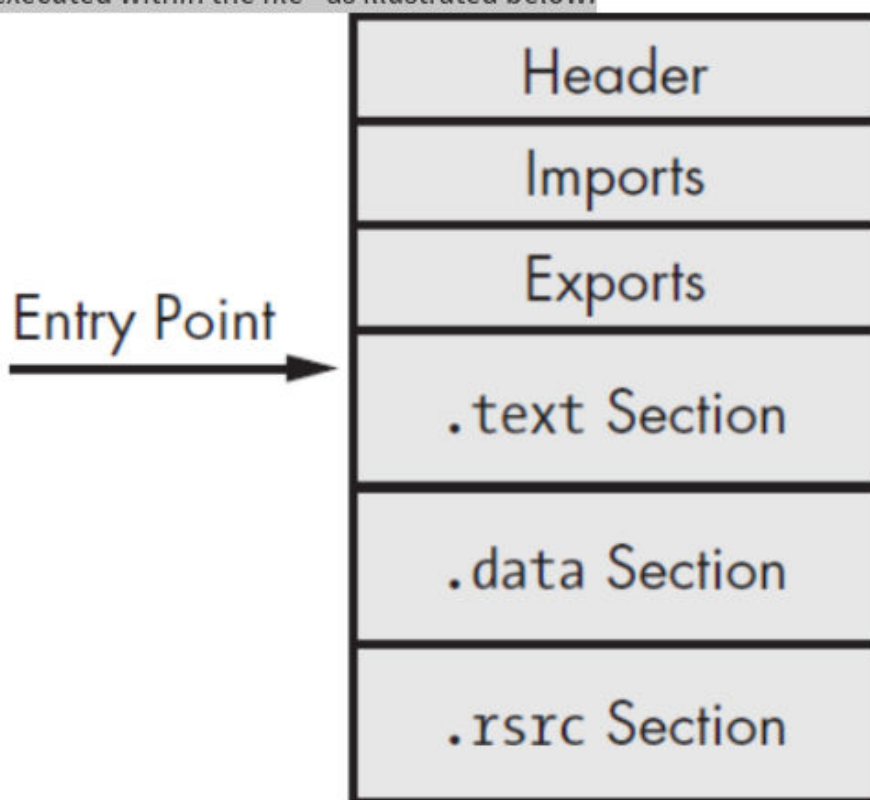
```
ImageBase          = 0x140000000
AddressOfEntryPoint = 0x00001230

VA = 0x140000000 + 0x1230
VA = 0x140001230
```

[!Important] Sólo puede emplearse `ImageBase + RVA` cuando el archivo se carga en su dirección preferida. `ImageBase` expresa una preferencia, no garantiza la dirección real. La fórmula general debe ser: `VA = MappedBase + RVA`

En análisis de malware suele hablarse de **Entry Point (EP)**. El término **Original Entry Point (OEP)** se utiliza especialmente al analizar binarios empaquetados para referirse al punto de entrada original del programa antes del empaquetado; por tanto, no debe utilizarse como sinónimo automático de `AddressOfEntryPoint`.

El punto de entrada resulta especialmente relevante durante el análisis dinámico porque representa la RVA desde la que comienza normalmente la ejecución del módulo una vez completadas las tareas de carga necesarias.



2.3.3.2. ImageBase

ImageBase indica la dirección virtual **preferida** en la que la imagen espera ser cargada.

Si esa dirección no está disponible, el sistema puede cargarla en otra posición siempre que el módulo disponga de la información necesaria para corregir las referencias dependientes de la base. En ese caso intervienen las **base relocations**.

2.3.3.3. SectionAlignment

SectionAlignment indica la alineación de las secciones cuando la imagen se encuentra en memoria.

Un valor muy habitual es:

```
SectionAlignment = 0x1000
```

pero no debe asumirse que siempre será ese valor.

2.3.3.4. FileAlignment

FileAlignment especifica la alineación utilizada para los datos de las secciones en el archivo.

Un valor frecuente es:

```
FileAlignment = 0x200
```

Esto explica por qué muchos `PointerToRawData` y `SizeOfRawData` aparecen alineados a múltiplos de `0x200`.

2.3.3.5. SizeOfImage

`SizeOfImage` representa el tamaño total que ocupará la imagen PE una vez organizada en memoria, respetando `SectionAlignment`.

Puede ser diferente del tamaño físico del archivo:

```
Tamaño del archivo en disco = 45 KB
SizeOfImage                  = 80 KB
```

Entre los factores que explican esta diferencia se encuentran:

- Alineamiento de secciones.
- Diferencias entre `VirtualSize` y `SizeOfRawData`.
- Datos que se inicializan a cero.
- Espacios de alineamiento entre regiones.

Este campo es especialmente importante al estudiar **manual mapping** o **PE Injection**, ya que permite conocer el espacio virtual necesario para reconstruir la imagen.

2.3.3.6. SizeOfHeaders

`SizeOfHeaders` indica el tamaño combinado y alineado de las cabeceras del PE, incluida la tabla de secciones.

Es relevante durante un mapeo manual porque la parte inicial de la imagen también debe quedar disponible en memoria.

2.3.3.7. Subsystem

`Subsystem` especifica el entorno esperado por el ejecutable. Dos valores frecuentes son:

```
2 -> Windows GUI
3 -> Windows Console
```

Este campo ayuda a determinar cómo espera interactuar el programa con el entorno, pero no describe por sí solo todo su comportamiento.

2.3.3.8. DllCharacteristics

`DllCharacteristics` contiene flags relacionados con características de carga y mitigaciones de seguridad.

Entre los más conocidos se encuentran:

```
IMAGE_DLLCHARACTERISTICS_DYNAMIC_BASE
IMAGE_DLLCHARACTERISTICS_NX_COMPAT
IMAGE_DLLCHARACTERISTICS_GUARD_CF
```

Estos flags pueden indicar compatibilidad con mecanismos como **ASLR**, **DEP/NX** o **Control Flow Guard**.

2.3.3.9. DataDirectory

El **Optional Header** contiene un array de **Data Directories**. Cada entrada describe la ubicación y el tamaño de una estructura lógica del **PE**.

Entre las entradas más importantes se encuentran:

```
Export Table
Import Table
Resource Table
Exception Table
Certificate Table
Base Relocation Table
Debug Directory
TLS Table
IAT
Delay Import Table
CLR Header
```

La mayoría de estas entradas se expresan mediante una pareja:

```
VirtualAddress / RVA
Size
```

La **Certificate Table** es una excepción importante: su campo de dirección se interpreta como un **offset** de archivo y no como una **RVA**.

2.3.3.9.1. Import Table

La Import Table describe bibliotecas y símbolos externos utilizados por el programa.

Ejemplo:

```
KERNEL32.dll
  CreateFileA
  WriteFile
  CreateProcessA

ADVAPI32.dll
```

```
RegSetValueExA
```

```
WS2_32.dll  
socket  
connect  
send  
recv
```

Estas importaciones pueden ayudar a formular hipótesis sobre el comportamiento del binario:

CreateFile / WriteFile	-> operaciones con archivos
RegSetValueEx	-> posible modificación del Registro
socket / connect	-> funcionalidad de red

Una combinación como:

```
VirtualAllocEx  
WriteProcessMemory  
CreateRemoteThread
```

puede ser compatible con técnicas de **process injection**, pero **las importaciones por sí solas no demuestran que la técnica se ejecute realmente**. Es necesario confirmar el flujo de código y los parámetros utilizados.

2.3.3.9.2. Export Table

La Export Table resulta especialmente relevante en **DLL** y otros módulos que exponen funciones.

Los nombres exportados pueden orientar el análisis, aunque tampoco constituyen por sí mismos evidencia de comportamiento malicioso.

2.3.3.9.3. Resource Table

La Resource Table puede contener:

- Iconos.
- Imágenes.
- Cadenas.
- Diálogos.
- Manifiestos.
- Información de versión.
- Datos binarios arbitrarios.

En muestras de malware también puede utilizarse para almacenar configuraciones, ejecutables embebidos u otros datos codificados o comprimidos.

2.3.3.9.4. Base Relocation Table

La Base Relocation Table contiene información utilizada para corregir determinadas referencias cuando una imagen se carga en una dirección distinta de su **ImageBase** preferido.

Habitualmente estos datos pueden encontrarse asociados a una sección denominada:

```
.reloc
```

El nombre de la sección, sin embargo, no es obligatorio.

2.3.3.9.5. TLS Table

La TLS Table puede incluir **TLS callbacks**. Estos **callbacks** pueden ejecutarse durante determinados eventos de carga antes de que se alcance el punto de entrada principal del módulo.

En análisis de malware son relevantes porque **pueden utilizarse para ejecutar código de inicialización o técnicas anti-análisis antes del Entry Point observado por el analista.**

2.4. Tabla de secciones del PE

Anatomía de la Tabla de Secciones PE: El Mapa de Memoria de Windows

Una guía visual de los descriptores de sección y cómo el cargador de Windows mapea archivos del disco a la memoria virtual.

ESTRUCTURA DE IMAGE_SECTION_HEADER



IMAGE_SECTION_HEADER

Identidad y Permisos
(Name & Characteristics)



Ubicación: Memoria (RVA) | VirtualAddress define inicio en RAM.



Ubicación: Disco (Pointer) | PointerToRawData indica posición en archivo.

VirtualSize: Tamaño lógico en memoria

SizeOfRawData: Tamaño real en disco

Diferencia entre tamaño lógico y físico.

MAPEO Y SECCIONES COMUNES

Conversión de RVA a File Offset

$\text{FileOffset} = \text{PointerToRawData} + (\text{RVA} - \text{VirtualAddress})$

La Discrepancia de Alineación



FileAlignment

Datos alineados en disco.

Disco (Alignment)



SectionAlignment

Datos alineados en memoria.

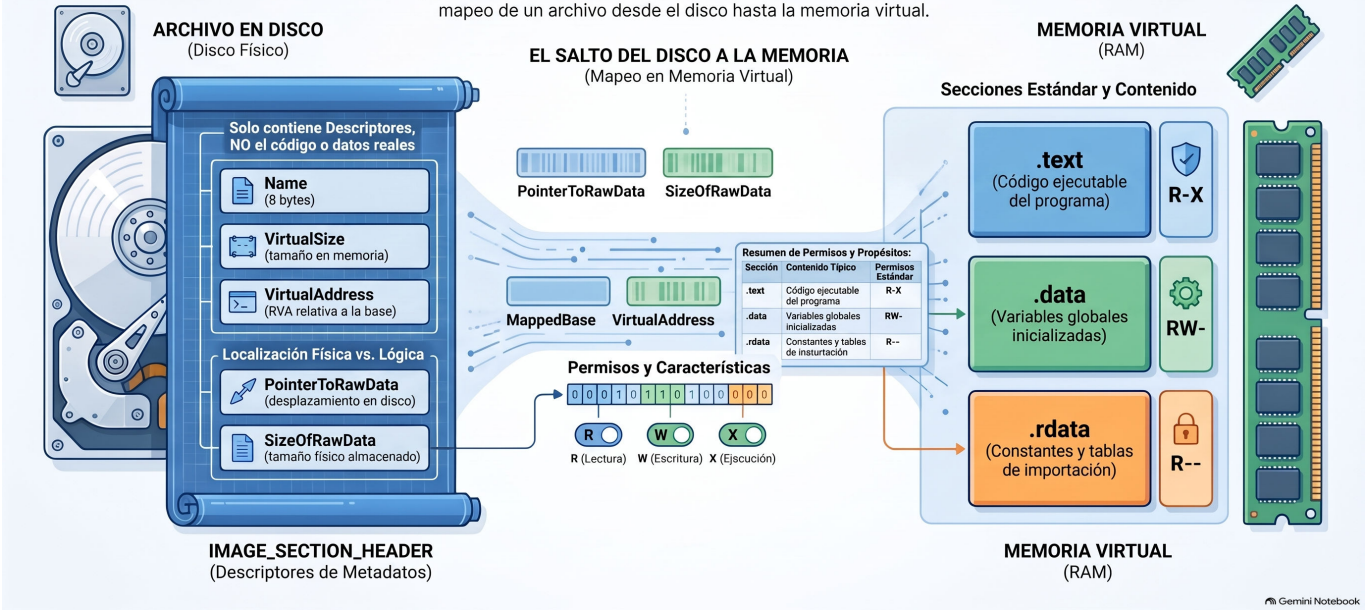
Memoria (Alignment)

Resumen de Secciones Estándar		
Sección	Contenido Típico	Permisos (Atributos)
 .text	Código ejecutable del programa	Lectura y Ejecución (R-X)
 .rdata	Datos de solo lectura e Importaciones (IAT)	Solo Lectura (R--)
 .data	Variables globales inicializadas	Lectura y Escritura (RW-)

20 / 95

La Tabla de Secciones del Formato PE: El Mapa del Ejecutable

Explica la estructura técnica de la Tabla de Secciones y cómo define el mapeo de un archivo desde el disco hasta la memoria virtual.



El contenido de un **PE** se organiza habitualmente en diferentes **secciones**. Cada sección está descrita mediante un **IMAGE_SECTION_HEADER**.

Algunas secciones frecuentes son:

Sección	Contenido habitual
.text	Código ejecutable
.rdata	Datos de solo lectura, constantes y tablas
.data	Variables globales inicializadas
.bss	Datos no inicializados. Puede no aparecer como sección independiente
.idata	Información relacionada con importaciones
.edata	Información relacionada con exportaciones
.rsrc	Recursos
.reloc	Base relocations
.pdata	Información de excepciones, especialmente habitual en x64

Los nombres de sección son principalmente una **convención**. El cargador no depende de que una sección se llame **.text**, **.data** o **.rdata**; utiliza los campos de sus cabeceras y características.

Por este motivo, un binario empaquetado u ofuscado puede utilizar nombres diferentes.

2.4.1. IMAGE_SECTION_HEADER

Cada sección está descrita mediante una estructura **IMAGE_SECTION_HEADER**.

Algunos campos relevantes son:

```
Name  
VirtualSize  
VirtualAddress  
SizeOfRawData  
PointerToRawData  
Characteristics
```

2.4.2. Name

Identifica el nombre convencional de la sección.

Ejemplos habituales:

```
.text  
.rdata  
.data  
.idata  
.rsrc  
.reloc
```

Algunos packers pueden utilizar nombres reconocibles, por ejemplo:

```
UPX0  
UPX1  
.aspack  
.mpress
```

Un nombre inusual puede justificar un análisis adicional, pero **no demuestra por sí mismo que el archivo sea malicioso**.

2.4.3. VirtualSize

Indica el tamaño lógico solicitado para la sección dentro de la imagen en memoria.

2.4.4. VirtualAddress

Contiene la **RVA** en la que comienza la sección dentro de la imagen.

Por ejemplo:

```
VirtualAddress = 0x1000  
ImageBase      = 0x00400000
```

La dirección virtual correspondiente sería:

```
0x00400000 + 0x1000 = 0x00401000
```

2.4.5. SizeOfRawData

Indica el tamaño de los datos de la sección almacenados físicamente en el archivo, normalmente alineado según `FileAlignment`.

2.4.6. PointerToRawData

Indica el offset del archivo donde comienzan los datos físicos de la sección.

Por ejemplo:

```
PointerToRawData = 0x400
```

significa que los bytes almacenados de la sección comienzan en el offset `0x400` del archivo.

2.4.7. Characteristics

Contiene `flags` que describen propiedades de la sección, entre ellas sus permisos previstos.

De forma conceptual:

```
R -> Read  
W -> Write  
X -> Execute
```

Configuraciones habituales:

```
.text -> R-X  
.data -> RW-
```

Una sección o región con permisos `RWX` puede ser relevante durante el análisis porque permite lectura, escritura y ejecución simultáneamente, aunque **no constituye por sí sola una prueba de actividad maliciosa**.

2.4.8. La Section Table contiene descriptores, no los datos de las secciones

La **Section Table** está formada por estructuras `IMAGE_SECTION_HEADER`. Estas estructuras describen dónde se encuentran los datos y cómo deben organizarse.

Por ejemplo:

IMAGE_SECTION_HEADER



.text

VirtualAddress
VirtualSize
PointerToRawData
SizeOfRawData
Characteristics

PointerToRawData permite localizar los bytes de la sección en el archivo:

Section Table

.text

PointerToRawData = 0x400



File Offset 0x400

bytes reales de .text

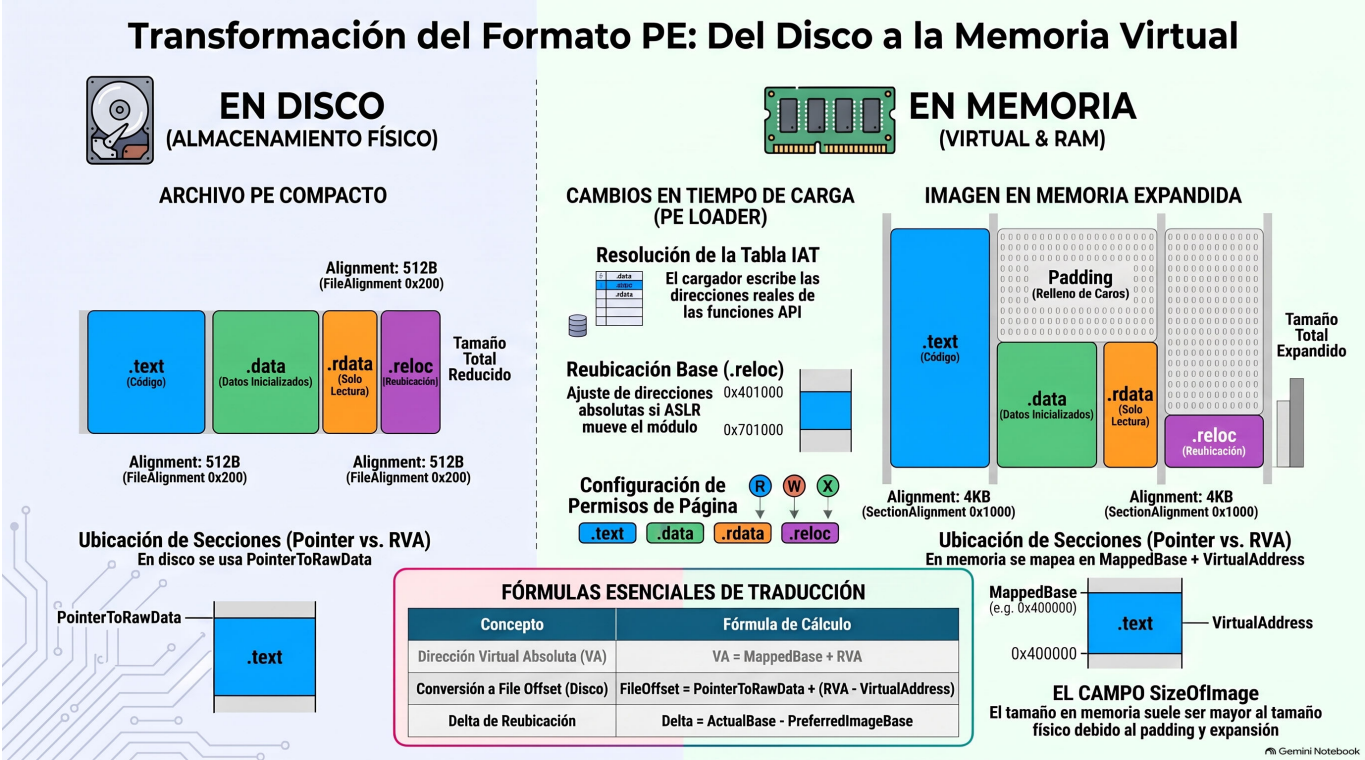
48 89 5C 24 ...

En memoria, la sección se localiza mediante:

MappedBase + VirtualAddress

Esta diferencia es fundamental para entender el mapeo de una imagen PE.

3. PE en disco frente a PE en memoria



Un PE no presenta exactamente la misma disposición en disco que una vez mapeado como imagen en memoria.

3.1. En disco

Los campos más relevantes son:

```
PointerToRawData
SizeOfRawData
```

3.2. En memoria

La posición de una sección se expresa mediante:

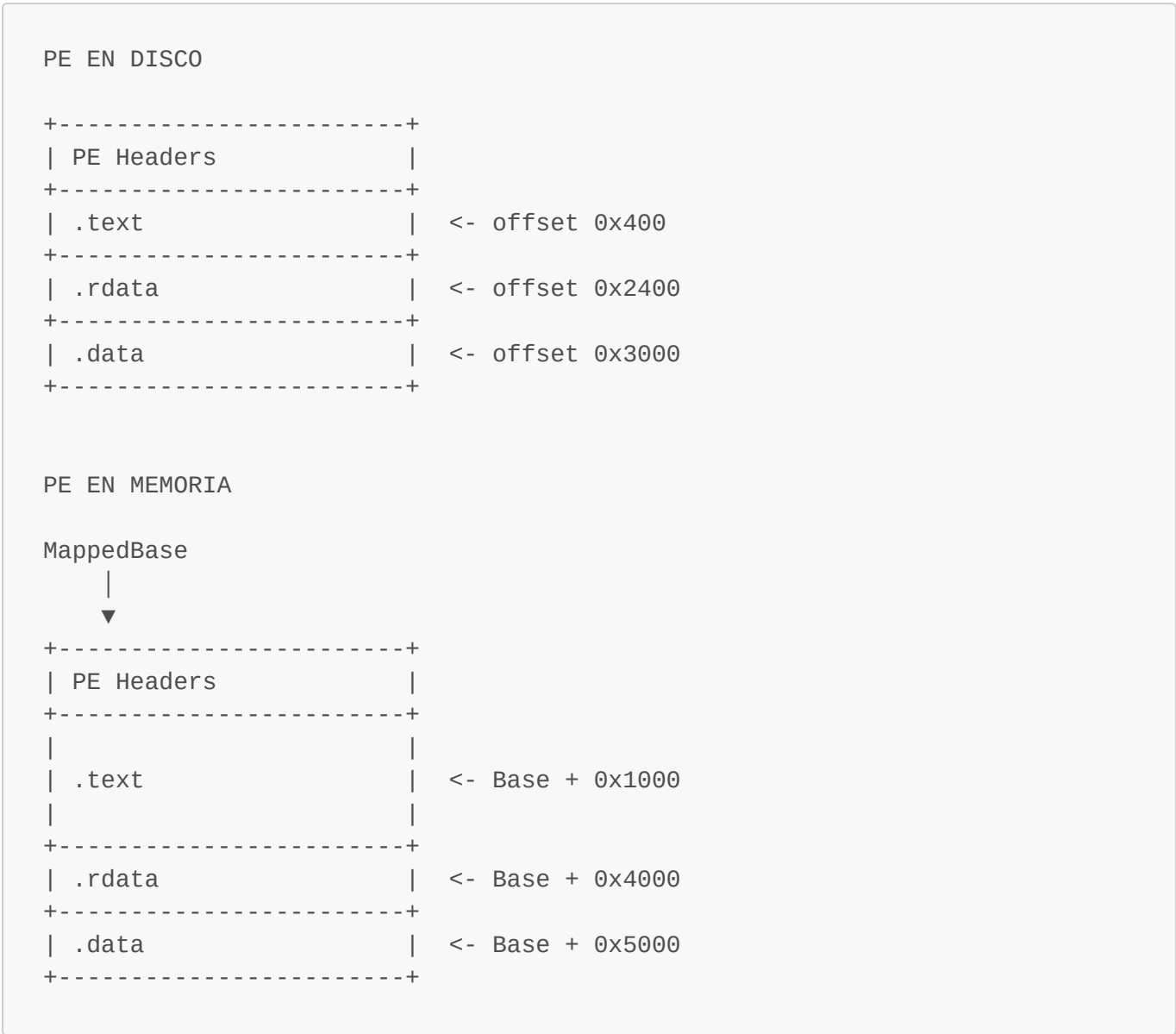
```
MappedBase + VirtualAddress
```

Por ejemplo:

DISCO		MEMORIA
.text		
PointerToRawData = 0x400	->	Base + RVA 0x1000
.rdata		
PointerToRawData = 0x2400	->	Base + RVA 0x4000

```
.data
PointerToRawData = 0x3000    ->    Base + RVA 0x5000
```

Visualmente:



Por ello, copiar un PE como un bloque continuo de bytes no equivale a reproducir su imagen cargada. El mapeo debe respetar la información contenida en las cabeceras y en la tabla de secciones.

4. Secciones típicas y señales de interés

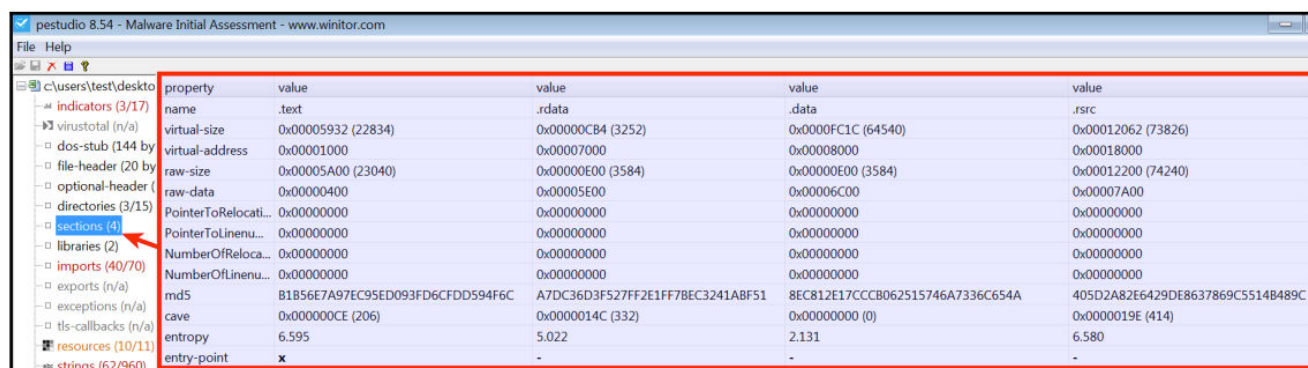
Sección	Función habitual	Observaciones de análisis
.text	Código ejecutable	Lo habitual es R-X; código fuera de secciones ejecutables merece revisión
.data	Variables globales inicializadas	Suele ser RW-; puede contener configuración o datos de estado

Sección	Función habitual	Observaciones de análisis
<code>.bss</code>	Datos no inicializados	Puede corresponder a espacio reservado y zero-filled en memoria
<code>.rdata</code>	Datos de solo lectura	Puede contener strings, constantes, RTTI y tablas
<code>.idata</code>	Imports	Útil para estudiar dependencias y APIs
<code>.edata</code>	Exports	Especialmente relevante en DLL
<code>.rsrc</code>	Recursos	Puede contener iconos, cadenas y datos embebidos
<code>.reloc</code>	Base relocations	Permite corregir referencias cuando cambia la base de carga
<code>UPX0</code> , <code>UPX1</code>	Nombres frecuentes en binarios UPX	Indican posible empaquetado, no necesariamente malware

Los nombres de sección no deben utilizarse como criterio aislado de clasificación.

Cuando una herramienta como **PeStudio** muestra la tabla de secciones, algunos campos habituales son:

- **Name:** nombre de la sección.
- **Virtual Size:** tamaño lógico de la sección en memoria.
- **Virtual Address:** RVA donde comienza la sección.
- **Raw Size:** tamaño almacenado en disco.
- **Raw Data / Raw Offset:** desplazamiento del archivo donde comienzan los datos.
- **Entry Point:** puede indicar en qué sección cae la RVA del punto de entrada.



5. Discrepancias útiles durante el análisis

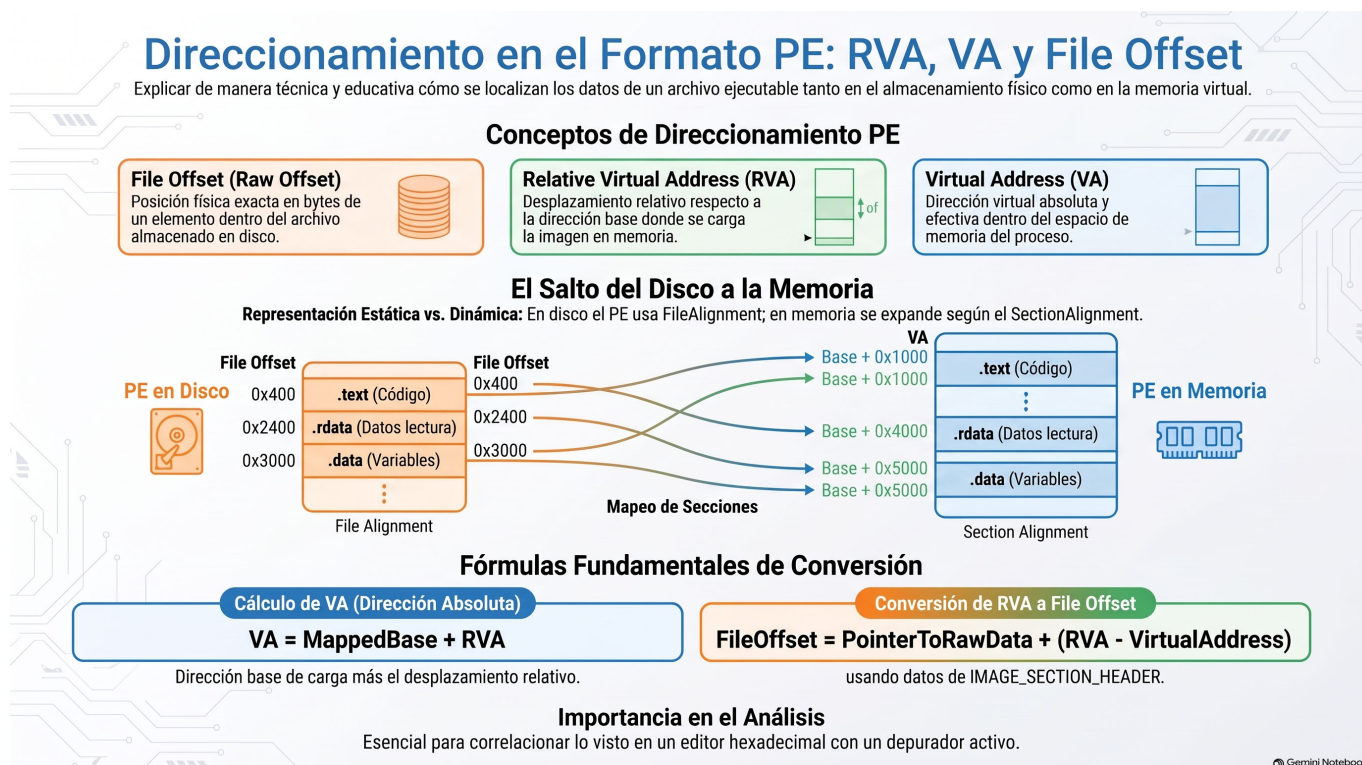
Algunas características pueden justificar una revisión más detallada:

- Nombres como `UPX0` y `UPX1` pueden indicar que el binario está empaquetado con UPX.
- Un **Entry Point** situado en una sección asociada al **packer** suele conducir primero a la rutina de desempaquetado.
- Diferencias entre **VirtualSize** y **SizeOfRawData** son normales hasta cierto punto debido al alineamiento y a los datos no inicializados.

- Una diferencia muy grande —por ejemplo, una sección con `SizeOfRawData = 0` pero un `VirtualSize` considerable— puede indicar que se reserva espacio que será rellenado durante la ejecución. En un binario empaquetado puede utilizarse para alojar código o datos desempaquetados.

Estas observaciones son **indicadores, no pruebas concluyentes de comportamiento malicioso**.

6. Direccionamiento en el Formato PE: RVA, VA y File Offset



Para estudiar correctamente un **PE** es necesario diferenciar tres conceptos:

- File Offset**: posición de un elemento dentro del archivo almacenado en disco.
- RVA (Relative Virtual Address)**: desplazamiento relativo respecto a la base de carga de la imagen.
- VA (Virtual Address)**: dirección virtual efectiva dentro del espacio de memoria del proceso.

La relación básica es:

$$VA = \text{MappedBase} + RVA$$

Cuando la imagen se carga exactamente en su dirección preferida:

$$VA = \text{ImageBase} + RVA$$

Ejemplo:

```
ImageBase = 0x140000000  
RVA       = 0x00001230  
  
VA        = 0x140001230
```

Para convertir una RVA perteneciente a una sección en un offset de archivo puede utilizarse:

```
FileOffset =  
    PointerToRawData +  
    (RVA - VirtualAddress)
```

Ejemplo:

```
RVA buscada:      0x1234  
  
Sección .text:  
VirtualAddress:   0x1000  
PointerToRawData: 0x400
```

Cálculo:

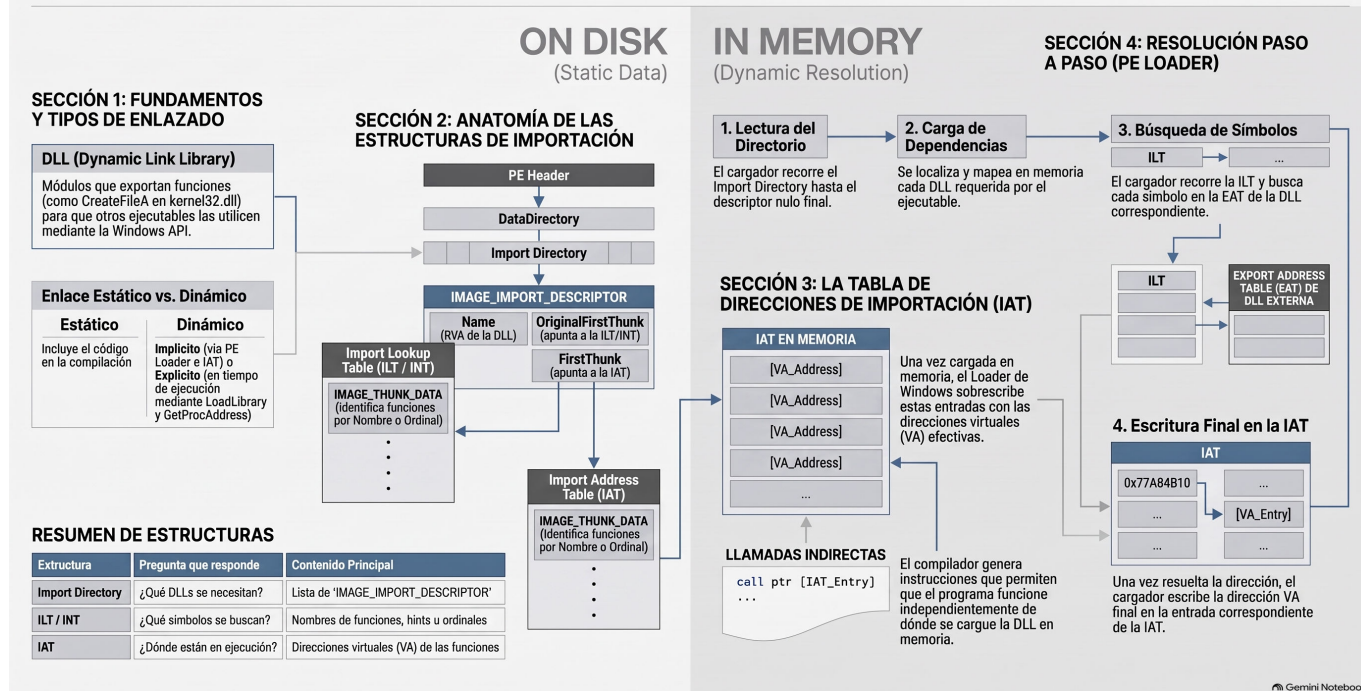
```
FileOffset = 0x400 + (0x1234 - 0x1000)  
FileOffset = 0x634
```

FileOffset, **Raw Offset** y el desplazamiento físico dentro del archivo se utilizan aquí con el mismo sentido.

Comprender estas conversiones es esencial para correlacionar lo observado en herramientas de análisis estático con las direcciones mostradas posteriormente por un debugger como x64dbg.

7. Sistema de importaciones y resolución de API en archivos PE

Importaciones e IAT: El Mecanismo de Dependencias del Formato PE



Un módulo PE —habitualmente una DLL, aunque también puede ser un ejecutable— puede exportar funciones o datos para que otros módulos los utilicen. El módulo proveedor describe estos símbolos mediante su directorio de exportaciones, mientras que el módulo consumidor declara sus dependencias mediante las estructuras de importación.

Las aplicaciones de Windows utilizan con frecuencia código proporcionado por bibliotecas externas. En un archivo **Portable Executable (PE)**, la información necesaria para localizar esas bibliotecas y sus funciones se organiza principalmente mediante:

- El **Import Directory**.
- La **Import Lookup Table**, también denominada habitualmente **Import Name Table**.
- La **Import Address Table (IAT)**.

El estudio de estas estructuras permite:

- conocer las dependencias externas de un binario;
- formular hipótesis iniciales sobre sus capacidades;
- localizar en el código las llamadas a API importadas;
- reconocer resolución dinámica u ocultación de API;
- entender qué trabajo debe reproducir un cargador manual al mapear una imagen PE.

[!IMPORTANT] Una importación es un **indicio**, no una prueba de comportamiento malicioso. La conclusión debe apoyarse en las referencias cruzadas, los argumentos utilizados, el flujo de ejecución y, cuando sea posible, el análisis dinámico.

7.1. Conceptos previos: DLL, exportaciones y Windows API

Una **DLL** es un módulo PE que puede exportar funciones o datos para que otros módulos los utilicen.

La **tabla de exportaciones** de una DLL describe los símbolos que ofrece.

La **tabla de importaciones** de un ejecutable o de otra DLL indica qué símbolos externos necesita.

El término **Windows API** designa el conjunto de interfaces que Windows expone a las aplicaciones. Muchas de sus funciones se ofrecen desde DLL del sistema, por ejemplo:

DLL	Ámbito habitual	Ejemplos
<code>kernel32.dll</code> / <code>KernelBase.dll</code>	Procesos, hilos, memoria, archivos y carga de módulos	<code>CreateProcessW</code> , <code>VirtualAlloc</code> , <code>CreateFileW</code> , <code>LoadLibraryW</code>
<code>ntdll.dll</code>	API nativa, loader y funciones de bajo nivel	<code>NtWriteVirtualMemory</code> , <code>NtProtectVirtualMemory</code> , <code>LdrLoadDll</code>
<code>advapi32.dll</code>	Registro, servicios, tokens y seguridad	<code>RegSetValueExW</code> , <code>CreateServiceW</code> , <code>AdjustTokenPrivileges</code>
<code>user32.dll</code>	Ventanas, teclado y ratón	<code>FindWindowW</code> , <code>GetAsyncKeyState</code> , <code>SetWindowsHookExW</code>
<code>ws2_32.dll</code>	Sockets	<code>socket</code> , <code>connect</code> , <code>send</code> , <code>recv</code>
<code>wininet.dll</code>	Comunicaciones HTTP/FTP de alto nivel	<code>InternetOpenW</code> , <code>HttpSendRequestW</code> , <code>InternetReadFile</code>
<code>winhttp.dll</code>	Cliente HTTP/HTTPS	<code>WinHttpOpen</code> , <code>WinHttpConnect</code> , <code>WinHttpSendRequest</code>
<code>urlmon.dll</code>	Operaciones con URL	<code>URLDownloadToFileW</code>
<code>crypt32.dll</code>	Certificados y DPAPI	<code>CryptProtectData</code> , <code>CryptUnprotectData</code>
<code>bcrypt.dll</code>	Criptografía CNG	<code>BCryptEncrypt</code> , <code>BCryptDecrypt</code> , <code>BCryptGenRandom</code>

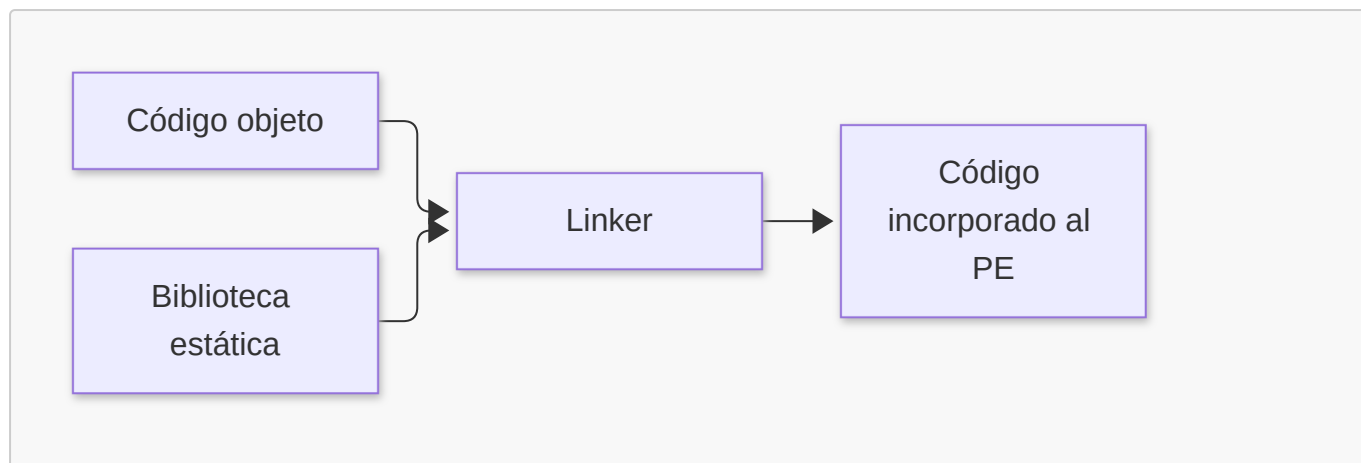
Esta asociación debe interpretarse con cautela. En versiones modernas de Windows existen **exportaciones reenviadas** y contratos **API Set**; por ello, una función importada desde `kernel32.dll` puede terminar ejecutándose en `KernelBase.dll` o alcanzar finalmente código de `ntdll.dll`. Que x64dbg muestre otro módulo como destino efectivo no implica necesariamente que la importación sea incorrecta.

7.2. Enlace estático y enlace dinámico

El **enlazado** es el proceso mediante el cual se resuelven las referencias del programa a funciones o datos definidos en otros módulos.

7.2.1. Enlace estático

En el enlace estático, el linker incorpora al ejecutable el código objeto necesario procedente de una biblioteca estática. El resultado suele ser un archivo más autónomo, aunque también puede ser mayor y contener código de biblioteca duplicado.

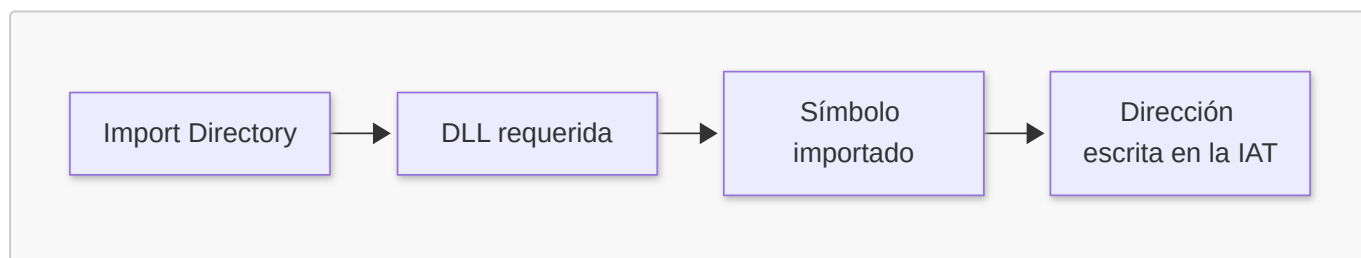


Desde el punto de vista del análisis, una función incorporada de este modo no aparecerá como una importación independiente. Puede ser necesario reconocerla mediante su implementación, símbolos, firmas o patrones del compilador.

[!NOTE] La extensión `.lib` no garantiza por sí sola que una biblioteca sea estática. En el entorno Microsoft también existen **import libraries** `.lib`, que ayudan al linker a generar referencias a funciones alojadas realmente en una DLL.

7.2.2. Enlace dinámico en tiempo de carga

En el enlace dinámico implícito o **load-time linking**, las DLL y los símbolos requeridos quedan descritos en las estructuras de importación del PE. Al crear el proceso, el loader de Windows localiza las dependencias y prepara la IAT antes de transferir el control al punto de entrada normal de la imagen.



7.2.3. Enlace dinámico en tiempo de ejecución

En el enlace dinámico explícito o **run-time linking**, el propio programa obtiene el módulo y resuelve los símbolos mientras se ejecuta. La secuencia más reconocible utiliza:

```

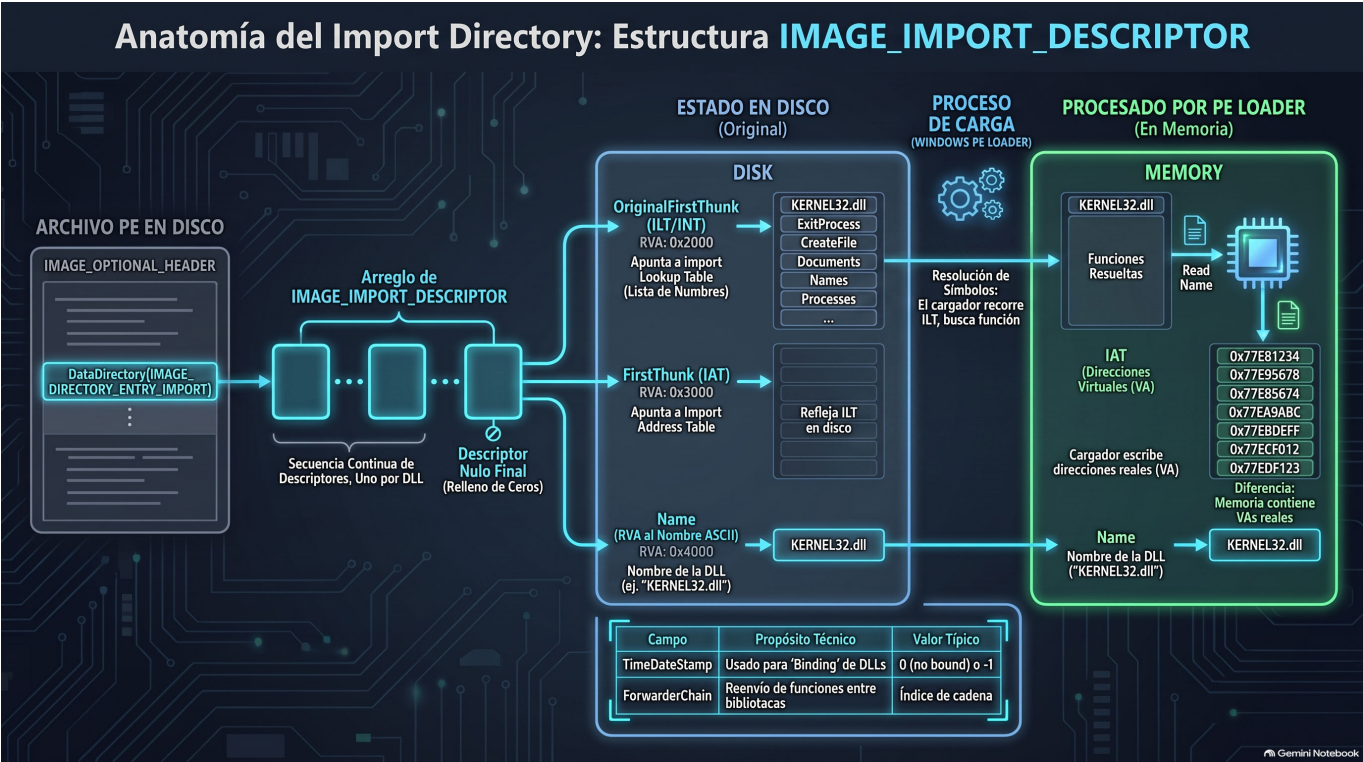
LoadLibraryA / LoadLibraryW / LoadLibraryEx
GetModuleHandleA / GetModuleHandleW
GetProcAddress
  
```

- **LoadLibrary** carga un módulo en el espacio de direcciones del proceso y devuelve un `HMODULE`.
- **GetModuleHandle** obtiene el identificador de un módulo ya cargado.
- **GetProcAddress** devuelve la dirección de una función o variable exportada.

GetProcAddress recibe el nombre del símbolo como una cadena estrecha (`LPCSTR`), aunque el módulo se haya cargado con **LoadLibraryW**.

También existe la **carga retardada** o *delay-load*. En este caso, el PE contiene estructuras específicas de importación retardada y la DLL se resuelve cuando se utiliza por primera vez, no necesariamente durante la carga inicial del proceso.

7.3. Import Directory e **IMAGE_IMPORT_DESCRIPTOR**

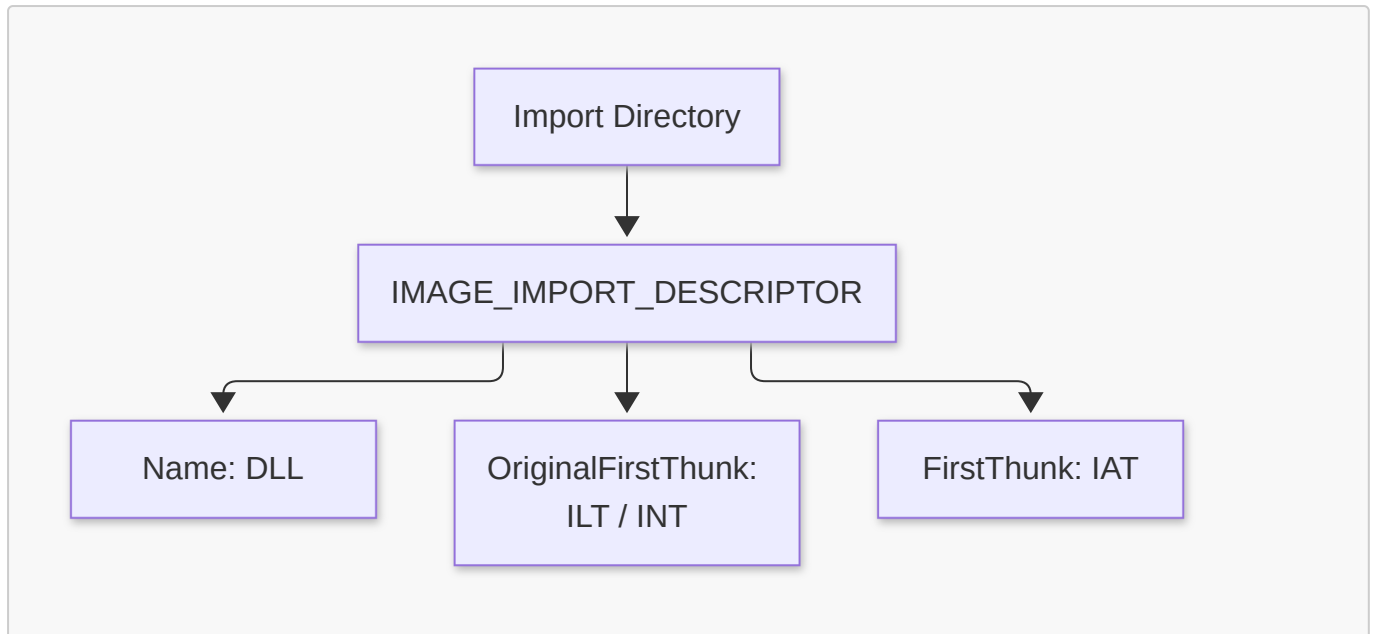


La entrada **IMAGE_DIRECTORY_ENTRY_IMPORT** del array **DataDirectory** contiene la RVA y el tamaño del **Import Directory**. Este directorio está formado por una secuencia de estructuras **IMAGE_IMPORT_DESCRIPTOR**, normalmente una por cada DLL importada, seguida de un descriptor nulo que marca el final.

Los campos más relevantes son:

Campo	Función
OriginalFirstThunk	RVA de la Import Lookup Table (ILT), denominada también INT en muchas herramientas y textos
TimeDateStamp	Campo relacionado con <i>bound imports</i> ; normalmente vale cero cuando no se utilizan
ForwarderChain	Información histórica relacionada con símbolos reenviados
Name	RVA de la cadena ASCII terminada en nulo que contiene el nombre de la DLL
FirstThunk	RVA de la Import Address Table correspondiente a esa DLL

Relación conceptual:



En algunos binarios **OriginalFirstThunk** puede ser cero. Los loaders y parsers suelen tratar entonces **FirstThunk** como fuente de la información necesaria para resolver las importaciones, siempre que sus entradas todavía conserven los valores adecuados.

7.4. Import Lookup Table o Import Name Table

La especificación de Microsoft utiliza el nombre **Import Lookup Table (ILT)**. En documentación, herramientas y conversaciones de análisis también es frecuente encontrar **Import Name Table (INT)**. En este documento se consideran nombres habituales para la tabla apuntada por **OriginalFirstThunk**.

Cada entrada identifica una importación **por nombre** o **por ordinal**:

Importación por nombre

La entrada contiene la RVA de una estructura **IMAGE_IMPORT_BY_NAME**, formada por:

- **Hint**: índice orientativo dentro de la tabla de nombres exportados de la DLL;
- **Name**: cadena ASCII terminada en nulo con el nombre del símbolo.

```
IMAGE_IMPORT_BY_NAME
├─ Hint
└─ Name: "CreateProcessW\0"
```

El **Hint** puede acelerar la búsqueda, pero el loader debe comprobar que el nombre coincida.

Importación por ordinal

El bit más significativo de la entrada indica que la importación se realiza por ordinal. El valor del ordinal se obtiene de los bits de menor peso correspondientes. En este caso no es necesario almacenar el nombre de la función en el PE.

```
DLL
└─ Ordinal #123
```

Las entradas se representan mediante estructuras del tipo `IMAGE_THUNK_DATA`:

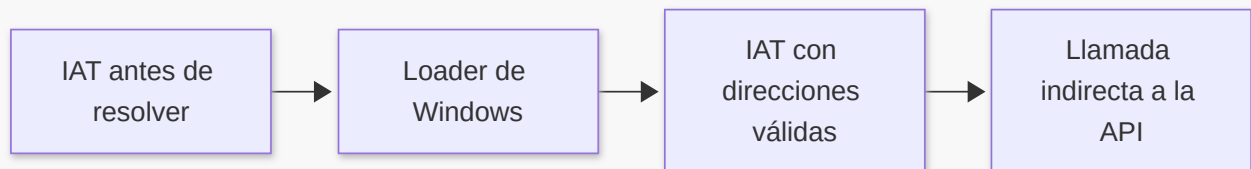
- 4 bytes en PE32;
- 8 bytes en PE32+.

Cada tabla termina con una entrada nula.

7.5. Import Address Table (IAT)

La **Import Address Table** contiene las posiciones que utilizará el código para llamar indirectamente a los símbolos importados.

Antes de la resolución, sus entradas pueden conservar valores equivalentes a los de la ILT. Durante la carga normal, el loader las sustituye por las direcciones virtuales efectivas de las funciones o datos exportados.



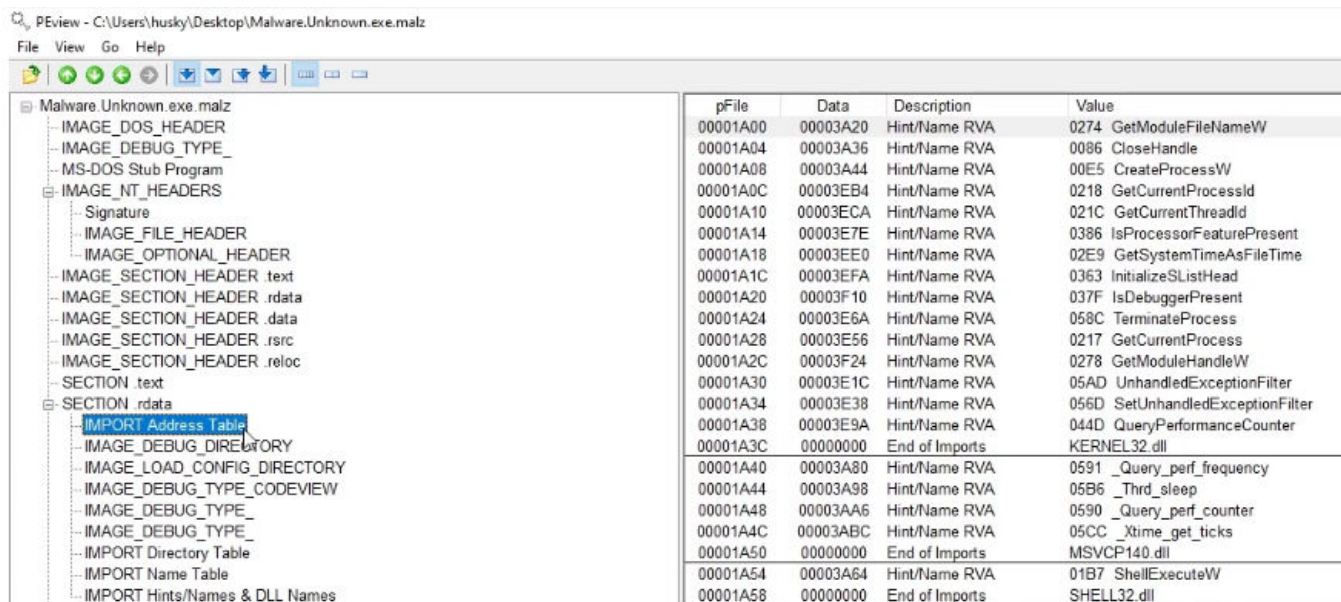
Ejemplo conceptual:

Entrada	Antes de resolver	Después de resolver
IAT[0]	Referencia a <code>CreateProcessW</code>	Dirección virtual efectiva de <code>CreateProcessW</code>
IAT[1]	Referencia a <code>VirtualAlloc</code>	Dirección virtual efectiva de <code>VirtualAlloc</code>
IAT[2]	Referencia a <code>IsDebuggerPresent</code>	Dirección virtual efectiva de <code>IsDebuggerPresent</code>

Una llamada compilada puede adoptar una forma semejante a:

```
call qword ptr [rip + desplazamiento] ; x64, acceso indirecto a una entrada de la IAT
```

o pasar por un pequeño *thunk* generado por el compilador. En Ghidra, IDA o x64dbg, las referencias a estas entradas permiten localizar qué partes del programa utilizan cada función importada.



Diferencia resumida entre Import Directory, ILT/INT e IAT

Estructura	Pregunta que responde	Contenido principal
Import Directory	¿Qué DLL necesita el PE y dónde están sus tablas?	Un IMAGE_IMPORT_DESCRIPTOR por DLL
ILT/INT	¿Qué símbolos quiere importar?	Nombres, <i>hints</i> u ordinales
IAT	¿Dónde están esos símbolos durante la ejecución?	Direcciones efectivas escritas por el loader

Por tanto, no es exacto afirmar que todas las dependencias «se almacenan en la IAT». La descripción de las dependencias se distribuye entre el Import Directory, los nombres de DLL, la ILT/INT y estructuras asociadas; la IAT es la tabla que termina proporcionando las direcciones utilizadas en ejecución.

7.6. Resolución de importaciones durante la carga

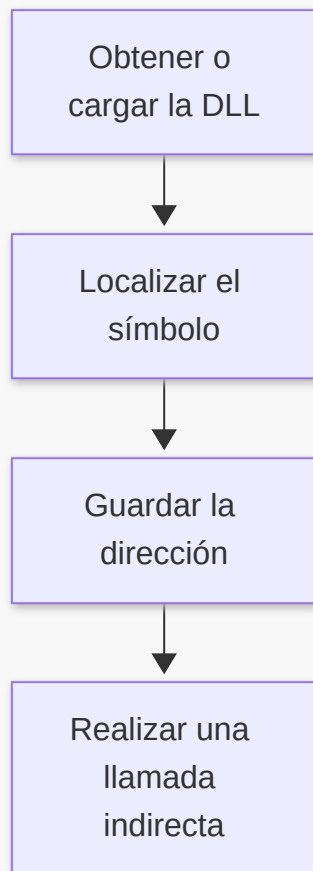
En una carga convencional, el proceso puede resumirse así:

1. El loader lee el Import Directory.
2. Recorre los descriptores hasta alcanzar el descriptor nulo.
3. Localiza o carga cada DLL requerida.
4. Recorre sus entradas de la ILT/INT.
5. Busca cada símbolo en la tabla de exportaciones de la DLL, por nombre u ordinal.
6. Sigue los reenviadores de exportaciones cuando sea necesario.
7. Escribe la dirección resuelta en la entrada correspondiente de la IAT.
8. Continúa con el resto de tareas de inicialización de la imagen.

Este resumen omite detalles internos del loader, pero resulta suficiente para comprender el papel de las estructuras de importación en el análisis y en el mapeo manual.

7.7. Resolución dinámica y ocultación de API

Un programa no está obligado a declarar todas sus funciones externas en la tabla de importaciones. Puede resolverlas en ejecución:



La variante más sencilla deja visibles el nombre de la DLL, el nombre de la API y las importaciones de `LoadLibrary` o `GetProcAddress`. Otras variantes pueden:

- descifrar los nombres justo antes de resolverlos;
- comparar *hashes* en lugar de nombres en texto claro;
- recorrer manualmente el PEB para localizar módulos cargados;
- analizar directamente la Export Address Table (EAT) de una DLL;
- guardar las direcciones en una tabla propia;
- resolver funciones mediante API nativas del loader, como `LdrLoadDll` o `LdrGetProcedureAddress`;
- reconstruir las importaciones después de desempaquetar el código.

Pistas para el análisis estático

- IAT muy pequeña para la funcionalidad aparente del binario.
- Presencia de `LoadLibrary`, `GetModuleHandle` o `GetProcAddress`.
- Cadenas con nombres de DLL o API, incluso si no aparecen como importaciones.
- Rutinas que recorren cabeceras PE y comparan nombres o valores calculados.
- Bucles de *hashing*, descifrado de cadenas y llamadas indirectas posteriores.

Pistas para el análisis dinámico con x64dbg

1. Establecer puntos de ruptura en `LoadLibraryA/W`, `LoadLibraryExA/W` y `GetProcAddress`.
2. Examinar los argumentos antes de ejecutar la función.
3. Tras el retorno de `GetProcAddress`, registrar la dirección devuelta en `RAX` en x64.
4. Seguir dónde se almacena esa dirección.
5. Localizar la llamada indirecta que la utiliza.

Si no aparece una cadena legible como argumento de `GetProcAddress`, puede existir resolución por ordinal, descifrado previo, hashing con recorrido manual de la EAT o un mecanismo distinto de resolución.

7.8. Interpretación de importaciones en análisis de malware

Las importaciones constituyen una **huella funcional parcial**. Resultan útiles para priorizar el análisis, pero pueden ser incompletas por enlace estático, carga dinámica, ofuscación, empaquetado o código no alcanzado.

Grupo de funciones	Posible capacidad que conviene investigar
<code>CreateFileW</code> , <code>ReadFile</code> , <code>WriteFile</code> , <code>DeleteFileW</code>	Acceso o modificación de archivos
<code>RegOpenKeyExW</code> , <code>RegCreateKeyExW</code> , <code>RegSetValueExW</code>	Lectura o modificación del Registro; posible persistencia según la clave
<code>CreateServiceW</code> , <code>StartServiceW</code>	Creación o control de servicios
<code>socket</code> , <code>connect</code> , <code>send</code> , <code>recv</code>	Comunicación de red
<code>InternetOpenW</code> , <code>HttpSendRequestW</code> , <code>InternetReadFile</code>	Tráfico HTTP(S), descarga o comunicación remota
<code>CreateProcessW</code> , <code>ShellExecuteW</code>	Creación de procesos o ejecución de recursos
<code>VirtualAlloc</code> , <code>VirtualProtect</code>	Gestión de memoria; posible JIT, desempaquetado o preparación de código
<code>IsDebuggerPresent</code> , <code>NtQueryInformationProcess</code>	Posible detección de depuradores, según parámetros y flujo
<code>QueryPerformanceCounter</code> , <code>GetTickCount64</code> , <code>Sleep</code>	Temporización; posible evasión temporal, además de usos legítimos
<code>CryptEncrypt</code> , <code>BCryptEncrypt</code> , <code>CryptProtectData</code>	Operaciones criptográficas o protección de datos

Combinaciones relacionadas con process injection

Una secuencia clásica puede incluir:

```

OpenProcess
  ↓
VirtualAllocEx / NtAllocateVirtualMemory
  ↓

```

```
WriteProcessMemory / NtWriteVirtualMemory  
↓  
CreateRemoteThread / NtCreateThreadEx
```

Esta combinación justifica investigar una posible inyección, pero **no la demuestra por sí sola**. Herramientas de depuración, administración, seguridad o instrumentación también pueden usar estas API. Además, otras variantes de inyección emplean APC, secciones compartidas, manipulación del contexto de un hilo, hooks u otros mecanismos y, por tanto, dejan una huella de importaciones diferente.

El procedimiento correcto es:

1. localizar las XREF de cada importación;
2. comprobar si pertenecen a un mismo flujo;
3. revisar los argumentos y el proceso objetivo;
4. determinar qué bytes se escriben y con qué permisos;
5. confirmar cómo se transfiere finalmente la ejecución.

7.9. Inspección de importaciones

Herramientas gráficas

- **PE-bear**, **PEStudio** o **CFF Explorer** para revisar cabeceras, directorios e importaciones.
- **Ghidra** o **IDA** para examinar importaciones y localizar sus referencias cruzadas.
- **x64dbg** para observar la IAT ya resuelta y seguir llamadas dinámicas.
- **Process Explorer** o herramientas equivalentes para observar los módulos cargados durante la ejecución.

Dependency Walker puede ser útil con binarios antiguos, pero su representación de dependencias puede resultar confusa en sistemas modernos debido a API Sets y carga retardada. No debería ser la única fuente utilizada.

Herramientas de línea de comandos

```
dumpbin /imports muestra.exe
```

```
objdump -p muestra.exe
```

Enumeración con Python y **pefile**

Instalación:

```
python -m pip install pefile
```

Script compatible con Python 3:

```
from pathlib import Path
import sys

import pefile

def text(value: bytes | None) -> str:
    return value.decode("ascii", errors="replace") if value else ""

def show_imports(path: Path) -> None:
    pe = pefile.PE(str(path))
    entries = getattr(pe, "DIRECTORY_ENTRY_IMPORT", [])

    if not entries:
        print("No se han encontrado importaciones convencionales.")
        return

    for descriptor in entries:
        print(text(descriptor.dll))

        for imported in descriptor.imports:
            if imported.name:
                symbol = text(imported.name)
            else:
                symbol = f"ordinal #{imported.ordinal}"

            print(f"  {symbol:<40} IAT VA: 0x{imported.address:X}")

if __name__ == "__main__":
    if len(sys.argv) != 2:
        raise SystemExit(f"Uso: {Path(sys.argv[0]).name} <archivo_pe>")

    show_imports(Path(sys.argv[1]))
```

Ejecución:

```
python enum_imports.py muestra.exe
```

Que el script no muestre una capacidad esperada no significa que esta no exista. Deben considerarse la resolución dinámica, las *delay imports*, el empaquetado y las tablas de importación dañadas o reconstruidas.

7.10. Detección y análisis de IAT hooking

La IAT puede ser objeto de manipulación. En un **IAT hook**, una entrada se modifica para que una llamada termine en una función distinta de la resuelta originalmente. Esa función puede inspeccionar, alterar o bloquear la operación antes de continuar opcionalmente hacia el destino legítimo.

Es importante separar dos conceptos:

- **Ocultar importaciones:** evitar que determinadas capacidades aparezcan claramente en el análisis estático, por ejemplo mediante resolución dinámica, hashing o cifrado de nombres.
- **IAT hooking:** redirigir una entrada de la IAT ya existente para interceptar llamadas.

Ambas técnicas afectan al análisis de llamadas externas, pero no son equivalentes. El hooking también se utiliza legítimamente en instrumentación, compatibilidad, monitorización y productos de seguridad.

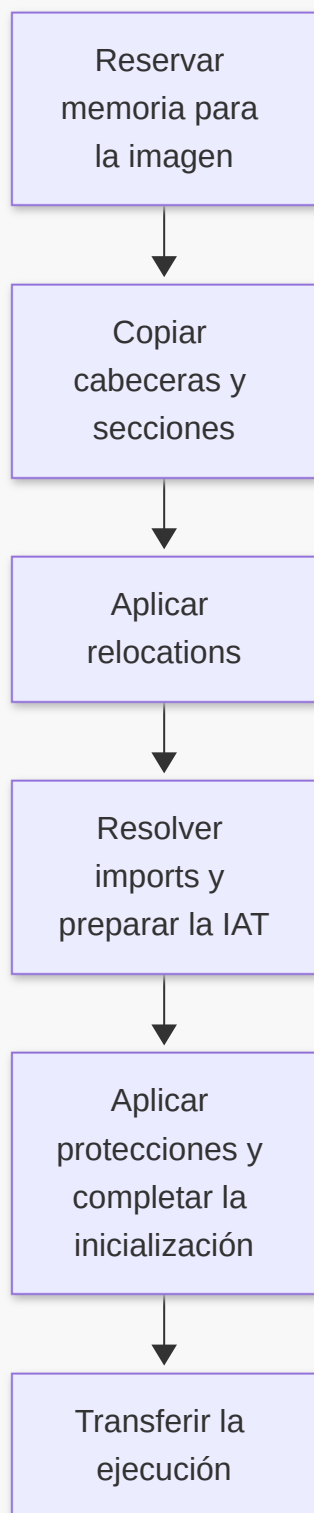
Posibles comprobaciones durante el análisis:

- verificar si cada puntero de la IAT cae dentro de un módulo esperado;
- comparar el destino efectivo con la exportación que debería resolver;
- buscar cambios de protección sobre las páginas que contienen la IAT;
- revisar escrituras sobre entradas de la IAT;
- comprobar si el destino comienza con un salto hacia memoria privada o no respaldada por una imagen.

7.11. Relación con Manual Mapping y PE Injection

Cuando Windows carga una imagen de forma convencional, su loader realiza el mapeo y la inicialización necesarios. En un **Manual Mapping** o en una variante de **PE Injection** que introduce una imagen PE completa sin utilizar el flujo normal del loader, el componente que realiza el mapeo debe reproducir las operaciones necesarias.

De forma simplificada:



Para reconstruir las importaciones, el mapper debe:

1. localizar el Import Directory;
2. recorrer cada `IMAGE_IMPORT_DESCRIPTOR`;
3. cargar o localizar la DLL indicada;
4. recorrer las entradas de la ILT/INT;
5. resolver cada símbolo por nombre u ordinal;
6. escribir su dirección en la IAT de la imagen mapeada.

Si la IAT no se prepara correctamente, una llamada importada puede saltar a una dirección inválida y provocar una excepción. Aun así, **no todas las técnicas de process injection inyectan una imagen PE completa**: la inyección de shellcode, por ejemplo, puede no necesitar una IAT convencional. Esta distinción evita atribuir los requisitos del Manual Mapping a todas las variantes de inyección.

Además de imports y relocations, un componente de mapeo completo puede tener que considerar callbacks TLS, permisos finales de las secciones, excepciones en x64, datos de configuración de carga y otros requisitos de inicialización. Su necesidad exacta depende de la imagen y de la técnica empleada.

7.12. Metodología práctica de análisis

Para estudiar las importaciones de una muestra de forma ordenada:

1. **Identificar las DLL importadas.** Separar funciones del sistema, librerías de terceros y componentes propios.
2. **Agrupar las API por capacidad.** Archivos, procesos, memoria, red, Registro, servicios, criptografía o interfaz gráfica.
3. **Buscar combinaciones significativas.** Una cadena de operaciones aporta más información que una API aislada.
4. **Localizar XREF.** Comprobar desde qué funciones se utiliza cada importación.
5. **Revisar los argumentos.** Rutas, permisos, PID, direcciones, tamaños, dominios, claves del Registro y flags.
6. **Detectar importaciones ausentes.** Buscar resolución dinámica, nombres cifrados, hashing, recorrido de la EAT o desempaquetado.
7. **Contrastar estático y dinámico.** Confirmar módulos cargados, direcciones resueltas y llamadas realmente ejecutadas.
8. **Documentar evidencia y grado de certeza.** Diferenciar entre capacidad potencial, comportamiento observado e inferencia.

Recursos de consulta

- [Microsoft: PE Format](#)
- [Microsoft: LoadLibraryW](#)
- [Microsoft: GetProcAddress](#)
- [Repositorio oficial de pefile](#)
- [MalAPI: catálogo orientativo de API frecuentes en malware](#)

MalAPI.io Contribute FAQ Other   mrd0x					
Injection ?	Evasion ?	Spying ?	Internet ?	Anti-Debugging ?	Ransomware ?
CreateFileMappingA	CreateFileMappingA	AttachThreadInput	WinExec	CreateToolhelp32Snapshot	CryptAcquireContextA
CreateProcessA	DeleteFileA	CallNextHookEx	FtpPutFileA	GetLogicalProcessorInformation	EncryptFileA
CreateRemoteThread	GetModuleHandleA	GetAsyncKeyState	HttpOpenRequestA	GetLogicalProcessorInformationEx	CryptEncrypt
CreateRemoteThreadEx	GetProcAddress	GetClipboardData	HttpSendRequestA	GetTickCount	CryptDecrypt
GetModuleHandleA	LoadLibraryA	GetDC	HttpSendRequestExA	OutputDebugStringA	CryptCreateHash
GetProcAddress	LoadLibraryExA	GetDCEX	InternetCloseHandle	CheckRemoteDebuggerPresent	CryptHashData
GetThreadContext	LoadResource	GetForegroundWindow	InternetOpenA	Sleep	CryptDeriveKey
HeapCreate	SetEnvironmentVariableA	GetKeyboardState	InternetOpenUrlA	GetSystemTime	CryptSetKeyParam
LoadLibraryA	SetFileTime	GetKeyState	InternetReadFile	GetComputerNameA	CryptGetHashParam

[!NOTE] MalAPI puede ayudar a priorizar funciones durante un triaje, pero es un recurso complementario. La semántica y los parámetros de cada API deben confirmarse en la documentación oficial de Microsoft y en el contexto real de la muestra.

8. Relocations

Una imagen PE tiene una dirección base preferida:

```
ImageBase
```

Si termina mapeada en una dirección distinta, determinadas referencias dependientes de la base pueden necesitar corrección.

Por ejemplo:

```
ImageBase preferida = 0x140000000
ActualBase          = 0x7FF700000000
```

El desplazamiento se calcula conceptualmente como:

```
Delta = ActualBase - PreferredImageBase
```

La diferencia o **delta de reubicación** se aplica únicamente a las posiciones identificadas por la **Base Relocation Table**; no se suma indiscriminadamente a todos los valores de la imagen.

El directorio de relocalaciones se organiza en bloques `IMAGE_BASE_RELOCATION`. Cada bloque corresponde normalmente a una página de la imagen y comienza con:

Campo	Función
<code>VirtualAddress</code>	RVA de la página a la que pertenecen las correcciones
<code>SizeOfBlock</code>	Tamaño total del bloque, incluida su cabecera y sus entradas

Después aparecen entradas de 16 bits. Sus 4 bits superiores indican el tipo de relocalación y los 12 bits inferiores contienen el desplazamiento dentro de la página:

```
Página afectada = MappedBase + VirtualAddress
Posición exacta = Página afectada + OffsetDe12Bits
```

Tipos habituales:

Arquitectura	Tipo	Operación habitual
PE32	<code>IMAGE_REL_BASED_HIGHLOW</code>	Ajustar un valor de 32 bits
PE32+	<code>IMAGE_REL_BASED_DIR64</code>	Ajustar un valor de 64 bits
Ambas	<code>IMAGE_REL_BASED_ABSOLUTE</code>	Entrada de relleno; no requiere corrección

Proceso simplificado:

1. Calcular `Delta = MappedBase - ImageBase`.
2. Recorrer los bloques de la tabla de relocalaciones.
3. Obtener el tipo y el desplazamiento de cada entrada.
4. Localizar el valor afectado dentro de la imagen mapeada.
5. Sumar el delta utilizando el tamaño correspondiente a la arquitectura.

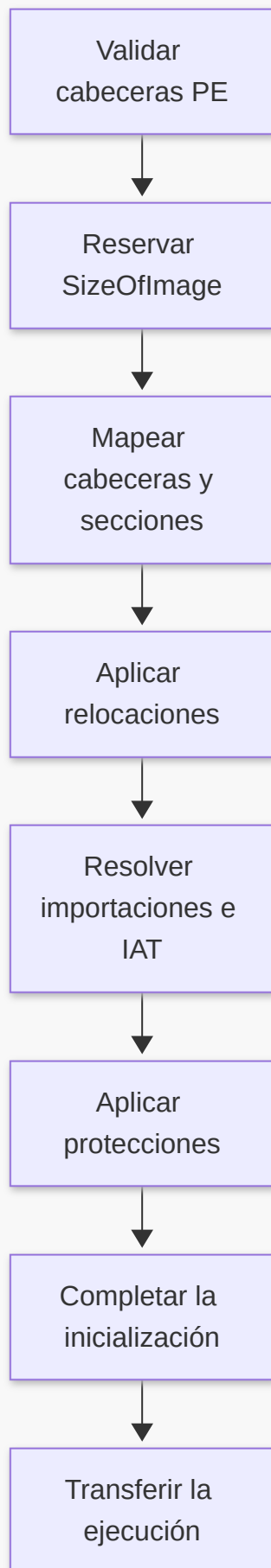
Si la imagen no contiene relocalaciones válidas —por ejemplo, porque presenta `IMAGE_FILE_RELOCS_STRIPPED`— y su base preferida no está disponible, la carga puede fallar. El flag `IMAGE_DLLCHARACTERISTICS_DYNAMIC_BASE` declara compatibilidad con ASLR, pero su aprovechamiento depende también de que exista la información necesaria para reubicar la imagen.

Este mecanismo es fundamental tanto para el cargador de Windows como para comprender una implementación de *Manual Mapping*.

9. Importancia del formato PE para estudiar PE Injection

Una técnica de `PE Injection` o `manual mapping` no consiste simplemente en escribir un bloque arbitrario de bytes en memoria.

Para reconstruir correctamente una imagen PE es necesario comprender elementos como:



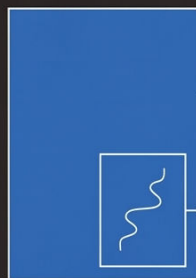
Por este motivo, antes de estudiar una técnica de inyección es fundamental entender cómo se organiza un PE en disco y cómo cambia su representación cuando se convierte en una imagen en memoria.

10. Carga de PE y creación de procesos

ANATOMÍA DE LA CARGA DE UN PE Y CREACIÓN DE PROCESOS EN WINDOWS

Explicar de manera técnica y secuencial cómo el sistema operativo Windows transforma un archivo Portable Executable (PE) en disco en un proceso activo en memoria, detallando las estructuras internas y las etapas críticas de cargo.

CONCEPTOS Y ABSTRACCIONES FUNDAMENTALES



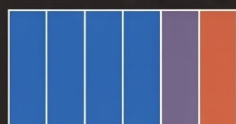
EL PROCESO COMO ABSTRACCIÓN DE DECÚREOS

Un proceso en Windows no se safe código; es un contenedor que agropa un espacio de sbaciócries intual, un tábcó de sagendó, el objeto EPKOCÉES en el sernel y el PEB en modo usuario.

EL HILO (THREAD): UNIDGO DE EJECUCIÓN

Es la rula de ojsanión dentro del proceso con su propia bile, reptieres y TEB, en o66 se ascade medicate el segmento PE y en e64 mediante el siguiente OS.

MEMORIA VIRTUAL Y PERMISOS DE PÁGINA



BIOR: INDICADOR CRÍTICO

El aislamiento de memoria utiliza permisos de lecture (R), ecritería (W) y ajecación (K), las páginas con perñisios (PEX) son indicadores unúZoa como el anúZífe de abalicoe o datos desempaguerodes.

ESTRUCTURAS CLAVE EN MODO USUARIO



TEB (THREAD ENVIRONMENT BLOCK)
Describe el estado del lño en modo usuario, comensando con el TIB e indleyando el identificador del lño y un pñtero el PEB del proceso.

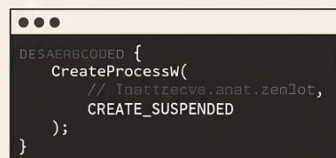


PER (PROCESS ENVIRONMENT BLOCK)
Contiane metadatos crucicias como **ImageBaseAddress**, la controhria **PER Lái** pero nerogar módulos corraíns y el flag **BelogBelmiggged** para técnicas anti-debugging.

PARÁMETROS DEL PROCESO (PROCESSPARAMETERS)

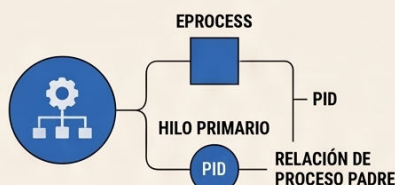
Localizado dentro del PEB, este puntero permita el sistema y el anúliets ecadar a la linne de comandos y a la rora de la imagen original del ejsutabla.

SECUENCIA DE CREACIÓN DEL PROCESO (PASOS 1-5)



1. LLAMADA A LA API DE CREACIÓN

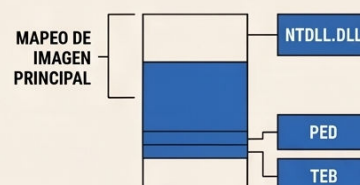
El fleje inicie con `CreateProcessW`, a menudo utilizando el flag `CREATE_SUSPENDED` para preparar el proceso antes de que se ejecte el código.



2. GENERACIÓN DE OBJETOS KERNEL

El tñrnal de Windoe cree el olásto ojacutivo EPROCESS y el diño primario, aspuando el PID (Process ID) y estableciendo la relación de proceso padre.

Paso 4: Activación del PE Loader

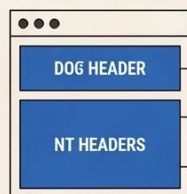


3. INICIALIZACIÓN DEL ESPACIO DE USUARIO

Su cree el mapo de memoria virtuiñó se mapas le imagen principal y se cargan les compoeadtes iniciais como `ntdll.dll`, además de inicializar el PEB y el TEB.

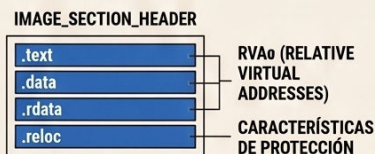
Paso 5: Transferencia de Control Final

CARGA DE LA IMAGEN PE (ANATOMÍA TÉCNICA)



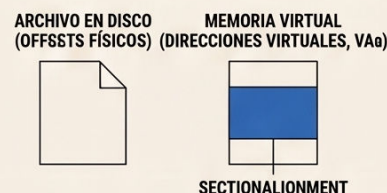
PASO 1: ANÁLISIS DE CABECERAS

El vargador valida el DOS Header y los NT Headers para extroer datos críticos como `ImageBase`, `SizeOfImage` y el número de secciones.



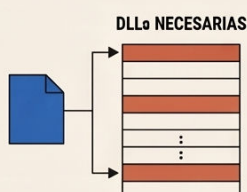
PASO 2: EVALUACIÓN DE LA SECTION TABLE

Se inspeccionan los `IMAGE_SECTION_HEADER` para determinar los RVA's (Relative Virtual Addresses) de cada sección y sus características de protección.



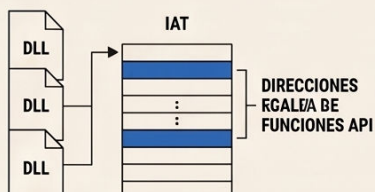
PASO 2: MAPEO EN MEMORIA VIRTUAL

Se espanda el archivo de disco a memoria respetando al `SectionAlignment`, consituyendo los offsets físicos en direcciones virtuales (VA) efectivas.



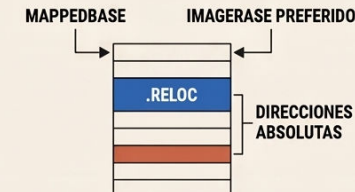
PASO 3: APLICACIÓN DE RELOCALIZACIONES BASE

Se expandió el olásto de disco usao rescpntando al `SectionAlignment`, consituyendo los efaato físicos en direcciones ell programa.



PASO 4: RESOLUCIÓN DE LA IAT (IMPORT TABLE)

Carga recursive de DLLs neousarias y aectitoe de las direcciones reales de las funciones de la API en la tabla de importaciones del programa.



PASO 5: APLICACIÓN DE RELOCALIZACIONES BASE

Si el `MappedBase` es olástimo al `ImageBase` preferido, as utilizá la tabla, `reloc` para ajustar todas las referencias a direcciones absolutas en el código.



PASO 6: SALTO AL ENTRY POINT
Ejecución del código en MappedBase + AddressOfEntryPoint, teniendo en cuenta que los TLS Callbacks pueden ejecutarse incluso antes de este punto.

COMPATIBILIDAD Y ANÁLISIS WOW64



CAPA DE EMULACIÓN WOW64
Permite ejecutar binarios x86 en entornos x64 mediante Emuladores de transición como wow64.dll, wow84cpu.dll y wow64win.dll.

RELEVANCIA EN ANÁLISIS DE MALWARE
Muchas muestras detectadas al estar corriendo bajo WOW64 mediante leMow64Process para adaptar su comportamiento o inyectar payloads específicos de arquitectura.

Gemini Notebook

Vamos a analizar cómo Windows carga un archivo PE, lo ejecuta y lo convierte en un programa activo.

Hasta ahora, la estructura PE se ha estudiado principalmente como un archivo almacenado en disco. Sin embargo, cuando Windows ejecuta un programa, ese archivo no se utiliza directamente tal y como está guardado. El sistema operativo debe leer sus cabeceras, preparar la memoria virtual del proceso, cargar sus secciones, resolver las librerías necesarias, aplicar relocations si es necesario y, finalmente, transferir la ejecución al punto de entrada del programa.

Este proceso es fundamental en análisis de malware, ya que muchas técnicas maliciosas dependen precisamente de manipular la forma en la que un PE se carga en memoria, se ejecuta o interactúa con otros componentes del sistema.

10.1. ¿Qué es un proceso?

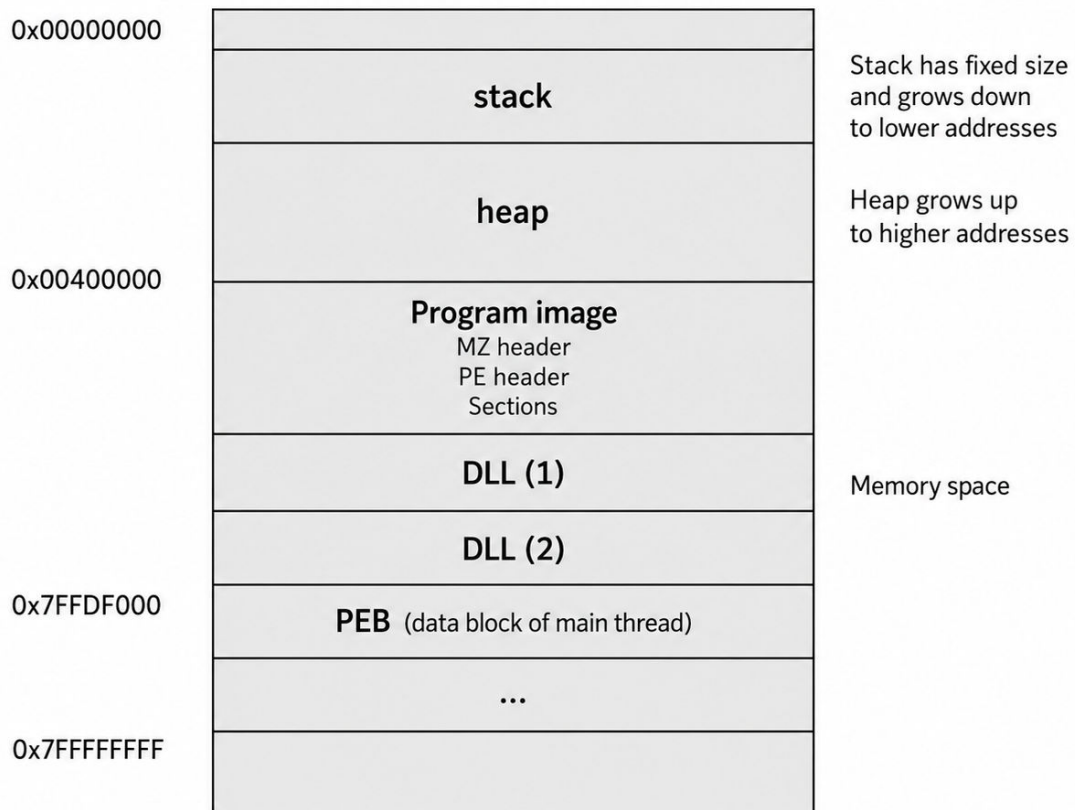
Un proceso es la abstracción con la que Windows representa una instancia de un programa en ejecución. No es únicamente una estructura ni el código de un ejecutable cargado: reúne un conjunto de objetos y recursos, entre ellos:

- un espacio de direcciones virtual;
- uno o más hilos;
- un token de seguridad;
- una tabla de identificadores para objetos abiertos;
- la imagen ejecutable y los módulos cargados;
- parámetros del proceso y variables de entorno;
- un identificador de proceso o PID;
- información de contabilidad, sincronización y planificación.

En el kernel, el proceso está representado principalmente por un objeto ejecutivo cuya estructura interna se conoce como **EPROCESS**. En modo usuario existen otras estructuras relacionadas, como el PEB. Ninguna de ellas, por sí sola, equivale a todo el concepto de proceso.

Cuando se ejecuta un archivo como **calc.exe**, Windows crea el proceso, su hilo primario y los objetos necesarios, prepara su espacio de direcciones y realiza la inicialización requerida antes de transferir el control al código de la aplicación.

En análisis de malware, entender qué es un proceso es importante porque el código malicioso suele interactuar con procesos para inyectar código, crear procesos hijos, manipular memoria o detectar herramientas de análisis.



Example of a 32-bit process memory layout

10.2. Memoria virtual y memoria física

Cada proceso tiene su propio espacio de memoria virtual. Esto significa que, desde el punto de vista del proceso, parece disponer de un rango de memoria propio e independiente.

En ese espacio de memoria virtual se cargan diferentes elementos, como:

- La imagen principal del programa.
- Las DLL necesarias.
- La pila.
- El heap.
- Las estructuras internas del proceso.
- Otros rangos auxiliares de memoria.

La memoria virtual no siempre corresponde directamente con la memoria física. Windows mantiene un sistema de mapeo que traduce direcciones virtuales a direcciones físicas cuando es necesario.

Además, cada región de memoria puede tener permisos distintos, por ejemplo:

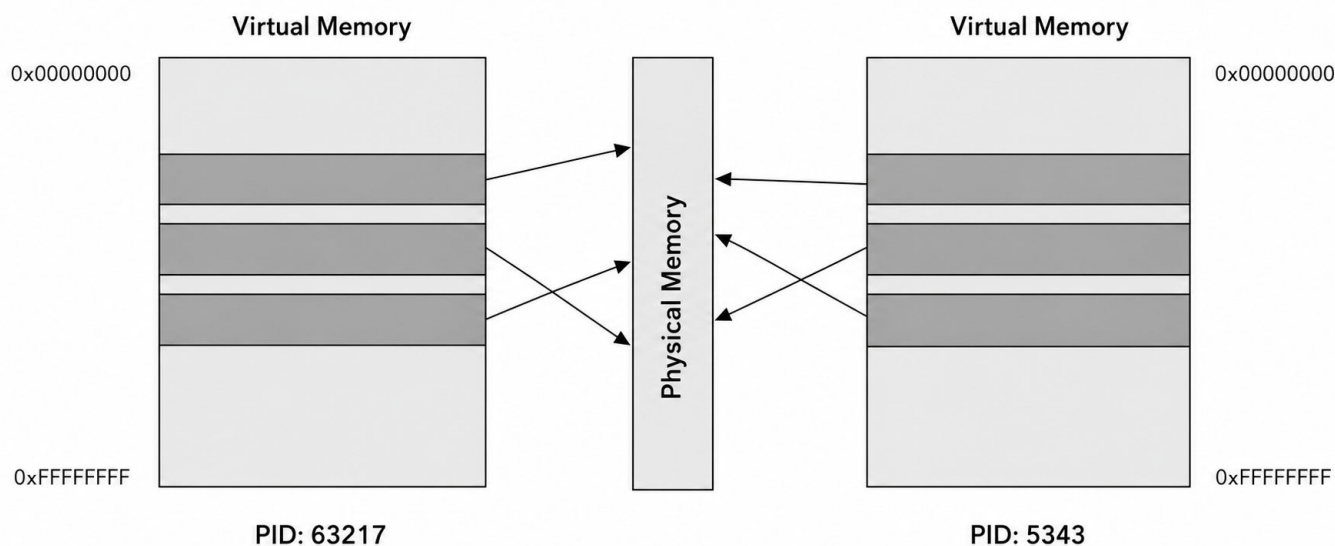
```

READ | WRITE
READ | EXECUTE
READ | WRITE | EXECUTE

```

Estos permisos son muy importantes en análisis de malware. Una región con lectura, escritura y ejecución simultáneas puede justificar una revisión porque puede alojar código automodificable, shellcode o datos desempaquetados. También existen usos legítimos, como determinados motores **JIT**, por lo que el permiso **RwX** es un indicio y no una conclusión.

La memoria virtual también permite aislar procesos entre sí. Un proceso no debería poder acceder directamente a la memoria de otro proceso salvo que utilice mecanismos permitidos por el sistema operativo, como determinadas APIs de Windows.



Mappings between physical and virtual memory

10.3. ¿Qué es un hilo?

Un hilo, o *thread*, representa una ruta de ejecución dentro de un proceso.

Un proceso puede tener uno o varios hilos ejecutándose de forma simultánea. Cada hilo mantiene su propio estado de ejecución, incluyendo:

- Registros.
- Pila.
- Puntero de instrucción.
- Último error.
- Identificador de hilo.
- Información relacionada con el manejo de excepciones.

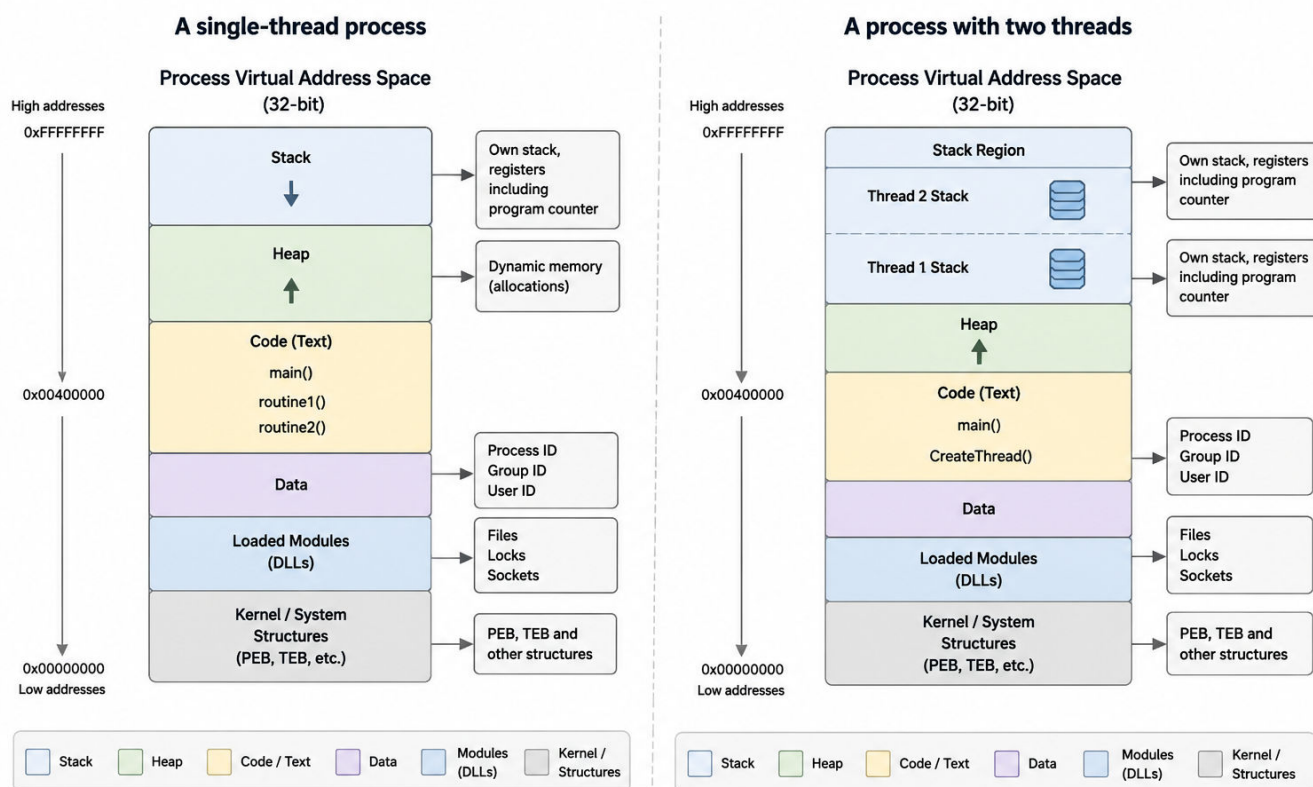
Aunque cada hilo tiene su propio estado, todos los hilos de un mismo proceso comparten los recursos del proceso. Esto incluye la memoria virtual, los archivos abiertos, los sockets, las DLL cargadas y otros objetos del sistema.

Windows asigna a cada hilo pequeños intervalos de tiempo de ejecución. Cuando el sistema operativo cambia de un hilo a otro, guarda el estado completo del hilo actual para poder reanudarlo posteriormente desde el mismo punto.

Este mecanismo se conoce como cambio de contexto.

En malware, los hilos son especialmente relevantes porque una muestra puede crear hilos adicionales para realizar tareas en paralelo, como:

- Comunicación con un servidor C2.
- Ejecución de payloads.
- Monitorización del sistema.
- Persistencia.
- Inyección de código.
- Ejecución de comandos recibidos remotamente.



Example processes with one and multiple threads (improved)

10.4. Estructuras TIB, TEB y PEB

Windows mantiene en el espacio de usuario varias estructuras internas relacionadas con el hilo y el proceso:

- **NT_TIB**, conocido habitualmente como TIB;
- **TEB**, o *Thread Environment Block*;
- **PEB**, o *Process Environment Block*.

Cada hilo posee su propio **TEB**, mientras que los hilos de un mismo proceso apuntan al **PEB** de ese proceso. En x86 se accede normalmente al **TEB** actual a través de **FS**; en x64, mediante **GS**.

Dos accesos frecuentes durante el análisis son:

```
mov eax, dword ptr fs:[0x30] ; puntero al PEB en un proceso x86
mov rax, qword ptr gs:[0x60] ; puntero al PEB en un proceso x64
```

Estos offsets son útiles para el análisis de las arquitecturas indicadas, pero **TEB** y **PEB** son estructuras internas y no deben tratarse como una interfaz estable para el desarrollo de aplicaciones.

10.4.1. TIB

El **TIB**, denominado **NT_TIB** en las cabeceras de Windows, contiene información básica del hilo, como referencias a los límites de su pila y otros datos utilizados por el entorno de ejecución. No es una estructura paralela e independiente del TEB: constituye su parte inicial.

10.4.2. TEB

El **TEB**, o *Thread Environment Block*, describe el estado de un hilo en modo usuario. Comienza con un **NT_TIB** y añade información como:

- el identificador del hilo y del proceso en **ClientId**;
- el valor de último error;
- slots de Thread Local Storage (TLS);
- un puntero al PEB del proceso;
- otros campos internos utilizados por Windows y los runtimes.

Aunque a veces se usan de forma informal como sinónimos, TIB y TEB no designan exactamente lo mismo.

10.4.3. PEB

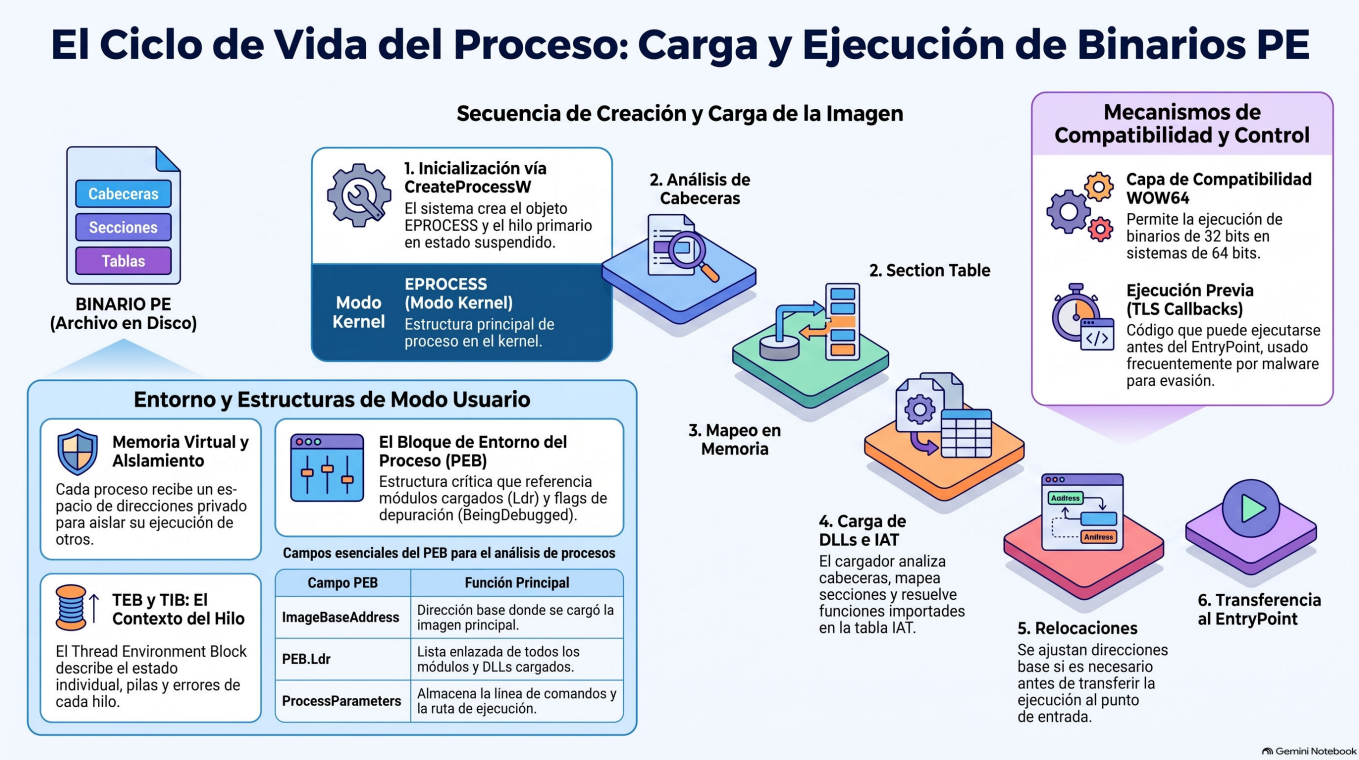
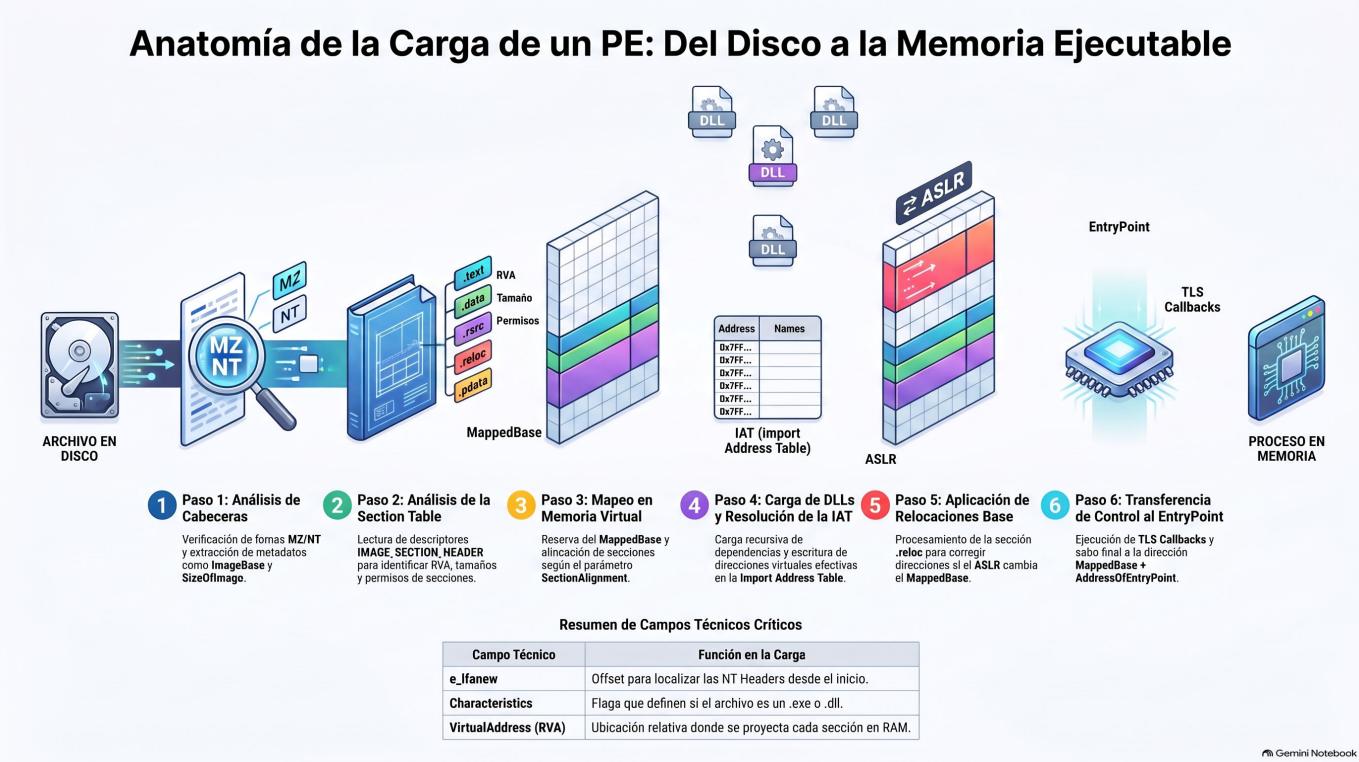
El **PEB**, o *Process Environment Block*, contiene o referencia información de modo usuario necesaria para el proceso y para el cargador de Windows. Entre sus elementos más relevantes para el análisis se encuentran:

- **ImageBaseAddress**: base de la imagen principal;
- **Ldr**: puntero a información sobre los módulos cargados;
- **ProcessParameters**: puntero a parámetros como la línea de comandos y la ruta de la imagen;
- **BeingDebugged**: indicador utilizado por algunas comprobaciones anti-debugging;
- información interna relacionada con heaps, flags y subsistemas.

El nombre del proceso y el **PID** no deben presentarse como campos directos del **PEB**. La ruta de la imagen puede obtenerse a través de **ProcessParameters**, mientras que el **PID** puede consultarse mediante APIs, estructuras del kernel o información asociada al hilo, según el contexto.

La información accesible a través de **PEB.Ldr** es especialmente interesante en análisis de malware: permite recorrer los módulos cargados y puede utilizarse para resolver **API** sin llamar directamente a **GetModuleHandle**. Algunas muestras también consultan **BeingDebugged** u otros campos internos para detectar depuradores.

10.5. Creación de un proceso paso a paso



Cuando Windows ejecuta un archivo PE, realiza una serie de pasos para crear el proceso y prepararlo para su ejecución.

10.5.1. Inicio del programa

Cuando el usuario ejecuta un programa, por ejemplo haciendo doble clic sobre `calc.exe`, otro proceso ya existente, como `explorer.exe`, solicita al sistema operativo la creación del nuevo proceso.

Para ello, normalmente se utiliza una API como:

```
CreateProcessA
```

La variante `W` recibe cadenas `Unicode` y es la opción habitual en código moderno. La llamada solicita la creación del nuevo proceso y de su hilo primario. Si se utiliza el flag `CREATE_SUSPENDED`, el hilo se crea, pero no comienza su ejecución normal hasta que se reanuda.

10.5.2. Creación del proceso y del hilo primario

Windows crea los objetos del kernel que representan el nuevo proceso y su hilo primario. La estructura interna asociada al objeto proceso se conoce como `EPROCESS`; el hilo está representado por sus propias estructuras internas.

Durante esta fase, Windows:

- crea el objeto proceso y su espacio de direcciones;
- asigna el PID y el identificador del hilo primario;
- establece la relación de creación o proceso padre utilizada por el sistema;
- crea el token y las tablas internas necesarias;
- prepara el contexto inicial del hilo primario.

Por ejemplo, si `explorer.exe` lanza `calc.exe`, normalmente `explorer.exe` aparecerá como proceso padre. Esta relación puede modificarse mediante mecanismos legítimos del sistema y también puede ser manipulada por determinadas técnicas, por lo que no debe utilizarse como única evidencia de procedencia.

10.5.3. Inicialización de la memoria virtual

Como parte de la creación, Windows prepara el espacio de memoria virtual y las estructuras iniciales necesarias en modo usuario.

En esta fase, el sistema operativo:

- crea el mapa de memoria virtual;
- mapea la imagen principal y componentes iniciales;
- inicializa el PEB y el TEB del hilo primario;
- prepara pilas, parámetros del proceso y datos de entorno.

En un proceso Win32 convencional interviene `ntdll.dll` y, habitualmente, módulos como `kernel32.dll` y `KernelBase.dll`, bien como dependencias directas o a través de exportaciones reenviadas:

```
ntdll.dll  
kernel32.dll  
KernelBase.dll
```

La combinación exacta depende del tipo de imagen, sus importaciones y la versión de Windows.

10.5.4. Inicialización del cargador y de la imagen PE

El hilo primario entra en las rutinas de inicialización de modo usuario. El cargador de Windows termina de preparar la imagen y sus dependencias antes de transferir el control al punto de entrada de la aplicación.

Durante esta fase, el cargador de Windows:

- valida las cabeceras del PE;
- prepara la representación en memoria de sus secciones;
- localiza o carga las DLL requeridas;
- resuelve las funciones importadas;
- escribe las direcciones resultantes en la IAT;
- aplica relocalaciones de base si son necesarias;
- configura protecciones y ejecuta inicializadores requeridos.

Esto permite que el código pueda llamar posteriormente a funciones externas como `CreateFileW`, `LoadLibraryW`, `GetProcAddress` o `VirtualAlloc`.

10.5.5. Transferencia a la aplicación

El hilo primario —creado al comienzo del proceso— realiza la inicialización en modo usuario y, cuando esta finaliza, transfiere la ejecución al punto de entrada del archivo PE.

El punto de entrada se encuentra definido en el campo:

AddressOfEntryPoint

Sin embargo, si existen callbacks TLS, pueden ejecutarse antes del punto de entrada principal. Esto es muy importante en análisis de malware, ya que algunas muestras los utilizan para ejecutar código antes de que el analista alcance el inicio aparente del programa.

10.6. Carga de un archivo PE paso a paso

La carga de un archivo PE en memoria sigue una secuencia concreta. Este proceso es realizado por el cargador de Windows.

10.6.1 Análisis de las cabeceras

Primero, Windows analiza la cabecera `DOS` para localizar la `cabecera PE`.

Después analiza las estructuras principales del `PE`, como:

- `IMAGE_FILE_HEADER`
- `IMAGE_OPTIONAL_HEADER`

De estas cabeceras obtiene información fundamental, como:

```
ImageBase  
NumberOfSections  
SizeOfImage
```

ImageBase indica la dirección preferida donde el PE debería cargarse en memoria.

NumberOfSections indica cuántas secciones tiene el archivo.

SizeOfImage indica el tamaño total que ocupará el PE una vez cargado en memoria.

10.6.2 Análisis de la tabla de secciones

Después, el cargador analiza la tabla de secciones.

Para cada sección obtiene información como:

```
VirtualAddress  
VirtualSize  
PointerToRawData  
SizeOfRawData  
Characteristics
```

Estos campos permiten saber:

- dónde se encuentra el contenido físico de la sección;
- en qué RVA debe representarse dentro de la imagen;
- qué tamaño ocupa en disco y en memoria;
- qué características y permisos tendrá;
- qué parte deberá inicializarse a cero.

Esta diferencia entre el archivo en disco y su representación en memoria es clave para entender cómo se ejecuta realmente un **PE**.

10.6.3 Mapeo del archivo en memoria

De forma conceptual, el cargador coloca las cabeceras y las secciones en sus **RVA** correspondientes. En una carga normal, Windows no necesita copiar de manera inmediata y continua todo el archivo: crea una sección de imagen, la mapea en el espacio virtual y las páginas pueden materializarse bajo demanda.

Para ello utiliza valores como:

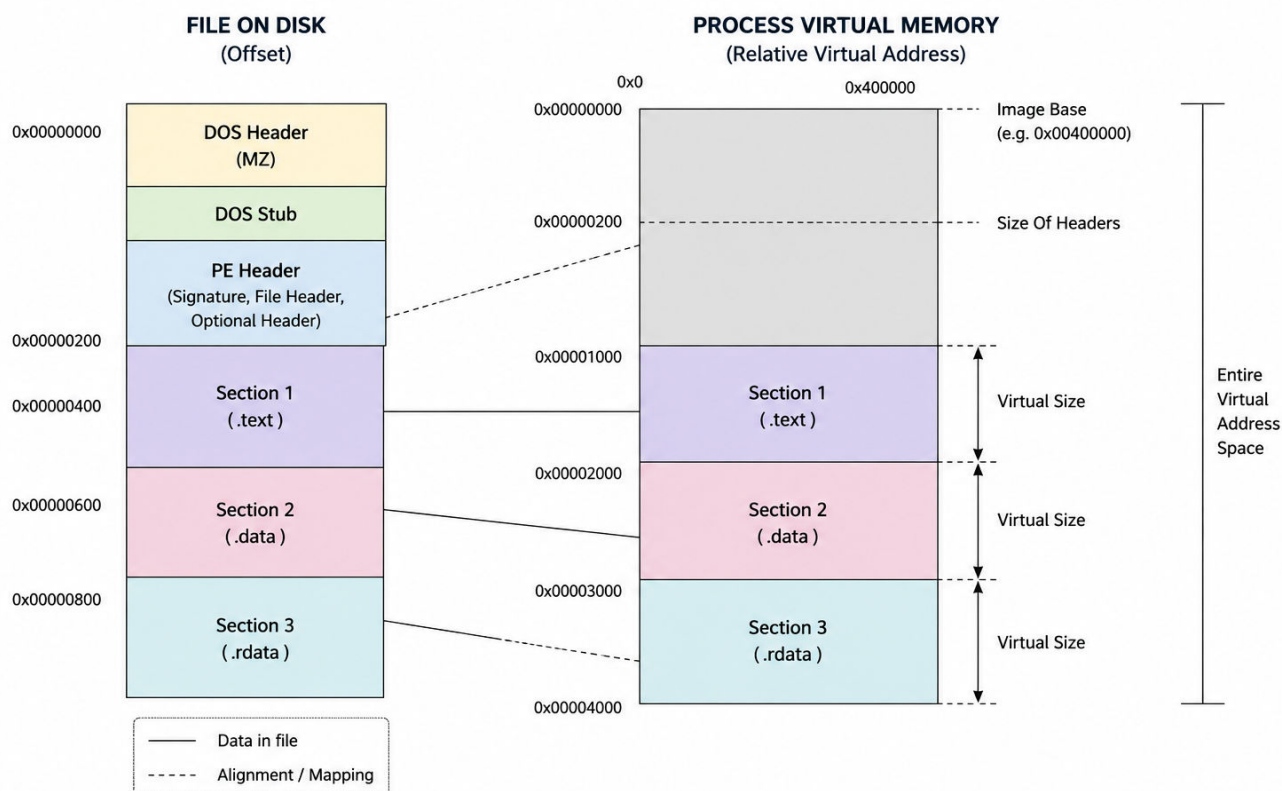
```
SectionAlignment  
VirtualAddress  
VirtualSize
```

Las secciones no tienen por qué ocupar exactamente la misma posición en disco que en memoria. Por ejemplo, una sección puede empezar en el offset `0x400` dentro del archivo, pero cargarse en memoria en la dirección relativa `0x1000`.

Las cabeceras deben respetar las reglas del formato PE. El cargador utiliza `SectionAlignment` y `FileAlignment` para interpretar la disposición declarada, pero no debe asumirse que reparará automáticamente valores incoherentes; una imagen mal formada puede ser rechazada.

Este paso explica por qué en análisis de malware es necesario diferenciar entre:

- offset en disco;
- RVA;
- VA.



Mapping sections from disk to memory (improved)

10.6.4 Carga de librerías externas

Después de mapear el PE principal, Windows carga las DLL necesarias.

Este proceso puede ser recursivo, ya que una DLL puede depender de otras DLLs.

Una vez cargadas las librerías, el cargador localiza las funciones importadas y escribe sus direcciones reales en memoria dentro de la IAT.

Esto permite que el programa pueda llamar correctamente a funciones externas durante su ejecución.

En malware, esta fase es muy importante porque algunas muestras intentan ocultar sus imports y resolver APIs dinámicamente mediante funciones como:

```
LoadLibraryA  
GetProcAddress
```

10.6.5 Aplicación de relocations

Si el PE no puede cargarse en su dirección preferida indicada por `ImageBase`, Windows debe cargarlo en otra dirección.

Cuando esto ocurre, las direcciones absolutas del código pueden dejar de ser válidas.

Para corregirlo, el cargador utiliza la tabla de relocations, si existe. Esta tabla permite ajustar las direcciones internas del programa según la nueva dirección base.

Este proceso se conoce como relocación.

Si un binario no tiene tabla de relocations y no puede cargarse en su `ImageBase`, pueden producirse errores de carga o ejecución.

10.6.6 Transferencia al punto de entrada

Una vez preparada la imagen, resueltas sus dependencias y aplicadas las relocalaciones necesarias, Windows puede transferir el control a la aplicación.

Para ello, el primer hilo del proceso acaba ejecutando el punto de entrada del programa.

La dirección efectiva del punto de entrada se calcula con la base en la que la imagen quedó realmente mapeada:

```
MappedBase + AddressOfEntryPoint
```

`ImageBase + AddressOfEntryPoint` solo coincide con esa dirección cuando la imagen se carga en su base preferida. Además, callbacks TLS y otros inicializadores pueden ejecutar código antes del punto de entrada principal.

10.7. Procesos WOW64

WOW64 es la capa de compatibilidad que permite ejecutar aplicaciones Win32 de 32 bits en ediciones de Windows de 64 bits.

El nombre WOW64 significa:

```
Windows 32-bit On Windows 64-bit
```

Cuando un proceso x86 se ejecuta en Windows x64, WOW64 proporciona las transiciones, el entorno y las adaptaciones necesarias para que el programa conserve una vista coherente de 32 bits.

En un sistema Windows x64 tradicional intervienen módulos como:

```
wow64.dll  
wow64cpu.dll  
wow64win.dll
```

Estos módulos actúan como intermediarios entre el código de 32 bits y los componentes nativos de 64 bits. La implementación exacta depende de la arquitectura y de la versión de Windows; no debe describirse de forma general como una simple «emulación» ni como un único cambio directo de modo x86 a x64.

WOW64 también puede aplicar redirecciones que resultan importantes durante el análisis, por ejemplo:

- vistas de 32 y 64 bits del Registro;
- redirección de determinadas rutas del sistema;
- diferencias entre `System32`, `SysWOW64` y el acceso explícito a la vista nativa;
- coexistencia de estructuras y módulos correspondientes a ambas arquitecturas.

10.8. Importancia de WOW64 en análisis de malware

WOW64 es relevante en análisis de malware porque muchas muestras necesitan saber en qué tipo de entorno se están ejecutando.

Una muestra puede comprobar la arquitectura del proceso y del sistema utilizando API como:

```
IsWow64Process  
IsWow64Process2
```

Esta información puede utilizarse para decidir:

- qué carga útil (payload) ejecutar;
- qué rutas o vista del Registro utilizar;
- qué técnica de inyección es compatible con el proceso objetivo;
- si necesita un componente auxiliar de otra arquitectura;
- si debe continuar ejecutándose o finalizar.

Por tanto, detectar el uso de APIs relacionadas con WOW64 puede ayudar a entender si la muestra adapta su comportamiento según la arquitectura del sistema.

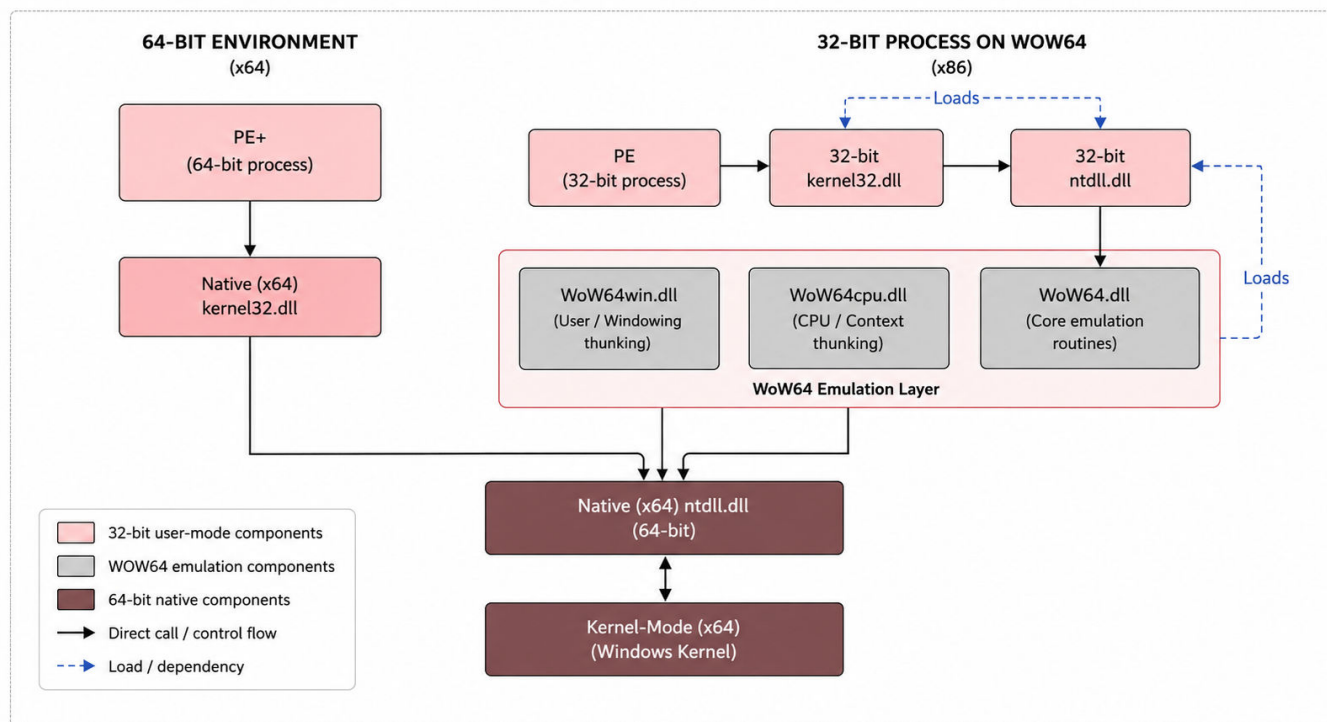


Figure 3.16 – WOW64 architecture (improved)

10.9. Resumen del proceso completo

La carga de un **PE** y la creación de un proceso pueden resumirse, de forma deliberadamente simplificada, así:

1. Un proceso existente solicita la creación mediante una API como **CreateProcessW**.
2. Windows crea el objeto proceso y su hilo primario.
3. Asigna identificadores, token, tablas internas y espacio de direcciones.
4. Mapea la imagen principal y componentes iniciales como **ntdll.dll**.
5. Inicializa el **PEB**, el **TEB** y los parámetros del proceso.
6. El hilo primario entra en las rutinas de inicialización de modo usuario.
7. El cargador valida las cabeceras y prepara las secciones de la imagen.
8. Localiza o carga las **DLL** necesarias y resuelve sus importaciones.
9. Escribe las direcciones resueltas en la **IAT**.
10. Aplica relocalaciones de base cuando la imagen no ocupa su dirección preferida.
11. Establece las protecciones finales y completa la inicialización requerida.
12. Ejecuta **callbacks** **TLS** y otros inicializadores cuando corresponda.
13. El mismo hilo primario transfiere finalmente el control al punto de entrada de la aplicación.

Este flujo es esencial para entender cómo Windows transforma un archivo PE almacenado en disco en un programa activo en memoria.

Desde el punto de vista del análisis de malware, conocer este proceso permite investigar ejecución previa al punto de entrada, importaciones resueltas dinámicamente, secciones con permisos anómalos, inyección de código, uso de **WOW64** o consulta de estructuras internas como el **PEB**.

11. Ficheros ejecutables Linux

En vez de **PE**, lo habitual es que tenga formato: **ELF**. ELF significa: **Executable and Linkable Format** y es el formato típico de ejecutables, librerías y objetos en Linux.

Un ejecutable Linux ELF suele empezar por:

```
7F 45 4C 46
```

Interpretación:

```
0x7F -> byte mágico
45    -> E
4C    -> L
46    -> F
```

Estructura general de un ELF:

```
+-----+
| ELF Header                               |
+-----+
| Program Header Table                     |
+-----+
| Sections                                |
| - .text                                 |
| - .rodata                               |
| - .data                                 |
| - .bss                                  |
| - .plt                                  |
| - .got                                  |
| - .dynamic                             |
| - .dynsym                              |
| - .dynstr                              |
| - .rel.plt / .rel.plt                   |
| - .init                                 |
| - .fini                                 |
| - otras...                              |
+-----+
| Section Header Table                     |
+-----+
```

Equivalencias PE vs ELF:

Windows PE	Linux ELF
MZ	7F 45 4C 46 / .ELF

Windows PE	Linux ELF
PE\0\0	ELF Header
Optional Header	ELF Header + Program Headers
Section Table	Section Header Table
.text	.text
.rdata	.rodata
.data	.data
.bss	.bss
Import Table	Dynamic Symbol Table / Dynamic Section
IAT	GOT / PLT
TLS Directory	TLS segmentos/secciones ELF
.rsrc	No hay equivalente directo estándar
Rich Header	No existe equivalente directo

12. EJEMPLO 1: Análisis de la estructura PE de una muestra

Análisis del fichero [Lab_03-1.malware](#): El análisis del fichero muestra que su estructura interna corresponde a un ejecutable PE32 de Windows, concretamente un ejecutable x86 de 32 bits.

[!CAUTION] La muestra debe manipularse únicamente en un laboratorio aislado, sin ejecutarla en el sistema anfitrión. Antes del análisis debe verificarse su identidad mediante un hash. Para la versión examinada:

SHA-256: dce7c942883810c535fab689ece1e366287336c79ed15c808038bb5863eddf66

No es necesario incluir un volcado completo del comienzo del archivo. Los rangos relevantes pueden obtenerse de forma selectiva:

```

└─$ xxd -g 1 Lab_03-1.malware | more
00000000: 4d 5a 90 00 03 00 00 00 04 00 00 00 ff ff 00 00  MZ.....
00000010: b8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00  .....@.....
00000020: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
00000030: 00 00 00 00 00 00 00 00 00 00 00 00 08 01 00 00  .....
00000040: 0e 1f ba 0e 00 b4 09 cd 21 b8 01 4c cd 21 54 68  ....!...L.!Th
00000050: 69 73 20 70 72 6f 67 72 61 6d 20 63 61 6e 6e 6f  is program canno
00000060: 74 20 62 65 20 72 75 6e 20 69 6e 20 44 4f 53 20  t be run in DOS
00000070: 6d 6f 64 65 2e 0d 0d 0a 24 00 00 00 00 00 00 00  mode....$.
00000080: 1b 51 88 7b 5f 30 e6 28 5f 30 e6 28 5f 30 e6 28  .Q.{_0.(_0.(_0.(
00000090: 56 48 75 28 55 30 e6 28 ba 69 e7 29 5d 30 e6 28  VHu(U0.(.i.)]0.(

```

```

000000a0: ba 69 e5 29 5e 30 e6 28 ba 69 e3 29 4e 30 e6 28 .i.)^0.(.i.)N0.(
000000b0: ba 69 e2 29 52 30 e6 28 82 cf 2d 28 5c 30 e6 28 .i.)R0.(..-(\0.(
000000c0: 5f 30 e7 28 6a 30 e6 28 ad 69 ef 29 5e 30 e6 28 _0.(j0.(.i.)^0.(
000000d0: ad 69 19 28 5e 30 e6 28 5f 30 71 28 5e 30 e6 28 .i.(^0.( _0q(^0.(
000000e0: ad 69 e4 29 5e 30 e6 28 52 69 63 68 5f 30 e6 28 .i.)^0.(Rich_0.(
000000f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000100: 00 00 00 00 00 00 00 00 50 45 00 00 4c 01 05 00 .....PE..L...
00000110: c0 b5 fb 55 00 00 00 00 00 00 00 00 e0 00 02 01 ...U.....
00000120: 0b 01 0e 00 00 0e 00 00 00 66 00 00 00 00 00 00 .....f.....
00000130: 0d 13 00 00 00 10 00 00 00 20 00 00 00 00 40 00 .....@.
00000140: 00 10 00 00 00 02 00 00 06 00 00 00 00 00 00 00 .....
00000150: 06 00 00 00 00 00 00 00 00 b0 00 00 00 04 00 00 .....
00000160: 00 00 00 00 03 00 40 81 00 00 10 00 00 10 00 00 .....@.....
00000170: 00 00 10 00 00 10 00 00 00 00 00 00 10 00 00 00 .....
00000180: 00 00 00 00 00 00 00 00 74 25 00 00 a0 00 00 00 .....t%.....
00000190: 00 40 00 00 50 52 00 00 00 00 00 00 00 00 00 00 ..@..PR.....
000001a0: 00 00 00 00 00 00 00 00 a0 00 00 58 01 00 00 .....X...
000001b0: 90 21 00 00 70 00 00 00 00 00 00 00 00 00 00 00 ..!.p.....
000001c0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000001d0: 00 22 00 00 40 00 00 00 00 00 00 00 00 00 00 00 .."@.....
000001e0: 00 20 00 00 dc 00 00 00 00 00 00 00 00 00 00 00 00 .....
000001f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000200: 2e 74 65 78 74 00 00 00 ad 0c 00 00 00 10 00 00 .text.....
00000210: 00 0e 00 00 00 04 00 00 00 00 00 00 00 00 00 00 .....
00000220: 00 00 00 00 20 00 00 60 2e 72 64 61 74 61 00 00 .... ..`rdata..
00000230: 6e 0b 00 00 00 20 00 00 00 0c 00 00 00 12 00 00 n....
00000240: 00 00 00 00 00 00 00 00 00 00 00 00 40 00 00 40 .....@..@
00000250: 2e 64 61 74 61 00 00 00 84 03 00 00 00 30 00 00 .data.....0..
00000260: 00 02 00 00 00 1e 00 00 00 00 00 00 00 00 00 00 .....
00000270: 00 00 00 00 40 00 00 c0 2e 72 73 72 63 00 00 00 ....@....rsrc...
00000280: 50 52 00 00 00 40 00 00 00 54 00 00 00 20 00 00 PR...@...T... ..
00000290: 00 00 00 00 00 00 00 00 00 00 00 00 40 00 00 40 .....@..@
000002a0: 2e 72 65 6c 6f 63 00 00 58 01 00 00 00 a0 00 00 .reloc..X.....
000002b0: 00 02 00 00 00 74 00 00 00 00 00 00 00 00 00 00 .....t.....
000002c0: 00 00 00 00 40 00 00 42 00 00 00 00 00 00 00 00 ....@..B.....
000002d0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000002e0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000002f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000300: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000310: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000320: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000330: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000340: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000350: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
--More--

```

Estructura en este ejemplo de malware:

```

Offset 0x000: 4D 5A                -> firma MZ
Offset 0x03C: 08 01 00 00          -> e_lfanew = 0x108
Offset 0x0E8: 52 69 63 68          -> marcador Rich
Offset 0x108: 50 45 00 00          -> firma PE\0\0

```

```
Offset 0x10C: 4C 01          -> Machine = IMAGE_FILE_MACHINE_I386
Offset 0x10E: 05 00          -> cinco secciones
Offset 0x200: 2E 74 65 78 74 00 00 00 -> nombre de sección .text
```

Estructura principal observada en la muestra:

```
Offset 0x00000000 -> DOS Header
Offset 0x00000040 -> DOS Stub
Offset 0x00000080 -> Rich Header
Offset 0x00000108 -> PE Signature: PE\0\0
Offset 0x0000010C -> IMAGE_FILE_HEADER
Offset 0x00000120 -> IMAGE_OPTIONAL_HEADER
Offset 0x00000180 -> Data Directories
Offset 0x00000200 -> Section Table
Offset 0x00000400 -> Primera sección en disco: .text
```

Destacamos `e_lfanew = 0x108`:

Por eso la firma `PE\0\0` no aparece exactamente en `0x100`, sino en `0x108`:

```
00000100: 00 00 00 00 00 00 00 00 50 45 00 00 4c 01 05 00
                        ^^ ^^ ^^ ^^
                        P  E 00 00
```

DOS Header:

El archivo empieza con:

```
00000000: 4d 5a 90 00 ...
```

Los dos primeros bytes son:

```
4D 5A
```

En ASCII:

```
MZ
```

Esto identifica una cabecera `DOS` válida, necesaria en los ejecutables `PE`.

El byte siguiente `90` no forma parte de la firma. Pertenece al campo siguiente del `IMAGE_DOS_HEADER`.

Campos principales del DOS Header: Interpretando los primeros bytes en little endian:

Offset	Bytes	Campo	Valor	Significado
0x00	4D 5A	e_magic	MZ	Firma DOS
0x02	90 00	e_cblp	0x0090	Bytes en la última página DOS
0x04	03 00	e_cp	0x0003	Número de páginas DOS
0x06	00 00	e_crlc	0	Relocations DOS
0x08	04 00	e_cparhdr	0x0004	Tamaño del header DOS en párrafos
0x0A	00 00	e_minalloc	0	Memoria mínima
0x0C	FF FF	e_maxalloc	0xFFFF	Memoria máxima
0x0E	00 00	e_ss	0	Stack segment DOS
0x10	B8 00	e_sp	0x00B8	Stack pointer DOS
0x18	40 00	e_lfarlc	0x0040	Offset tabla relocations DOS
0x3C	08 01 00 00	e_lfanew	0x00000108	Offset del PE Header

El campo clave es:

```
00000030: ... 08 01 00 00
```

Eso está en el **offset 0x3C**.

Como es **little endian**:

```
08 01 00 00 = 0x00000108
```

Por tanto:

```
e_lfanew = 0x108
```

Esto significa: La **cabecera PE** real empieza en el offset **0x108** del archivo.

DOS Stub:

A partir del offset **0x40** aparece el código **DOS Stub**:

```
00000040: 0e 1f ba 0e 00 b4 09 cd 21 b8 01 4c cd 21 54 68
00000050: 69 73 20 70 72 6f 67 72 61 6d 20 63 61 6e 6e 6f
```

```
00000060: 74 20 62 65 20 72 75 6e 20 69 6e 20 44 4f 53 20
00000070: 6d 6f 64 65 2e 0d 0d 0a 24
```

La cadena ASCII es:

```
This program cannot be run in DOS mode.
```

Termina con:

```
0D 0D 0A 24
```

Donde:

```
0D = carriage return
0A = line feed
24 = '$'
```

El carácter \$ es importante porque la función DOS `int 21h / AH=09h` imprime una cadena hasta encontrar \$.

El código ensamblador del `DOS Stub` imprime: `This program cannot be run in DOS mode..` En Windows moderno, este `stub` normalmente no se ejecuta. El `loader` de Windows lee `e_lfanew`, salta a `0x108` y procesa el `PE Header`.

Rich Header:

Entre el `DOS Stub` y la `cabecera PE` aparece un `Rich Header`:

```
00000080: 1b 51 88 7b 5f 30 e6 28 ...
...
000000e0: ad 69 e4 29 5e 30 e6 28 52 69 63 68 5f 30 e6 28
                        R i c h
```

La marca visible es:

```
52 69 63 68 = Rich
```

Justo después aparece la clave XOR:

```
5F 30 E6 28
```

En little endian:

```
0x28E6305F
```

El **Rich Header** es una estructura añadida por herramientas de Microsoft, normalmente **Visual Studio / MSVC**. No forma parte estricta de la especificación **PE** oficial, pero es muy común.

Sirve para dejar metadatos del proceso de compilación, como versiones de compilador, **linker** y objetos usados. En malware puede ser útil para **fingerprinting**, pero también puede ser manipulado. Debe tenerse en cuenta que el **Rich Header** puede ser eliminado o manipulado, por lo que esta información debe usarse como indicador auxiliar y no como prueba concluyente de atribución.

El **Rich Header** es:

- Una zona no oficial del **PE**, generada normalmente por herramientas Microsoft.
- Está situada entre el **DOS Stub** y el **PE Header**.
- Cifrada/ofuscada con **XOR**.
- Marcada por **DanS** al inicio descifrado y **Rich** al final.
- Contiene información sobre compilador, **linker**, versiones y componentes usados.

En el análisis de malware puede ayudar a:

- formular hipótesis sobre la cadena de herramientas;
- comparar muestras;
- detectar posibles manipulaciones;
- apoyar, nunca demostrar por sí solo, una atribución técnica;
- diferenciar código del runtime y lógica propia de la muestra.

PE Signature:

En el offset indicado por **e_lfanew**, **0x108**, aparece:

```
50 45 00 00
```

En ASCII:

```
PE\0\0
```

Lo vemos aquí:

```
00000100: 00 00 00 00 00 00 00 00 50 45 00 00 4c 01 05 00
                    50 45 00 00
```

Por tanto:

```
Offset 0x108 -> PE Signature
Offset 0x10C -> IMAGE_FILE_HEADER
```

IMAGE_FILE_HEADER:

Después de `PE\0\0` viene el `IMAGE_FILE_HEADER`. Empieza en:

```
0x10C
```

Los bytes relevantes son:

```
00000108: 50 45 00 00 4c 01 05 00 c0 b5 fb 55 00 00 00 00
00000118: 00 00 00 00 e0 00 02 01
```

Offset	Bytes	Campo	Valor	Significado
0x108	50 45 00 00	Signature	PE\0\0	Firma PE
0x10C	4C 01	Machine	0x014C	Intel x86, 32 bits
0x10E	05 00	NumberOfSections	5	Tiene 5 secciones
0x110	C0 B5 FB 55	TimeStamp	0x55FBB5C0	Fecha de compilación
0x114	00 00 00 00	PointerToSymbolTable	0	Sin tabla COFF
0x118	00 00 00 00	NumberOfSymbols	0	Sin símbolos COFF
0x11C	E0 00	SizeOfOptionalHeader	0xE0	Tamaño normal para PE32
0x11E	02 01	Characteristics	0x0102	Ejecutable + 32 bits

IMAGE_OPTIONAL_HEADER:

El `Optional Header` empieza en: `0x120`. Aunque se llame “optional”, en un **PE ejecutable normal es obligatorio**. Fragmento:

```
00000120: 0b 01 0e 00 00 0e 00 00 00 66 00 00 00 00 00 00
00000130: 0d 13 00 00 00 10 00 00 00 20 00 00 00 00 40 00
00000140: 00 10 00 00 00 02 00 00 06 00 00 00 00 00 00 00
```

```
00000150: 06 00 00 00 00 00 00 00 00 b0 00 00 00 04 00 00
00000160: 00 00 00 00 03 00 40 81 00 00 10 00 00 10 00 00
```

Campo	Valor	Significado
Magic	0x010B	PE32
MajorLinkerVersion	14	Linker 14.x, compatible con Visual Studio 2015
SizeOfCode	0xE00	Tamaño de código
SizeOfInitializedData	0x6600	Datos inicializados
SizeOfUninitializedData	0	Sin bloque declarado de datos no inicializados
AddressOfEntryPoint	0x130D	RVA del punto de entrada
BaseOfCode	0x1000	RVA donde empieza código
BaseOfData	0x2000	RVA donde empiezan datos
ImageBase	0x00400000	Base preferida en memoria
SectionAlignment	0x1000	Alineación en memoria
FileAlignment	0x200	Alineación en disco
SizeOfImage	0xB000	Tamaño total en memoria
SizeOfHeaders	0x400	Tamaño de cabeceras en disco
Subsystem	3	Windows Console
DllCharacteristics	0x8140	Mitigaciones/flags
NumberOfRvaAndSizes	16	16 Data Directories

Entry Point:

Uno de los primeros campos útiles para orientar el análisis dinámico es:

```
AddressOfEntryPoint = 0x130D
```

Esto es una **RVA**, no una dirección absoluta. Como:

```
ImageBase = 0x00400000
```

Si la imagen se cargara en su base preferida, la VA preferida del punto de entrada sería:

```
VA = ImageBase + RVA
VA = 0x00400000 + 0x0000130D
```

```
VA = 0x0040130D
```

Por tanto:

```
VA preferida del punto de entrada = 0x0040130D
```

Como el binario declara compatibilidad con ASLR, la dirección observada durante la ejecución puede ser distinta. La dirección real debe calcularse así:

```
VA real = MappedBase + 0x130D
```

Ahora hay que ver en qué sección cae la **RVA 0x130D**.

La sección **.text** tiene:

```
VirtualAddress = 0x1000  
VirtualSize    = 0x0CAD
```

El rango virtual de **.text**, expresado con límite superior exclusivo, es:

```
.text: [0x1000, 0x1CAD)
```

Como:

```
0x130D está dentro de [0x1000, 0x1CAD)
```

el punto de entrada cae dentro de:

```
.text
```

Esta ubicación es coherente con un ejecutable convencional: el punto de entrada se encuentra en una sección de código ejecutable.

Conversión del Entry Point a offset de archivo: Para convertir **RVA** a **Raw Offset** (desplazamiento físico):

```
RawOffset = RVA - VirtualAddress_de_la_sección + PointerToRawData
```

Para `.text`:

```
RVA EntryPoint      = 0x130D
.text VA            = 0x1000
.text Raw Pointer   = 0x0400
```

Cálculo:

```
RawOffset = 0x130D - 0x1000 + 0x0400
RawOffset = 0x070D
```

Así que el código inicial del programa se encuentra en:

```
VA preferida: 0x0040130D
RVA:          0x0000130D
Raw offset:   0x0000070D
```

El código situado en `.text` a partir del punto de entrada es:

```
└─$ xxd -g 1 -s 0x70d -l 0x80 Lab_03-1.malware | more
0000070d: e8 93 03 00 00 e9 7a fe ff ff 55 8b ec 6a 00 ff  ....z...U..j..
0000071d: 15 44 20 40 00 ff 75 08 ff 15 48 20 40 00 68 09  .D @..u...H @.h.
0000072d: 04 00 c0 ff 15 40 20 40 00 50 ff 15 3c 20 40 00  ....@ @.P..< @.
0000073d: 5d c3 55 8b ec 81 ec 24 03 00 00 6a 17 e8 55 09  ].U...$.j..U.
0000074d: 00 00 85 c0 74 05 6a 02 59 cd 29 a3 18 31 40 00  ....t.j.Y.)..1@.
0000075d: 89 0d 14 31 40 00 89 15 10 31 40 00 89 1d 0c 31  ...1@....1@....1
0000076d: 40 00 89 35 08 31 40 00 89 3d 04 31 40 00 66 8c  @..5.1@..=.1@.f.
0000077d: 15 30 31 40 00 66 8c 0d 24 31 40 00 66 8c 1d 00  .01@.f..$1@.f...
```

Si analizamos el código ensamblador que se observa, obtenemos que:

```
0040130D  E8 93 03 00 00      call 004016A5
00401312  E9 7A FE FF FF      jmp  00401191
...
...
```

- El punto de entrada contiene una llamada a `0x004016A5` seguida de un salto a `0x00401191`.
- La función llamada podría pertenecer a una rutina de inicialización, pero esta interpretación debe confirmarse mediante el desensamblado de la función, sus referencias cruzadas y el flujo posterior.
- Con este fragmento aislado no puede concluirse todavía dónde comienza la lógica maliciosa principal.

Subsystem:

El campo:

```
Subsystem = 0x0003
```

significa:

```
Windows CUI / Console
```

Es decir, está marcado como programa de consola. No significa necesariamente que siempre imprima algo por pantalla. Sólo indica cómo lo tratará Windows al cargarlo.

DllCharacteristics:

El binario declara compatibilidad con:

```
ASLR
DEP / NX
Terminal Server Aware
```

Data Directories:

Directory	RVA	Size	Sección	Raw offset aproximado
Export Table	0x00000000	0x00000000	—	—
Import Table	0x00002574	0x000000A0	.rdata	0x1774
Resource Table	0x00004000	0x00005250	.rsrc	0x2000
Exception Table	0x00000000	0x00000000	—	—
Certificate Table	0x00000000	0x00000000	—	—
Base Relocation Table	0x0000A000	0x00000158	.reloc	0x7400
Debug Directory	0x00002190	0x00000070	.rdata	0x1390
Architecture	0x00000000	0x00000000	—	—
Global Ptr	0x00000000	0x00000000	—	—
TLS Table	0x00000000	0x00000000	—	—
Load Config Table	0x00002200	0x00000040	.rdata	0x1400
Bound Import	0x00000000	0x00000000	—	—
IAT	0x00002000	0x000000DC	.rdata	0x1200
Delay Import	0x00000000	0x00000000	—	—

Directory	RVA	Size	Sección	Raw offset aproximado
CLR Header	0x00000000	0x00000000	—	—

Import Table:

```
Import Table RVA = 0x2574
Import Table Size = 0xA0
```

Cae dentro de **.rdata**.

Conversión a **raw offset**:

```
.rdata VirtualAddress = 0x2000
.rdata PointerToRawData = 0x1200

RawOffset = 0x2574 - 0x2000 + 0x1200
RawOffset = 0x1774
```

Para inspeccionar los descriptores de **imports**:

```
xxd -g 1 -s 0x1774 -l 0xa0 Lab_03-1.malware
```

Esto es importante para ver qué DLLs y APIs importa.

Resource Table:

```
Resource Table RVA = 0x4000
Resource Table Size = 0x5250
```

Cae dentro de:

```
.rsrc
```

Raw offset:

```
RawOffset = 0x4000 - 0x4000 + 0x2000
RawOffset = 0x2000
```

La sección de recursos es relativamente grande:

```
VirtualSize = 0x5250  
RawSize     = 0x5400
```

En análisis de malware conviene revisarla porque puede contener:

```
iconos  
version info  
manifiestos  
configuración  
payloads embebidos  
datos cifrados o comprimidos
```

TLS Table:

```
TLS Table RVA = 0  
TLS Table Size = 0
```

Esto indica que, según la **cabecera PE**, no hay **TLS callbacks** declarados.

Conclusión: No hay indicio de ejecución previa al **Entry Point** mediante **TLS callbacks**.

CLR Header:

```
CLR Header = 0
```

Por tanto: No parece un binario **.NET**. Es código nativo **PE32**.

Section Table:

La tabla de secciones empieza en: **0x200**.

Sección .text:

Campo	Valor
Name	.text
VirtualSize	0x0CAD
VirtualAddress	0x1000
SizeOfRawData	0x0E00
PointerToRawData	0x0400
Characteristics	0x60000020

Campo	Valor
Permisos	Read + Execute

El **Entry Point 0x130D** cae dentro de esta sección.

Sección **.rdata**:

Campo	Valor
Name	.rdata
VirtualSize	0x0B6E
VirtualAddress	0x2000
SizeOfRawData	0x0C00
PointerToRawData	0x1200
Characteristics	0x40000040
Permisos	Read

.rdata suele contener:

- strings de solo lectura
- punteros a imports
- tablas constantes
- estructuras de runtime
- metadatos

Sección **.data**:

Campo	Valor
Name	.data
VirtualSize	0x0384
VirtualAddress	0x3000
SizeOfRawData	0x0200
PointerToRawData	0x1E00
Characteristics	0xC0000040
Permisos	Read + Write

Sección **.rsrc**:

Campo	Valor
Name	.rsrc

Campo	Valor
VirtualSize	0x5250
VirtualAddress	0x4000
SizeOfRawData	0x5400
PointerToRawData	0x2000
Characteristics	0x40000040
Permisos	Read

Contiene los recursos del ejecutable:

- iconos
- version info
- manifiesto
- diálogos
- strings
- posibles datos embebidos

Sección .reloc:

Campo	Valor
Name	.reloc
VirtualSize	0x0158
VirtualAddress	0xA000
SizeOfRawData	0x0200
PointerToRawData	0x7400
Characteristics	0x42000040
Permisos	Read + Discardable

El loader puede cargar el binario en una dirección distinta a 0x00400000 y corregir direcciones absolutas usando `.reloc`.

13. EJEMPLO 2 : Malware que tiene una dll incrustada en la sección .rsrc

En este laboratorio se observa cómo el archivo `Lab_03-1.malware` contiene una DLL incrustada como recurso en la sección `.rsrc`. El procedimiento de extracción y análisis se documenta en [RC_DATA-IDR_DLL1.md](#).

La presencia de una DLL embebida demuestra que el ejecutable transporta otro módulo, pero no permite concluir por sí sola que vaya a utilizarlo para PE Injection. Para determinar su finalidad hay que localizar la rutina que accede al recurso, comprobar si lo descifra o descomprime y seguir posteriormente su escritura, carga o mapeo.

The first screenshot shows the 'Section Headers' view in PESTudio. The left sidebar lists various file properties, with 'sections (file > executable)' highlighted. The main pane displays a table of section headers with columns for property, value, and name. The table lists sections like .text, .data, .rsrc, and .reloc, along with their addresses, sizes, and characteristics.

property	value	value	value	value	value
section	section[0]	section[1]	section[2]	section[3]	section[4]
name	.text	.data	.data	.rsrc	.reloc
sha256	09B08612A7F13D45966DBC6...	370C35AD66488E14EE4C760...	001CC148D185D7D29246EFF...	6DE3425063A83E93E50C4F6...	B0785D0FCDF99598711
entropy	5.933	4.680	0.280	5.605	4.805
file > ratio (96.61%)	11.86 %	10.17 %	1.69 %	71.19 %	1.69 %
raw-address (begin)	0x00000400	0x00001200	0x00001E00	0x00002000	0x00007400
raw-address (end)	0x00001200	0x00001E00	0x00002000	0x00007400	0x00007600
raw-size (29184 bytes)	0x00000E00 (3584 bytes)	0x00000C00 (3072 bytes)	0x00000200 (512 bytes)	0x00005400 (21504 bytes)	0x00000200 (512 bytes)
virtual-address (begin)	0x00001000	0x00002000	0x00003000	0x00004000	0x0000A000
virtual-address (end)	0x00001CAD	0x00002B6E	0x00003384	0x00009250	0x0000A158
virtual-size (28487 bytes)	0x00000CAD (3245 bytes)	0x00000B6E (2926 bytes)	0x00000384 (900 bytes)	0x00005250 (21072 bytes)	0x00000158 (344 bytes)

The second screenshot shows the 'Characteristics' view. The left sidebar lists various file properties, with 'characteristics (signature > executal)' highlighted. The main pane displays a table of characteristics with columns for property, value, and name. The table lists characteristics like write, execute, share, self-modifying, and virtual, along with their values.

property	value	value	value	value	value
characteristics	0x60000020	0x40000040	0xC0000040	0x40000040	0x42000040
write	-	-	x	-	-
execute	x	-	-	-	-
share	-	-	-	-	-
self-modifying	-	-	-	-	-
virtual	-	-	-	-	-

The third screenshot shows the 'Resources' view. The left sidebar lists various file properties, with 'resources (signature > executal)' highlighted. The main pane displays a table of resources with columns for name, instance, signature, location, size, footprint, entropy, and language. The table lists resources like manifest, RC_DATA, and IDR_DLL1, along with their details.

name	instance (2)	signature (2)	location (from-to)	size (20861 bytes)	footprint (sha256)	entropy	language (1)
manifest	1	manifest	0x000070D0 - 0x0000724D	0x0000017D (381 bytes)	4BB79DCFA0A901F7D9EAC5AA05728AE...	4.912	English-US (0x0000040)
RC_DATA	IDR_DLL1	executable (cpu: 32-bit)	0x000020D0 - 0x000070D0	0x00005000 (20480 bytes)	00ECF17B51E58264E77483B0F60F4BF4...	5.675	English-US (0x0000040)

14. Herramientas para análisis de archivos PE

14.1. pecheck.py

pecheck.py es una utilidad de Didier Stevens para inspeccionar archivos PE desde la línea de comandos. Según la versión y las opciones elegidas, permite consultar cabeceras, secciones, hashes, entropía, firmas y otros indicadores. La ayuda instalada es la referencia adecuada para comprobar la sintaxis disponible:

```
python pecheck.py --help
python pecheck.py archivo.exe
```

Sus resultados son indicios para orientar el análisis; por ejemplo, una entropía alta puede ser compatible con compresión o cifrado, pero no demuestra por sí sola que la muestra esté empaquetada.

14.2. pe-tree

pe-tree representa las estructuras PE en forma de árbol y facilita la navegación por cabeceras, secciones, directorios de datos, importaciones, exportaciones y recursos. También resalta determinados valores anómalos o de interés. Estas heurísticas deben contrastarse con el contenido del archivo y, cuando sea necesario, con otras herramientas.

14.3. peframe

PEframe automatiza parte del análisis estático y genera un resumen de metadatos e indicadores. Una instalación reproducible en un entorno virtual puede realizarse así:

```
python3 -m venv peframe-env
source peframe-env/bin/activate
python -m pip install --upgrade pip
git clone https://github.com/guelfoweb/peframe.git
cd peframe
python -m pip install .
peframe --help
peframe archivo.exe
```

En PowerShell, la activación del entorno se realiza con `.\peframe-env\Scripts\Activate.ps1`. No se debe ejecutar el archivo analizado para obtener este informe estático.

Entre los modos documentados por el proyecto se encuentran:

```
peframe -i archivo.exe      # modo interactivo
peframe -j archivo.exe      # informe completo en JSON
peframe -x CADENA archivo.exe # buscar una cadena codificada mediante XOR
peframe -s archivo.exe      # mostrar cadenas
```

14.4. radare2

[radare2](#) es un entorno de código abierto para análisis e ingeniería inversa creado por Sergi Alvarez, conocido como *pancake*. [Cutter](#) proporciona una interfaz gráfica basada en Rizin. El [libro oficial de radare2](#) documenta los comandos y los cambios entre versiones.

El modificador `-A` solicita análisis automático al abrir el archivo; no hace falta ejecutar `aaa` inmediatamente después como paso obligatorio:

```
r2 -A archivo.exe
```

- Radare es una herramientas opensource para realizar tareas de ingeniería inversa. Su creador es Sergi Álvarez ([pancake](#)).
- Cuenta con una interfaz gráfica llamada [Cutter](#).
- Libro de radare: <https://book.rada.re/>
- Para abrir un fichero en radare debemos pasar la ruta del fichero como primer parámetro.

```
sudo apt install radare2
r2 -A archivo.exe
a?                # Ayuda para los comandos de análisis de binarios
aaa              # Carga toda la información del binario
aaaa            # Información de cadenas, llamadas (referencias cruzadas),
listas de funciones, etc.
ae              # Está relacionado con la emulación ESIL
af             # Define función en dirección actual
afl            # Lista de funciones encontradas
afl~main       # Busca funciones que contengan "main"
axt            # Referencias a una dirección
axtj           # Referencias (en JSON)
axg            # Grafo de referencias cruzadas
dcu            # Continuar la depuración hasta una dirección
i              # Muestra información básica sobre el binario, como
protecciones activadas durante la compilación, fecha de compilación, tipo
de binario, Sistema Operativo...
iI             # Info del formato PE (arch, machine, tamaño)
iD             # Directivas del PE como imports/exports
iS             # Muestra todas las secciones (.text, .data, etc.)
iS~text        # Filtra por sección de código
iH             # Cabecera del binario
ii             # Imports
iij            # Muestra imports pero en formato JSON (útil para scripts)
ii~VirtualAlloc # Busca funciones específicas, por ejemplo VirtualAlloc
ii~kernel32    # Ver funciones importadas de kernel32.dll
ii~WriteProcessMemory
ii~CreateRemoteThread
ii~LoadLibrary
ii~GetProcAddress
ii~WinExec
iE             # Exported symbols (símbolos exportados)
ie             # muestra entrypoints del binario
iz             # Strings encontradas (utf-8, ascii)
```

```

iz~=[A-Za-z0-9]{6,} # Para buscar cadenas raras que podrían estar cifradas
iz~xor              # Busca strings y llamadas relacionadas con cifrado
iz~decrypt          # Busca strings y llamadas relacionadas con cifrado
iz~key              # Busca strings y llamadas relacionadas con cifrado
ii~RtlDecompress    # Busca strings y llamadas relacionadas con cifrado
iz                  # Strings del binario
iz~http             # Filtrar strings que contengan "http"
/                  # Comando para buscar cadenas o patrones de bytes.
/?                 # Lista los modificadores para utilizar con el comando /
de búsqueda
/x                  # Comando para buscar cadenas de bytes
/x 90 90           # Busca la secuencia 0x90 0x90
/m                 # Busca cabeceras "mágicas" (firmas de tipos de archivo)
usando libmagic. Sirve para detectar archivos o estructuras embebidas
dentro de un binario
/ WinExec          # Buscador de patrones
/ http             # Buscar la palabra "http"
/ bin.sh           # Buscar posibles llamadas a shells
/x 90              # Buscar NOPs (hexadecimal)
/C                 # Búsqueda de material criptográfico.
/R                 # Busca gadgets ROP en el rango de búsqueda actual.
/V                 #
pdf                # Imprime el código desensamblado correspondiente a la
función que nos encontremos actualmente.
pdf                # p: print - d: desensamblado - f: función □ Print
disassembly function
pd 5               # print disasm 5 □ imprime las 5 instrucciones de
ensamblador siguientes desde la dirección actual (el offset donde estás
posicionado)
pd?                # muestra la ayuda de pd en radare2, una descripción del
comando, su sintaxis y las opciones/variantes disponibles
p?                # Muestra la ayuda del comando y sus posibles
modificadores
px                 # p: print - x: hexadecimal □ imprime en hexadecimal
q                 # Salir
s main            # Comando seek que permite movernos al lugar indicado como
parámetro. En este caso a la función main
s 0x00401574       # Para movernos a una dirección de memoria concreta
V                 # Entra en modo visual (desensamblado)
                  h j k l → moverte (como en vim): izquierda, abajo, arriba,
derecha
                  p → cambia el panel de vista (hexdump, ensamblador, gráfico,
etc.)
                  V → alterna entre los distintos modos visuales.
                  q → salir al modo normal.
                  ? → muestra ayuda dentro del modo visual
....
r2 -d archivo.exe □ Depura el ejecutable con radare
d?                # Lista de comandos de debugger
db                # Permite añadir y quitar breakpoints en las direcciones
de memoria deseadas.

```

ae pertenece a la familia de emulación ESIL; no significa «punto de entrada analizado». En modo depuración, **dcu dirección** continúa la ejecución hasta una dirección, no desensambla hasta una llamada.

El comando **r2 -d archivo.exe** intenta iniciar el archivo mediante el *backend* de depuración disponible. Un sistema Linux no puede ejecutar de forma nativa un PE de Windows: para depurarlo hace falta un entorno compatible correctamente configurado —por ejemplo, Windows en una máquina virtual— y deben mantenerse las medidas de aislamiento.

14.5. Otras utilidades

Herramientas de uso habitual:

- **PE-bear**: visor y editor de estructuras PE.
- **Detect It Easy (DIE)**: identificación de tipos de archivo, compiladores, protectores y empaquetadores mediante firmas y heurísticas.
- **PEstudio**: vista estática con indicadores y elementos destacados.
- **Explorer Suite/CFF Explorer**: inspección y edición de estructuras PE.
- **Ghidra**: desensamblado y descompilación multiplataforma.
- **x64dbg**: depuración de aplicaciones x86 y x64 en Windows.
- **pefile**, **LIEF** y **objdump**: automatización y consulta mediante scripts o línea de comandos.

Proyectos como PEBrowse Professional, PPEE y **pescanner.py** aparecen en materiales históricos, pero pueden llevar años sin mantenimiento o depender de enlaces antiguos. Si se emplean para reproducir un laboratorio, conviene verificar primero su procedencia, versión y compatibilidad, y contrastar sus resultados con una herramienta mantenida.

14.6. Interpretación de mitigaciones

Algunas utilidades resumen las mitigaciones declaradas por el PE. La terminología concreta depende de la herramienta y no siempre representa una comprobación exhaustiva:



Indicador	Interpretación prudente
ASLR	DYNAMIC_BASE indica que la imagen admite reubicación aleatoria. La eficacia también depende de que existan relocalaciones utilizables y de la plataforma.

Indicador	Interpretación prudente
DEP/NX	NX_COMPAT declara compatibilidad con páginas de datos no ejecutables. Su ausencia no causa un desbordamiento de búfer; puede reducir una barrera frente a la ejecución del contenido inyectado después de un fallo.
CFG	Control Flow Guard valida determinados destinos de llamadas indirectas. No impide por sí solo todas las técnicas ROP ni todos los secuestros de flujo.
Stack cookies	Indica el uso detectado de canarios de pila, pero no garantiza que todas las funciones estén instrumentadas.
SEH o SafeSEH	El uso de SEH no es por sí mismo una mitigación. SafeSEH es una propiedad distinta, aplicable a determinadas imágenes x86 y registrada en la configuración de carga.

La ausencia de uno o varios indicadores significa que el binario declara menos mitigaciones en esas categorías; no basta para demostrar que sea explotable. Esa conclusión requiere identificar un defecto concreto y evaluar el entorno de ejecución y las defensas del sistema.

14.7. Recursos PE

Los iconos, menús, diálogos, cadenas y datos específicos de la aplicación suelen almacenarse mediante el directorio de recursos, normalmente dentro de **.rsrc**. Una muestra también puede ocultar allí binarios, documentos señuelo o configuración. Los metadatos de versión sirven como pista, pero pueden estar ausentes o haber sido manipulados y no deben atribuirse automáticamente a su supuesto autor.

15. Conversión de RVA a File Offset en archivos PE

La conversión de una **RVA (Relative Virtual Address)** a un **File Offset** —también denominado *Raw Offset* en algunas herramientas— permite correlacionar una posición de la imagen PE en memoria con la ubicación física de los bytes correspondientes dentro del archivo almacenado en disco.

Esta conversión es útil para localizar, entre otros elementos:

- el código situado en el punto de entrada;
- tablas de importaciones y exportaciones;
- recursos;
- relocalaciones de base;
- estructuras TLS;
- datos referenciados mediante una RVA.

[!IMPORTANT] La fórmula de conversión solo es válida cuando la RVA corresponde a bytes realmente almacenados en una sección del archivo. Algunas regiones existen únicamente en memoria y no tienen un File Offset equivalente.

15.1. Conceptos necesarios

Concepto	Descripción
RVA	Desplazamiento relativo respecto a la base donde está mapeada la imagen.
VA	Dirección virtual efectiva dentro del proceso.
VirtualAddress	RVA inicial de una sección.
VirtualSize	Tamaño lógico de la sección en memoria.
PointerToRawData	Offset del archivo donde comienzan los datos almacenados de la sección.
SizeOfRawData	Tamaño de los datos que la sección ocupa físicamente en el archivo.
File Offset	Posición medida desde el comienzo del archivo en disco.
MappedBase	Dirección virtual donde la imagen quedó realmente mapeada.

La relación entre VA y RVA es:

$$VA = \text{MappedBase} + RVA$$

Por tanto, si se parte de una VA observada durante la ejecución:

$$RVA = VA - \text{MappedBase}$$

No debe utilizarse automáticamente el **ImageBase** preferido para este cálculo, ya que ASLR puede haber situado la imagen en otra dirección.

15.2. Fórmula de conversión

Para una RVA asociada a bytes físicos de una sección:

$$\text{FileOffset} = \text{PointerToRawData} + (RVA - \text{VirtualAddress})$$

El término situado entre paréntesis representa el desplazamiento relativo dentro de la sección:

$$\Delta_{\text{sección}} = RVA - \text{VirtualAddress}$$

La conversión puede expresarse entonces como:

$$\text{FileOffset} = \text{PointerToRawData} + \Delta_{\text{sección}}$$

Condición imprescindible

Antes de aplicar la fórmula debe comprobarse que el desplazamiento relativo pertenece a los datos almacenados en el archivo:

$$0 \leq \Delta_{\text{sección}} < \text{SizeOfRawData}$$

Si se cumple:

$$RVA - \text{VirtualAddress} \geq \text{SizeOfRawData}$$

la RVA puede pertenecer a una zona que existe en memoria pero no contiene bytes equivalentes en el archivo. En ese caso, la fórmula produciría un offset engañoso.

15.3. Procedimiento paso a paso

Paso 1. Obtener la RVA de interés

La RVA puede proceder, por ejemplo, de:

- `AddressOfEntryPoint`;
- una entrada de `DataDirectory[]`;
- un operando o referencia observado en el código;
- una VA capturada durante el análisis dinámico, después de restar `MappedBase`.

Paso 2. Comprobar si pertenece a las cabeceras

Si la RVA es menor que `SizeOfHeaders` y esa posición existe dentro del archivo, normalmente se cumple:

$$\text{FileOffset} = \text{RVA}$$

Por ejemplo, para `SizeOfHeaders = 0x400`, la RVA `0x180` corresponde normalmente al File Offset `0x180`.

Paso 3. Localizar la sección candidata

Debe recorrerse la tabla de secciones, formada por estructuras `IMAGE_SECTION_HEADER`, y comparar la RVA con el rango de cada sección.

En una imagen convencional, el rango lógico en memoria viene descrito por `VirtualAddress` y `VirtualSize`:

$$\text{VirtualAddress} \leq \text{RVA} < \text{VirtualAddress} + \text{VirtualSize}$$

Algunos parsers utilizan el siguiente tamaño efectivo para tolerar diferencias producidas por el alineamiento y determinados casos límite:

$$\text{TamañoEfectivo} = \max(\text{VirtualSize}, \text{SizeOfRawData})$$

Esta ampliación sirve para identificar una sección candidata, pero **no elimina la obligación de comprobar `SizeOfRawData`** antes de devolver un offset físico.

Paso 4. Calcular el desplazamiento dentro de la sección

$$\Delta_{\text{sección}} = \text{RVA} - \text{VirtualAddress}$$

Paso 5. Verificar que existen bytes en disco

Debe cumplirse:

$$\Delta_{\text{sección}} < \text{SizeOfRawData}$$

Si no se cumple, la RVA no tiene una correspondencia física válida dentro de los datos de esa sección.

Paso 6. Calcular el File Offset

$$\text{FileOffset} = \text{PointerToRawData} + \Delta_{\text{sección}}$$

Paso 7. Validar los límites del archivo

El resultado debe encontrarse dentro del tamaño real del archivo. Si se pretende leer más de un byte, también debe comprobarse que todo el rango solicitado esté disponible:

$$\text{FileOffset} + \text{TamañoLectura} \leq \text{TamañoArchivo}$$

Esta validación ayuda a detectar cabeceras manipuladas, secciones truncadas y archivos PE malformados.

15.4. Ejemplos prácticos

15.4.1. Conversión general

Datos conocidos:

Campo	Valor
RVA buscada	0x1234
VirtualAddress de .text	0x1000
PointerToRawData de .text	0x0400
SizeOfRawData de .text	0x0E00

Desplazamiento dentro de la sección:

$$0x1234 - 0x1000 = 0x0234$$

Como $0x0234 < 0x0E00$, los bytes están presentes en el archivo. Por tanto:

$$\text{FileOffset} = 0x0400 + 0x0234 = \mathbf{0x0634}$$

15.4.2. Localización del punto de entrada

Datos del ejemplo:

Campo	Valor
AddressOfEntryPoint	0x130D
Sección	.text
VirtualAddress	0x1000
VirtualSize	0x0CAD
SizeOfRawData	0x0E00
PointerToRawData	0x0400

El rango virtual de `.text`, utilizando un límite superior exclusivo, es:

```
[0x1000, 0x1CAD)
```

La RVA `0x130D` pertenece a ese rango. Su desplazamiento dentro de la sección es:

$$0x130D - 0x1000 = 0x030D$$

Como `0x030D < 0x0E00`, existe contenido físico correspondiente:

$$\text{FileOffset} = 0x0400 + 0x030D = \mathbf{0x070D}$$

Los bytes asociados al punto de entrada comienzan, por tanto, en el offset `0x070D` del archivo.

Una comprobación selectiva puede realizarse con:

```
xxd -g 1 -s 0x70d -l 0x40 muestra.exe
```

15.4.3. Localización de la Import Table

Datos del ejemplo:

Campo	Valor
RVA de la Import Table	<code>0x2574</code>
Sección	<code>.rdata</code>
<code>VirtualAddress</code>	<code>0x2000</code>
<code>VirtualSize</code>	<code>0x0B6E</code>
<code>SizeOfRawData</code>	<code>0x0C00</code>
<code>PointerToRawData</code>	<code>0x1200</code>

Cálculo del desplazamiento relativo:

$$0x2574 - 0x2000 = 0x0574$$

Como `0x0574 < 0x0C00`, la tabla dispone de bytes físicos en la sección:

$$\text{FileOffset} = 0x1200 + 0x0574 = \mathbf{0x1774}$$

El comienzo de los descriptores de importación puede examinarse mediante:

```
xxd -g 1 -s 0x1774 -l 0xa0 muestra.exe
```

15.4.4. RVA sin File Offset equivalente

Considérese una sección `.data` con estos valores:

Campo	Valor
RVA buscada	0x3250
VirtualAddress	0x3000
VirtualSize	0x0384
SizeOfRawData	0x0200
PointerToRawData	0x1E00

El desplazamiento relativo es:

$$0x3250 - 0x3000 = 0x0250$$

La RVA pertenece al rango virtual porque $0x0250 < 0x0384$. Sin embargo:

$$0x0250 \geq 0x0200$$

La posición se encuentra más allá de `SizeOfRawData`. Esos bytes no proceden del archivo: el cargador inicializa esa parte de la sección a cero en memoria.

Por tanto, **la RVA 0x3250 no tiene un File Offset equivalente**. Calcular $0x1E00 + 0x0250$ produciría un resultado incorrecto que podría apuntar a otra región del archivo.

15.4.5. RVA situada en las cabeceras

Si:

```
RVA          = 0x0180
SizeOfHeaders = 0x0400
```

y el archivo contiene esa posición, la correspondencia habitual es directa:

$$\text{FileOffset} = \mathbf{0x0180}$$

15.5. Algoritmo de conversión

El siguiente código muestra la lógica de validación. Las secciones se representan como diccionarios con los campos relevantes de `IMAGE_SECTION_HEADER`:

```
def rva_to_file_offset(rva, size_of_headers, sections, file_size):
    """Devuelve el File Offset o None si la RVA no tiene bytes en disco."""

    if 0 <= rva < size_of_headers and rva < file_size:
        return rva

    for section in sorted(sections, key=lambda item:
```

```

item["VirtualAddress"]):
    virtual_address = section["VirtualAddress"]
    virtual_size = section["VirtualSize"]
    raw_size = section["SizeOfRawData"]
    raw_pointer = section["PointerToRawData"]

    effective_size = max(virtual_size, raw_size)

    if virtual_address <= rva < virtual_address + effective_size:
        relative_offset = rva - virtual_address

        if relative_offset >= raw_size:
            return None # Región inicializada a cero solo en memoria.

        file_offset = raw_pointer + relative_offset

        if file_offset >= file_size:
            return None # Archivo truncado o cabecera incoherente.

        return file_offset

return None

```

Para leer varios bytes habría que añadir el tamaño de lectura a la comprobación de límites.

15.6. Casos especiales y limitaciones

15.6.1. Datos inicializados a cero

Cuando `VirtualSize > SizeOfRawData`, la parte adicional de la sección se crea en memoria y se inicializa a cero. No existe contenido físico equivalente en el archivo.

Este comportamiento es habitual en datos no inicializados, como los asociados conceptualmente a `.bss`.

15.6.2. Certificate Table

La entrada `IMAGE_DIRECTORY_ENTRY_SECURITY`, denominada **Certificate Table**, constituye una excepción dentro de `DataDirectory[]`.

Su campo `VirtualAddress` no contiene una RVA: contiene directamente un **File Offset**. Por tanto, no debe aplicarse la fórmula `RVA → File Offset` a esta entrada.

15.6.3. Overlay

Un archivo puede contener datos añadidos después de la última sección, conocidos como *overlay*. Esos bytes poseen File Offsets, pero normalmente no tienen una RVA porque no forman parte de la imagen mapeada.

15.6.4. Secciones alineadas o malformadas

`VirtualSize` y `SizeOfRawData` no tienen por qué coincidir. Además, un PE manipulado puede presentar secciones solapadas, offsets fuera del archivo o tamaños incoherentes.

Por ello, una herramienta de análisis no debe limitarse a aplicar la fórmula: también debe validar rangos, alineamientos y límites físicos.

15.6.5. ASLR y base real de carga

ASLR no modifica la RVA interna de una estructura, pero sí puede cambiar su VA efectiva. Si el análisis comienza a partir de una VA observada en memoria, debe utilizarse la base real:

$$\text{RVA} = \text{VA observada} - \text{MappedBase}$$

Después puede aplicarse la conversión de RVA a File Offset.

15.7. Errores frecuentes

- Sumar `ImageBase` al intentar obtener un File Offset.
- Confundir `VirtualAddress` con una VA absoluta.
- Utilizar únicamente `VirtualSize` sin comprobar `SizeOfRawData`.
- Aplicar la fórmula a una región inicializada a cero.
- Tratar el campo de dirección de la Certificate Table como una RVA.
- Asumir que todos los datos del archivo tienen una representación en memoria.
- Ignorar que ASLR puede cambiar la base real de la imagen.
- No comprobar que el offset calculado esté dentro del archivo.

15.8. Resumen operativo

1. Obtener la RVA.
2. Si se parte de una VA, restar `MappedBase`.
3. Comprobar si la RVA pertenece a `SizeOfHeaders`.
4. Localizar la sección mediante `VirtualAddress` y su rango virtual.
5. Calcular: `delta = RVA - VirtualAddress`.
6. Verificar: `delta < SizeOfRawData`.
7. Calcular: `FileOffset = PointerToRawData + delta`.
8. Confirmar que el resultado está dentro del archivo.

La fórmula fundamental es sencilla, pero su uso correcto depende de distinguir la representación virtual de la imagen de los bytes que existen realmente en disco.

15.9. Referencias

- [Microsoft — PE Format](#)
- [Documentación de `pefile`](#)

16. Técnicas que manipulan archivos o imágenes PE

Las principales técnicas empleadas por malware que modifican, ocultan, cargan o manipulan archivos **PE (Portable Executable)** son las siguientes.

Nota: no todas estas técnicas alteran el archivo almacenado en disco. Algunas modifican la imagen PE una vez cargada en memoria, intervienen en su proceso de carga o afectan a las estructuras utilizadas para resolver funciones.

16.1. Alteración del archivo PE

1. PE infection

Inserta código malicioso en un ejecutable legítimo y modifica su flujo para ejecutarlo.

2. Packing

Comprime o transforma el contenido del PE y añade una rutina que lo reconstruye en memoria.

3. Cifrado de código o datos

Mantiene cifradas secciones, cadenas o configuraciones hasta el momento de utilizarlas.

4. Polimorfismo

Genera versiones diferentes del mismo malware cambiando el cifrado, las claves o determinadas instrucciones equivalentes.

5. Metamorfismo

Reescribe su propio código manteniendo el comportamiento, sin depender necesariamente de una capa de cifrado.

6. Virtualización de código

Convierte instrucciones nativas en bytecode ejecutado por una máquina virtual incluida en el PE.

7. Manipulación de cabeceras PE

Modifica campos como `e_lfanew`, `AddressOfEntryPoint`, tamaños, RVA o directorios para dificultar el análisis o alterar la carga.

8. Manipulación de secciones

Añade, elimina, renombra, solapa o cambia los permisos y tamaños de secciones como `.text`, `.data` o `.rsrc`.

9. Modificación del Entry Point

Cambia `AddressOfEntryPoint` para ejecutar primero un cargador, descifrador o bloque de código malicioso.

10. Abuso de TLS Callbacks

Ejecuta código mediante callbacks TLS antes de alcanzar el punto de entrada principal.

11. Code caves

Introduce código en espacios no utilizados dentro de una sección existente.

12. Section slack

Ocultar información en el espacio existente entre el contenido útil de una sección y su tamaño alineado en el archivo.

13. Overlay

Añadir datos después del final lógico del PE, fuera de las secciones que el cargador mapea normalmente.

14. Abuso de recursos

Almacena ejecutables, shellcode, configuraciones o contenido cifrado dentro de la sección `.rsrc`.

15. Manipulación de reubicaciones

Modificar o utilizar la tabla `.reloc` para alterar cómo se ajustan las direcciones cuando ASLR cambia la base de carga.

16. Abuso de la Certificate Table

Ocultar datos en el área de certificados o aprovechar particularidades del cálculo de la firma Authenticode.

17. PE políglota

Construye un archivo que puede interpretarse simultáneamente como un PE y como otro formato.

18. Suplantación de metadatos

Modificar marcas temporales, `Checksum`, Rich Header, información de versión, iconos o datos de compañía para aparentar legitimidad.

16.2. Manipulación de importaciones y exportaciones

19. Reescritura de la Import Table

Cambia descriptores, nombres de DLL, nombres de funciones, ordinales o thunks en el archivo PE.

20. IAT hooking

Sustituye en memoria una dirección resuelta de la Import Address Table para redirigir las llamadas de un módulo.

21. Delay-IAT hooking

Manipula las entradas utilizadas por las importaciones configuradas con carga retardada.

22. Manipulación de bound imports

Altera marcas temporales y direcciones previamente enlazadas, aunque resulta una técnica frágil en sistemas modernos.

23. EAT hooking

Modificar la Export Address Table de una DLL para que una exportación apunte a otra dirección.

24. Resolución dinámica de API

Obtiene funciones durante la ejecución mediante mecanismos como `LoadLibrary` y `GetProcAddress`, evitando importaciones estáticas visibles.

25. API hashing

Sustituye los nombres de las API por hashes y recorre la EAT para localizar las funciones

correspondientes.

26. **Cifrado de nombres de API**

Mantiene cifrados los nombres de DLL y funciones y los descifra únicamente cuando necesita resolverlos.

27. **DLL proxy y export forwarding**

Utiliza una DLL intermediaria que exporta las funciones esperadas y reenvía algunas llamadas mientras ejecuta código adicional.

16.3. Manipulación durante la carga o en memoria

28. **DLL side-loading**

Consigue que una aplicación legítima cargue una DLL maliciosa debido al orden de búsqueda o a una dependencia manipulada.

29. **Manual mapping**

Mapea manualmente un PE en memoria y procesa sus secciones, importaciones y reubicaciones sin seguir el flujo normal de `LoadLibrary`.

30. **Reflective DLL loading**

Una DLL contiene una rutina capaz de localizarse y cargarse a sí misma directamente desde memoria.

31. **PE injection**

Copia una imagen PE dentro de otro proceso y prepara su ejecución en ese espacio de direcciones.

32. **Process hollowing**

Sustituye la imagen de un proceso legítimo suspendido por otro PE antes de reanudarlo.

33. **Module stomping**

Sobrescribe en memoria el contenido de una DLL legítima ya cargada, conservando aparentemente sus metadatos de módulo.

34. **Process Doppelgänger**

Utiliza transacciones del sistema de archivos y secciones de imagen para ejecutar un PE sin dejar el archivo convencional esperado.

35. **Process Herpaderping**

Modifica el archivo de respaldo después de crear su sección ejecutable, provocando diferencias entre el archivo inspeccionado y la imagen ejecutada.

36. **Process Ghosting**

Ejecuta una imagen creada desde un archivo marcado para eliminación.

37. **Desempaquetado en memoria**

Reconstruye el PE original durante la ejecución, dejando en disco únicamente su versión empaquetada.

38. **Código automodificable**

Modifica sus propias instrucciones o datos mientras se está ejecutando.

39. Patching de módulos cargados

Sobrescribe instrucciones o estructuras de DLL ya mapeadas para cambiar su comportamiento.

40. Unhooking de DLL del sistema

Restaura código limpio —habitualmente de `ntdll.dll`— para retirar hooks instalados por herramientas de monitorización.

16.4. Distinción fundamental

Técnica	Elemento afectado	Diferencia principal
Reescritura de la Import Table	Descriptores, nombres y thunks del archivo PE	Modifica las estructuras que indican qué DLL y funciones deben importarse.
IAT hooking	Punteros resueltos de la IAT en memoria	Redirige llamadas realizadas mediante una entrada de importación concreta.
EAT hooking	Export Address Table del módulo exportador	Cambia la dirección publicada por una función exportada.
Inline hooking	Instrucciones de la función objetivo	Modifica directamente el código ejecutable para desviar su flujo.
Resolución dinámica	Variables o estructuras privadas creadas en ejecución	Obtiene direcciones de funciones sin depender necesariamente de la IAT estática.

16.5. Clasificación resumida

- **Técnicas estáticas:** modifican el archivo PE almacenado en disco.
- **Técnicas de carga:** intervienen en cómo Windows localiza, valida o mapea el ejecutable.
- **Técnicas en memoria:** modifican la imagen ya cargada o introducen un PE en otro proceso.
- **Técnicas de ocultación:** dificultan reconocer el código, las API utilizadas o el comportamiento real del ejecutable.

17. Laboratorios progresivos

El tutorial se puede organizar en cuatro prácticas:

LAB 1
PE Hello World

¿Qué es el PE?

ImageBase
Entry Point
Sections
Imports

↓

LAB 2

PE Mapping local

SizeOfImage
VirtualAlloc
Headers
Sections
RVA -> memoria

↓

LAB 3

PE Loader educativo

Relocations
Imports / IAT
Protecciones
Entry Point

↓

LAB 4

Análisis de malware

Buscar patrones reales
Comparar APIs
Examinar memoria
Determinar la técnica